Part II

Sensors for Localization and Navigation

Simultaneous Localization and Mapping (SLAM)

How to Build a Vision-Based Navigation and Localization Drone



Editor: Lee Shihyuan (李世淵)

LINE OA @prof

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Part II Sensors for Localization and Navigation

Chapter 4

Indoor Positioning Technologies

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Part I has settled the SLAM problem, how the subject is taught around the world, and the overall blueprint of a navigation system. From this chapter we enter Part II and examine the sensors on a robot one by one—what question each can answer, and under what conditions each fails.

This chapter opens Part II, and its subject is indoor positioning. It has a crucial division of labor with the chapters that follow: satellite navigation in Chapter 5 fails indoors; the geomagnetic field in Chapter 6 and the IMU in Chapter 7 supply direction but not absolute position; and the LiDAR of Chapter 8 and the camera of Chapter 9 need a map before they can localize. The technologies treated here—Wi-Fi, Bluetooth, UWB, RFID, and visual markers—are the only class that gives absolute coordinates indoors directly.

This matters greatly for SLAM. Chapter 1 noted SLAM's two great difficulties: accumulated drift and loop closure detection. The technologies of this chapter supply, from outside, a drift-free absolute reference that becomes an additional constraint edge in the graph optimization of Chapter 12. In other words, this chapter is not a substitute for SLAM but an anchor for it.

Figure 4-0 How Chapter 4 Connects to the Rest of the Book

[Building on the previous chapters]

The system blueprint of Chapter 3 told us that localization is one module of navigation; this chapter supplies the solution for indoor settings.

What does the "absolute localization" of Section 3.3 rely on indoors? This chapter is the

answer.

[This chapter is responsible for]

Radio ranging (Wi-Fi / BLE / UWB) + landmark identification (RFID / QR / AprilTag) = absolute indoor position

[Where it leads]

→ Chapter 5 The outdoor counterpart: GNSS (complementary to this chapter, indoors and out)

→ Chapters 6 and 7 Direction sensors: geomagnetic field and IMU (supplying the heading and the high update rate this chapter lacks)

→ Chapters 9 and 21 The camera intrinsics and PnP mathematics behind AprilTag

→ Chapter 12 Writing the absolute observations of this chapter as a constraint edge in graph optimization

→ Chapters 24 and 29 Integrating AprilTag and Wi-Fi on a real robot

Learning Objectives

From this chapter we enter Part II, "Sensors for Localization and Navigation," and take apart the robot's various senses one at a time. The first stop is indoor positioning—the localization technology closest to your daily life, and the best starting point for understanding how absolute localization and SLAM complement each other. After completing this chapter, you should be able to:

1. Explain why the GPS signal fails indoors, and describe the physical causes of signal attenuation and multipath.
2. Describe the two-stage workflow of Wi-Fi RSSI fingerprint positioning—offline database construction and online matching—and implement a simple version.

3. Compare the three positioning modes of BLE beacons (proximity detection, trilateration, fingerprinting) and the accuracy of each.
4. Explain the source of UWB's high accuracy, and distinguish the two ranging architectures TWR and TDoA.
5. Compare RFID, NFC, QR codes, and visual markers (AprilTag / ArUco) as landmark-based positioning, in principle and in applicable setting.
6. Explain how indoor positioning and SLAM complement and fuse with each other, and know the trade-offs each technology makes in accuracy, cost, and deployment effort.

Core Concepts

Chinese Term	English Term	Explanation
接收訊號強度指標	RSSI	The signal strength measured at the receiver, which attenuates with distance; the cheapest basis for ranging.
指紋定位	Fingerprinting	Build a database of signal signatures for each position first, then match a live signal against it to find the most similar position.
多路徑效應	Multipath	The signal arrives by several paths after reflecting off walls, making the range too long or unstable.
非視距	NLOS	The transmitter and receiver are obstructed and the signal must penetrate or diffract; a principal cause of error.
三邊定位	Trilateration	Find the intersection from the distances to three or more known points to infer the unknown position.

Chinese Term	English Term	Explanation
三角定位	Triangulation	Infer position from angle information rather than distance.
超寬頻	UWB	A radio technology using extremely short pulses over an extremely wide spectrum, reaching decimeter or even centimeter ranging.
雙向測距	TWR	The tag and anchor communicate back and forth to measure time of flight, requiring no clock synchronization among anchors.
到達時間差定位	TDoA	Compare the difference in a signal's arrival time at several anchors; requires precise anchor synchronization and gives large capacity.
低功耗藍牙信標	BLE Beacon	A low-power Bluetooth device that broadcasts an identifier periodically; commonly used for indoor guidance.
視覺基準標記	Fiducial Marker	An artificial marker such as AprilTag or ArUco from which a relative pose can be resolved precisely from an image.
六自由度位姿	6-DoF Pose	Three axes of position plus three of rotation; a visual marker can resolve the complete pose at once.

4.1 Why GPS Alone Will Not Do Indoors

Walk into the basement parking garage of a department store and the blue dot on your phone map begins to wander, or simply reports that it cannot find you. Behind this everyday experience lie three physical limitations, one on top of another.

1. The Signal Is Simply Too Weak

GPS satellites orbit at medium altitude about twenty thousand kilometers above the surface and transmit at only about 25 to 50 watts—roughly the level of a household light bulb. After traveling twenty thousand kilometers to the ground the signal has attenuated to the order of −130 dBm, weaker than the ambient background noise, and the receiver must pull it out of that noise by correlating against a spreading code. A signal this fragile may lose several tens of decibels more passing through a reinforced concrete floor slab, which extinguishes what little remains.

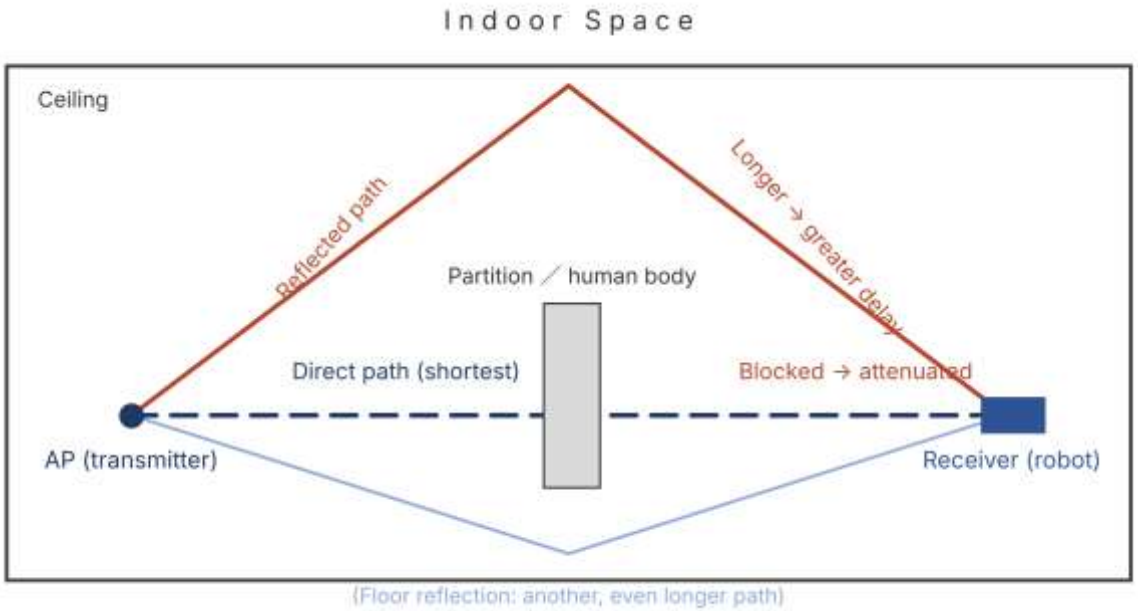
2. Multipath Distorts the Range Measurement

Even when part of the signal leaks in through a window, it has usually bounced off walls, ceilings, and metal cabinets. GPS ranging works by multiplying signal time of flight by the speed of light, and a reflected path is longer than the direct path, so the computed range comes out too large. Worse, indoors the reflected path is often stronger than the direct one (which the wall has blocked), the receiver cannot tell them apart, and the position estimate therefore swings wildly—this is the main reason the blue dot jumps around in a parking garage.

The phenomenon is easy to understand geometrically: what reaches the receiver is not one signal but a set of copies with different delays and different strengths. If the ranging algorithm mistakes a reflected copy for the direct arrival, the computed range

is too long—and always too long, never too short, because a reflected path is necessarily farther than a straight one. This one-sided systematic bias cannot be averaged away over repeated measurements, which makes it the most intractable source of error in indoor radio positioning.

Figure 4-1 Indoor Multipath



Three paths arrive at the receiver at once: the direct wave is weakened by a partition or a human body, while the reflections off ceiling and floor may be stronger. If the ranging algorithm takes the latter, the computed range exceeds the true range—and always in that direction.

Why This Has Anything to Do with SLAM

The LiDAR of Chapter 8 and the visual ranging of Chapter 21 meet a similar problem of false correspondence: a wrong observation taken for a right one pulls the estimate off course.

The treatment is also shared—the RANSAC of Chapter 21 and the robust

kernels of Chapter 12 are, in essence, both saying: do not trust a few anomalous observations.

So although this chapter is about radio, the intuition you gain here—that outliers bias systematically to one side—applies equally in the geometric chapters that follow.

3. The Geometry Deteriorates

Satellite positioning requires at least four well-distributed satellites (three for position and one to solve for clock error). Indoors, the only satellites still visible are usually the few toward a window, so the geometric distribution is very poor. Positioning theory describes this with the dilution of precision (DOP): when the satellites cluster in one direction the DOP rises sharply, and even if every individual range is accurate, the error of the intersected position is amplified several times over.

A Clarification: The Trilemma of Indoor Positioning

① Accuracy: meter level or centimeter level? Each order of magnitude in accuracy usually costs an order of magnitude more.

② Cost: how much infrastructure must be installed in the environment (anchors, beacons, markers)?

③ Coverage: one building? one floor? one room? The larger the coverage, the higher the cost of deployment and maintenance.

Every technology introduced in this chapter is, in essence, a different answer to the trade-off among these three dimensions.

It is precisely because no single indoor positioning technology satisfies all three requirements at once that SLAM matters so much: SLAM requires no pre-installed infrastructure, and the robot can map and localize in an unfamiliar environment with its own sensors. Conversely, SLAM has the weaknesses of drift and relocalization—which is the subject of fusion treated in Section 4.7 at the end of this chapter.

4.2 Wi-Fi RSSI Fingerprint Positioning

Wi-Fi is the most ubiquitous indoor wireless infrastructure—access points (APs) are everywhere in classrooms, offices, and shopping malls, and every phone and laptop has a receiver built in. Since the signal is everywhere, can it be used for positioning? It can, but by a method entirely unlike GPS.

Why RSSI Cannot Be Converted Directly into Distance

The intuitive idea is that signal strength decays with distance, so measuring strength should give distance, and trilateration should then give the intersection point. In theory, signal strength in free space follows the log-distance path loss model:

$$RSSI(d) = RSSI(d_0) - 10 \cdot n \cdot \log_{10}(d / d_0)$$

where n is the path loss exponent (about 2 in free space, commonly 2 to 4 indoors because of obstruction). The problem is that indoors n varies sharply with wall material, foot traffic, and furniture layout: measured RSSI at the same distance can differ by more than 10 dB, which converts into range errors of several meters. The human body is also an excellent absorber—a phone held in the hand reads differently from one lying on a desk. Indoor Wi-Fi positioning therefore does not take the ranging route but the fingerprinting route.

Fingerprint Positioning: A Two-Stage Workflow

The core insight of fingerprint positioning is this: do not try to understand how the signal propagates; simply record what each place looks like. The workflow has two stages:

- Offline (training) stage: divide the site into grid points (for example one reference point every 2 meters), stand at each point for a few seconds, and record the MAC address and RSSI of every visible AP, forming one signal fingerprint. The fingerprints of all reference points constitute the fingerprint database, or radio map.
- Online (positioning) stage: the user's device scans the APs and RSSI values it sees at that moment, compares them for similarity against every fingerprint in the database, and takes the most similar one (or a weighted average of the top K) as the estimated position.

Figure 4-2 The Two Stages of Wi-Fi Fingerprint Positioning

[Offline stage: building the fingerprint database] (slow, but done only once)

(1) Select reference points across the site (for example one every 2 meters)

↓

(2) Stand at each reference point for a few seconds and record the MAC and RSSI of every

visible AP

↓

(3) Store as a database: position (x, y) ↔ fingerprint vector F = [$RSSI_1$, $RSSI_2$, …, $RSSI_n$]

[Online stage: positioning in real time] (fast, runs continuously)

(4) The device scans current RSSI values, giving the observation vector S

↓

(5) Compute the distance D(S, F_i) against every fingerprint F_i in the database

↓

(6) Take the K closest and average their coordinates with weights → output the estimated position

Key idea: the "distance" here is a distance in signal space, not a physical distance.

You will meet this two-stage structure repeatedly in later chapters. The geomagnetic fingerprint positioning of Chapter 6 is the same workflow with field strength substituted for RSSI; the bag-of-words loop closure detection of Chapter 20 follows the same logic—build a feature dictionary offline, compare similarity online. Even the particle weighting of Gmapping in Chapter 14 is, in essence, asking how closely the current observation resembles the observation this position on the map should produce.

Once you see this, you will recognize that localization is in large part matching. The only difference is what is being matched: signal strength, magnetic field, image features, or point cloud geometry.

The Matching Algorithm: K Nearest Neighbors

The most common matching method is KNN (K-nearest neighbors). Let the

currently measured fingerprint vector be S and the fingerprint of the i-th reference point in the database be F_i. The Euclidean distance between them is:

$$D(S, F_i) = \sqrt{\Sigma_j (S_j - F_{ij})^2}$$

where j runs over all commonly visible APs. Take the K reference points with the smallest D and average their coordinates, with weights, to obtain the estimated position. The reciprocal of distance is commonly used as the weight, giving WKNN.

Strengths and Limitations

- Strengths: zero hardware cost (it uses existing APs), broad coverage, and implementable on a phone, which makes it very well suited as a teaching entry point.
- Limitations: accuracy is usually 3 to 8 meters; building the fingerprint database is laborious (one floor may need several hundred reference points); and changes in the environment (an AP moved, partitions rebuilt, crowds) invalidate the database, so it must be rebuilt periodically.

Recent improvements include replacing KNN matching with deep learning, smoothing the trajectory with pedestrian dead reckoning (PDR) from the phone's IMU, and ranging directly with Wi-Fi RTT (round-trip time, IEEE 802.11mc)—the last of which can raise accuracy to 1 to 2 meters, but requires support in both the AP and the device.

4.3 Bluetooth Beacon / BLE Positioning

A Bluetooth Low Energy (BLE) beacon is a small device that runs for one to two years on a single coin cell and periodically broadcasts its own identifier. It became widespread after Apple introduced the iBeacon specification in 2013, and Google proposed the Eddystone specification as well. Compared with Wi-Fi APs, the advantage

of beacons is that you choose where to put them—both density and placement can be designed around the positioning requirement.

Three Modes of Use

- Proximity detection: the simplest and most reliable. No coordinates are computed; the system only judges whether the user is near a given beacon, in three coarse levels—immediate (within 10 cm), near (about 1 m), and far (within 10 m). Museum exhibit guides and in-store promotions commonly use this mode. For a robot it amounts to "I have passed a checkpoint."
- Trilateration: RSSI from three or more beacons is converted to distance and the intersection is solved. The theory is elegant, but because of the RSSI instability discussed in Section 4.2 the practical accuracy is usually 2 to 5 meters, and a good geometric layout of beacons is required.
- Fingerprinting: identical to the Wi-Fi fingerprint method, with beacons in place of APs. Because beacon density is under your control, accuracy is usually somewhat better than with Wi-Fi fingerprints.

Advanced: Bluetooth Direction Finding

From Bluetooth 5.1 onward, direction finding measures the angle of arrival (AoA) or angle of departure (AoD) of the signal with an antenna array. Because angle information is far more stable than RSSI, a well-designed system can push accuracy to the decimeter or even centimeter level—the most important recent advance in BLE positioning. The cost is array antenna hardware, which raises both price and deployment complexity.

Engineering Considerations

- Broadcast rate and battery life are a seesaw: ten broadcasts per second responds quickly but drains the cell, one per second saves power but produces noticeable positioning latency.
- BLE uses the 2.4 GHz band, shared with Wi-Fi and microwave ovens, so interference is significant in crowded venues (the human body has high water content and absorbs 2.4 GHz particularly strongly).
- Beacons must be inspected and have their batteries replaced on a schedule—a long-term maintenance cost that is easily overlooked and considerable when several hundred units are deployed.

4.4 Ultra-Wideband (UWB) Positioning

If Wi-Fi and BLE guess distance from signal strength, ultra-wideband (UWB) measures time directly and computes distance from it—which is the fundamental reason for its leap in accuracy.

Why UWB Is So Accurate

UWB transmits extremely short pulses lasting only a few nanoseconds, occupying a bandwidth of 500 MHz or more. Time resolution is inversely proportional to bandwidth: the shorter the pulse, the more precisely the receiver can determine the instant of arrival. Since light travels about 30 centimeters per nanosecond, resolving arrival time to the sub-nanosecond level yields range at the centimeter level.

More important still is immunity to multipath. A reflected signal arrives later than the direct one; with ordinary narrowband signals the waveform is drawn out, so direct and reflected arrivals overlap and cannot be separated. The UWB pulse is so short that the first arriving peak (the direct wave) is clearly separated from the later reflections,

and the receiver need only take that first peak—exactly the property most needed indoors.

Two Ranging Architectures

- Two-way ranging (TWR): the tag transmits to the anchor and the anchor replies; from the total round-trip time minus the processing time the tag computes the one-way time of flight and the distance. The advantage is that no time synchronization between anchors is needed; the drawback is that every measurement requires a round trip, so the channel congests when there are many tags.
- Time difference of arrival (TDoA): the tag broadcasts one way only, several anchors record their own times of arrival, and a server solves for position from the time differences. The advantages are that the tag only transmits and never receives, which is extremely power-efficient, and that system capacity is almost independent of the number of tags; the drawback is that every anchor must be synchronized to the nanosecond, which makes deployment demanding.

The Truth About Accuracy

Vendor literature often advertises "10 cm accuracy," which is indeed achievable under good line of sight (LOS) with an ideal anchor geometry. In a real site, however, error rises noticeably as soon as a body or a piece of equipment creates a non-line-of-sight (NLOS) condition; comparative studies show that the newer IEEE 802.15.4z standard can hold mean ranging error under NLOS within 0.2 meters, a clear improvement over the older standard. A pragmatic engineering expectation is decimeter level under good deployment, with errors of tens of centimeters or more where obstruction is severe.

Where UWB Stands Today

UWB has moved from the laboratory into consumer and industrial use: precise item finding with phones and tracking tags, automotive digital keys (secure ranging resistant to relay attacks), real-time location systems (RTLS) in factories and warehouses that keep forklifts from colliding, and even ball tracking in sports. For robots, UWB is particularly well suited as a drift-free absolute reference fused with SLAM—the subject of Section 4.7 and a common solution for indoor UAV positioning.

4.5 Landmark Positioning with RFID / NFC / QR Codes

The technologies of the previous three sections all try to estimate position continuously. This section introduces a different idea: discrete landmark positioning—not knowing the coordinates at every instant, but being absolutely certain of where you are when you pass a particular place. This approach is very cheap and very reliable, and is for that reason more popular in industrial settings.

RFID (Radio Frequency Identification)

A passive RFID tag has no battery of its own: it is powered by induction from the field emitted by the reader and returns its identifier, at a cost as low as a few NT dollars. By frequency band they divide into low frequency (LF, a few centimeters), high frequency (HF, about 10 cm), and ultra-high frequency (UHF, several meters). The classic approach for a warehouse AGV is to embed an array of RFID tags under the floor; a reader on the underside of the vehicle reads the identifier as it passes and corrects the position—accuracy depends on tag density, so denser laying means better accuracy.

NFC (Near Field Communication)

NFC can be regarded as a specialized subset of HF RFID with a working range of only about 4 centimeters. The short range is itself an advantage: a successful read means the device is practically touching the tag, so positional certainty is extremely high. It suits charging-dock alignment, tool-cabinet check-in and check-out, and other situations that require zero ambiguity.

QR Codes and Data Matrix Codes

The great advantage of the QR code is that it costs nothing—a sticker from an office printer is a landmark, and a damaged one is peeled off and replaced. This is exactly the approach used by Amazon's warehouse robots (the Kiva system): QR codes are laid out in a regular pattern across the floor, a downward-facing camera under the robot photographs them, and reading the identifier tells the robot which cell of the grid it occupies, while the position and rotation of the code within the image give the fine offset and heading. Hundreds of thousands of robots run in warehouses worldwide on this seemingly humble scheme.

Technology	Working Range	Cost	Information Provided	Typical Application
Passive RFID	Centimeters to several meters	Very low	Identifier	Warehouse floor tags, logistics tracking
NFC	About 4 centimeters	Very low	Identifier	Charging dock alignment, tool check-out
QR Code	Depends on the camera	Almost zero	Identifier + coarse pose	Warehouse robots, teaching experiments
AprilTag / ArUco	Centimeters to several meters	Almost zero	Complete 6-DoF pose	Robot navigation, camera calibration

Table 4-1 Comparison of Four Landmark-Based Positioning Technologies

4.6 Visual Landmarks and AprilTag / ArUco Positioning

A QR code can tell a robot which cell it is in, but what if we want more precise information—the complete three-dimensional position and orientation of the camera relative to the marker? That is where fiducial markers come in. AprilTag (developed at the University of Michigan) and ArUco (developed at the University of Córdoba in Spain and integrated into OpenCV) are the two most widely used systems.

Why a Full Pose Can Be Recovered

The principle is quite intuitive. The marker is a square of known side length, and the three-dimensional coordinates of its four corners are fully known in the marker's own frame (for a marker 10 cm on a side, the corners are (0,0,0), (0.1,0,0), (0.1,0.1,0), and (0,0.1,0)). When a camera photographs the marker, the two-dimensional pixel coordinates of those four corners can also be detected precisely. Given four 3D-to-2D point correspondences plus the camera intrinsics (introduced in detail in Chapter 9), the rotation and translation of the camera relative to the marker can be solved—what computer vision calls the PnP problem (perspective-n-point), examined in depth in Chapter 21.

The Detection Pipeline

1. Binarization: convert the grayscale image to black and white to bring out the high-contrast boundary of the marker.
2. Contour detection and quadrilateral filtering: find closed contours and keep those that approximate a square.

3. Perspective rectification: warp the obliquely imaged quadrilateral back into a square (a homography).
4. Code reading: read the bit pattern of the internal black-and-white cell array, match it against the dictionary to confirm the ID, and reject false detections using the built-in Hamming error correction.
5. Pose solution: solve PnP from the four corners and the camera intrinsics, outputting a 6-DoF pose.

AprilTag Compared with ArUco

- AprilTag: error correction and false-detection rejection are designed more rigorously, so it remains comparatively robust at long range, at low resolution, and under partial occlusion; it is preferred in academia and robotics.
- ArUco: fast detection, a dictionary of configurable size, and deep integration with OpenCV (a few lines of code suffice), which makes it very convenient for teaching and rapid prototyping.

Their Role in a Robotic System

- Absolute pose correction: place markers at the charging dock, by the elevator, and at the laboratory door, and the robot eliminates accumulated drift the moment it sees one—the most common use.
- Precise docking: actions requiring millimeter alignment, such as charging or picking and placing material, use a visual marker for the final stage of visual servoing (Chapter 24).
- UAV landing: a marker on the landing pad lets the aircraft touch down precisely.

- Ground truth for a SLAM system: in teaching experiments, densely placed markers can supply an approximate ground-truth trajectory for evaluating SLAM accuracy (used in the capstone projects of Chapter 31).

It is worth noting that a visual marker is essentially an artificial feature point guaranteed to be reliable. Understanding it is understanding a simplified version of the natural feature points of visual SLAM in Chapter 20—the difference being that natural features are not guaranteed stable, not guaranteed unique, and have no known size, which is precisely where the difficulty of visual SLAM lies.

4.7 Fusing Indoor Positioning with SLAM

By this point you will have noticed a pattern: the strengths and weaknesses of indoor positioning and of SLAM are exactly complementary.

Characteristic	Indoor positioning (Wi-Fi / BLE / UWB / markers)	SLAM
Error behavior	Bounded; does not accumulate with time	Accurate in the short term; drifts in the long term
Infrastructure	Must be deployed and maintained beforehand	None needed; usable on arrival
Initial position	Knows where it is at power-on	Must start from the origin or perform relocalization
Map information	Coordinates only; no environmental structure	Produces a complete map usable for obstacle avoidance and planning
Update rate	Lower (a few times per second)	Higher (tens of times per second)

Table 4-2 How Indoor Positioning and SLAM Complement Each Other

Three Levels of Fusion

- Loosely coupled: the two systems run independently, each outputting its own position estimate, and the two are then combined with weights by a Kalman filter or similar. This is the simplest to implement and the first choice for teaching and rapid prototyping.
- Tightly coupled: the raw measurements of the indoor positioning system (UWB ranges, marker corners) enter the SLAM back-end optimization directly as constraint terms and are solved together. Accuracy is best because all information is handled consistently within one probabilistic framework—a natural extension of the pose-graph optimization of Chapter 12, in which a UWB range is simply an edge between a pose and an anchor.
- Aiding initialization and relocalization: the most pragmatic use. SLAM runs on its own most of the time, and UWB or a visual marker supplies the initial pose at start-up or a relocalization cue after loss of track. At very low integration cost this solves SLAM's two most troublesome problems.

A Representative Architecture

The complete localization system of a campus service robot might look like this: 2D LiDAR SLAM as the mainstay, supplying pose and map ten times per second; AprilTags posted by the elevators and in the lobbies of every floor, so that the robot corrects its absolute pose each time it passes one; NFC at the charging dock to ensure error-free docking; and BLE beacons to decide which floor it is on when moving between levels (a question LiDAR SLAM cannot answer at all, since the laser profile of every floor may look identical). Each technology does its own job—which is the essence

of systems engineering: there is no universal sensor, only an appropriate combination.

Formulations and Procedures

1. The Mathematics of Trilateration

Let there be three anchors at known positions $A_1(x_1, y_1)$, $A_2(x_2, y_2)$, and $A_3(x_3, y_3)$, with measured tag-to-anchor distances d_1, d_2, and d_3. The tag position (x, y) then satisfies three circle equations:

$$(x - x_i)^2 + (y - y_i)^2 = d_i^2 \quad , \quad i = 1, 2, 3$$

Subtracting the second and third equations from the first cancels the squared terms and leaves two linear equations, from which (x, y) can be solved.

In the ideal error-free case the three circles meet at exactly one point; with real measurement error they bound a small triangular region instead. The solution is then obtained by least squares—find the point that minimizes the sum of squared deviations from the three circles:

$$\min \Sigma_i \left(\sqrt{(x - x_i)^2 + (y - y_i)^2} - d_i\right)^2$$

This idea of using redundant measurements to fight noise is exactly the spirit of the SLAM back-end optimization of Chapter 12—you will find that from trilateration to bundle adjustment the mathematical skeleton is one and the same.

2. Computing Time of Flight in TWR

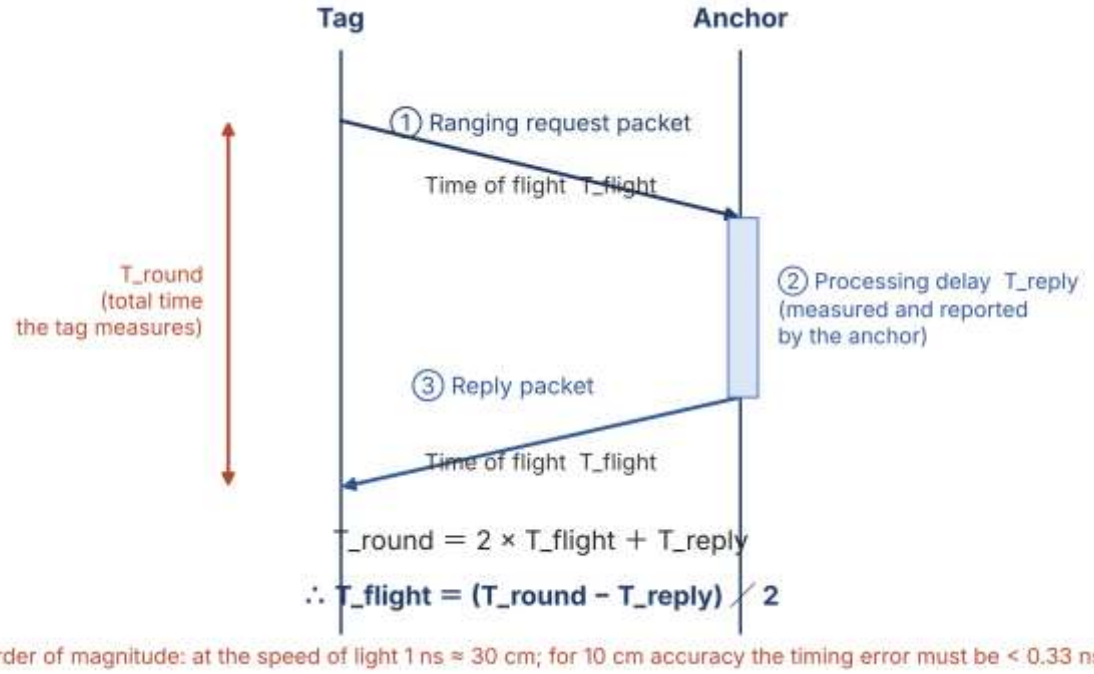
In two-way ranging, let T_round be the total time from the tag's transmission to its receipt of the reply, and T_reply the processing time at the anchor from receiving the signal to sending the reply. The one-way time of flight and the distance are then:

$$T_flight = (T_round - T_reply) / 2$$

$$d = c \times T_flight \quad (c \text{ is the speed of light, about } 3\times10^{8}\ m/s)$$

This shows how critical timing accuracy is: at the speed of light a 1-nanosecond error equals 30 centimeters of range error, and achieving 10 cm accuracy requires timing resolution better than 0.33 nanoseconds. It also explains why ordinary Wi-Fi and Bluetooth chips cannot do this—their clock resolution falls far short of what such a measurement demands.

Figure 4-3 Timing Relationships in Two-Way Ranging (TWR)



This figure explains both why UWB is accurate and why it is expensive. T_reply is measured and reported by the anchor itself, which cancels the effect of clock offset between the two ends—the chief advantage of TWR over one-way ranging, which requires strict clock synchronization. The price is two packets per range measurement, which limits the update rate, and the fact that every tag must take its turn conversing with the anchors, which makes system capacity the practical bottleneck.

3. A Procedure for Choosing an Indoor Positioning Technology

Figure 4-4 A Decision Procedure for Selecting an Indoor Positioning Technology

Question 1: what accuracy is required?

├─ Meter level will do → Wi-Fi / BLE fingerprint positioning (lowest cost)

├─ Decimeter level → UWB or BLE direction finding (anchors must be deployed)

└─ Centimeter level → UWB (with good LOS) or visual markers (at short range)

↓

Question 2: may equipment be installed in the environment?

├─ Not at all → SLAM is the only option (Parts IV through VI)

├─ Paper may be posted → QR Code / AprilTag (zero cost)

└─ Construction and wiring are possible → UWB anchors / an array of BLE beacons

↓

Question 3: is a continuous trajectory needed, or discrete correction?

├─ Continuous → Wi-Fi / BLE / UWB

└─ Discrete correction → RFID / NFC / visual markers + SLAM as the mainstay

4.8 The Contemporary Evolution of Indoor Positioning: A Race Beyond Accuracy

The technical skeleton described in the first seven sections has been fairly stable over the past decade, but several changes worth knowing appeared in 2025 and 2026. Interestingly, most of them are not about greater accuracy but about easier deployment and wider availability.

1. Bluetooth Channel Sounding: A Low-Cost Challenger to UWB

The Bluetooth 6.0 specification incorporates channel sounding, which replaces distance estimation from RSSI with phase-based ranging (PBR). Its theoretical accuracy is on the order of a meter, and under good conditions it can approach the decimeter level, at a cost and power consumption far below UWB, and along the evolutionary path of existing Bluetooth silicon.

Industry assessments, however, are quite sober: line-of-sight obstruction, multipath distortion, scalability, and battery life keep it from being a viable solution for large-scale indoor mobile positioning today. For students this is an instructive case—the theoretical accuracy of a technology and its deployability are two different things, and engineering selection always looks at the latter. That is precisely the habit of mind the selection flowchart in Section 4.7 means to convey.

2. Wi-Fi RTT and 802.11az: The Standard Exists, Adoption Does Not

Wi-Fi round-trip-time ranging (RTT, IEEE 802.11mc) can deliver sub-meter accuracy, but support remains the bottleneck—industry figures put it at roughly thirty percent of Android devices, and iOS does not expose it. An improved one-way RTT scheme works with any AP, at the price of accuracy falling back to three to five meters and requiring extensive post-processing and calibration. The new 802.11az standard brings positioning formally into the Wi-Fi specification and is expected to push accuracy to the decimeter level, but infrastructure replacement cycles are measured in years.

This leads to a judgment that matters to an engineer: in a campus or laboratory setting, the RSSI fingerprint method of Section 4.2, however unglamorous, is the only one that works on every device. That is why Exercise 4-1 in this chapter uses fingerprinting rather than RTT.

3. UWB: The Most Accurate, Still a Premium Fitting

UWB can localize to within thirty centimeters, making it the most accurate of all the wireless technologies in this chapter. Its adoption is limited by hardware cost, support only in premium iOS and Android models, and the infrastructure requirement of deployed anchors. In warehouses, hospitals, factories, and other applications where the site is fixed and the value at stake is high enough, UWB is already a mature solution; in consumer and teaching settings it remains expensive.

4. New Developments in Visual Markers: Making the Marker Invisible

The AprilTag and ArUco markers of Section 4.6 have one long-standing complaint against them: the black-and-white squares are ugly and look badly out of place on the wall of a hospital, an office, or a home. iMarkers, published in 2026, propose an interesting remedy—replacing printing ink with cholesteric spherical reflectors (CSR), which yields markers that are nearly invisible to the human eye yet clearly legible to a camera under particular polarization conditions.

The measured results are worth noting: detection range is comparable to printed ArUco markers of the same size, pose estimation error differs by only 1.3 to 3.9 percentage points, and recognition takes 21 to 105 milliseconds; in very low light below five lux the detection rate still reaches 78 to 89 percentage points, whereas printed markers drop to 6 percentage points. For a robot that must work a dim corridor or patrol at night, this is a substantive improvement.

Another line of work replaces the traditional image-processing pipeline with deep learning. YoloTag, for example, applies an object detection model to marker detection and proves more robust than the conventional binarization pipeline on platforms such as UAVs, which shake violently and often produce motion-blurred images. This belongs to the same thread as the role of YOLO in navigation in Chapter 24 and AI-assisted navigation in Chapter 32.

5. The Real Trend: From Tuning Toward Calibration-Free Operation

Taken together, these changes reveal a common direction: ease of deployment is displacing theoretical accuracy as the first criterion in system selection. Self-calibrating systems that require no fingerprint survey are gradually becoming the industry baseline, because the cost of collecting and maintaining fingerprints noted in Section 4.2 is often harder to accept in a real commercial setting than being a meter less accurate.

Three Criteria for Students

1. Before asking how accurate it must be, ask what the accuracy is for. Navigating to a doorway needs thirty centimeters; finding a particular room needs only three meters.

2. Beyond accuracy there are three further dimensions: deployment cost, device compatibility, and maintenance burden. Table 4-1 and the selection procedure of Section 4.7 are about balancing all four.

3. Absolute localization need not carry all the accuracy alone—fused with the IMU of Chapter 7 and the SLAM of Chapter 13, even three-meter Wi-Fi positioning suffices to support dependable navigation. This is the central claim

of Section 4.7 and the most important idea in this chapter.

Case Studies

Case 4-1 Amazon's Warehouse Robots: Global Logistics Carried on Stickers

The warehouse robots of Amazon Robotics (formerly Kiva Systems) are a textbook case of low technology solving a hard problem. Hundreds of thousands of orange transport robots move through warehouses worldwide, and yet their localization scheme is startlingly plain: QR codes are pasted across the floor at a fixed pitch, a downward-facing camera under the robot reads the identifier to confirm the cell, the offset and rotation of the code in the image give the fine pose, and between cells wheel odometry carries the short-range dead reckoning.

The brilliance of this scheme lies in exploiting the controllability of the environment to the full: the warehouse is built by the company itself, so the floor can be marked at will, the lighting can be provided, and the routes can be constrained to a grid. Given all that, why make the robot struggle with visual SLAM? A service robot, by contrast, enters someone else's building and cannot modify the environment, which is exactly why it must rely on SLAM. The contrast carries an important lesson: before choosing a localization technology, ask first how far you are able to modify the environment—this is often more decisive than the algorithm itself.

Case 4-2 A UWB Real-Time Location System in a Hospital

Large hospitals face a costly problem: mobile equipment such as infusion pumps and ultrasound carts is constantly "wheeled off somewhere," nursing staff spend a

great deal of time each day hunting for it, and the hospital ends up over-purchasing. The remedy offered by a UWB real-time location system (RTLS) is this: deploy a grid of anchors in the ceiling, attach one UWB tag to each piece of equipment, broadcast one way in a TDoA architecture, and have a server solve for all tag positions in real time and display them on the floor plan.

This case demonstrates the value of the TDoA architecture: an equipment tag only transmits and never receives, so one battery lasts for years, and system capacity is nearly independent of the number of tags, so tens of thousands of tags in one hospital still cause no congestion. The challenge in a hospital is the NLOS conditions created by metal equipment and foot traffic—which is also why practical accuracy often falls short of the advertised figure. With small adjustments the same architecture serves forklift collision avoidance in a factory, personnel safety-zone management in a warehouse, and absolute pose correction for a robot.

Case 4-3 An AprilTag Navigation Experiment in the Classroom

A typical teaching experiment in a university intelligent robotics classroom: post ten AprilTags on the four walls, with the world coordinates of each marker measured and recorded in advance. The teaching vehicle carries a single forward-facing camera, and whenever any marker appears in the field of view it can solve for its own 6-DoF pose relative to that marker and convert it into world coordinates—obtaining drift-free absolute localization instantly.

The teaching value of this experiment is very high. First, it lets students verify with their own hands the seemingly magical fact that four corner points plus the camera intrinsics suffice to recover a pose. Second, by covering half a marker, backing away,

and turning off the lights, students experience the fragile points of visual localization directly. Third, and best of all, running it in parallel with LiDAR SLAM and displaying both trajectories at once in RViz—the SLAM trajectory smooth but slowly drifting away, the AprilTag trajectory jumpy but always correct. That one picture is worth a thousand words and conveys the whole motivation for fusion in Section 4.7 at a glance.

Hands-On Activities

Exercise 4-1 A Wi-Fi Signal Heat Map for Classroom Positioning

Purpose: to carry out one complete fingerprint positioning workflow by hand—build the database, take measurements, match, and validate—and so understand both the capability and the limits of RSSI positioning. This exercise requires only a phone and suits high school and undergraduate groups alike.

Equipment

- A smartphone (on Android, install a Wi-Fi analyzer app; because iOS restricts this more tightly, use the AirPort utility instead or substitute BLE beacons).
- Graph paper or spreadsheet software, a tape measure, and tape for marking reference points.

Procedure

1. Site planning: choose a classroom, measure its length and width with the tape, mark grid points on the floor at a 2-meter pitch (an ordinary classroom gives about 12 to 20 reference points), and number them on the graph paper.
2. Offline database construction: stand at each reference point facing a fixed direction (toward the lectern throughout, say, to reduce differences caused by shadowing by the body), record the name and RSSI of the five strongest visible

APs, take three measurements per point and average them, and enter the result in the spreadsheet.

3. Drawing the heat map: for the strongest AP, fill in the RSSI value at each reference point on the graph paper as a color scale (red for strong, blue for weak), and observe whether the spatial distribution matches the expectation that signal weakens with distance from the AP.
4. Online positioning test: one student is led with eyes closed to an undisclosed reference point and measures the fingerprint there; the others estimate the position by KNN (compute the Euclidean distance to every fingerprint in the database and average the coordinates of the three closest).
5. Error statistics: repeat the test at least 8 times, record the estimated and true positions each time, and compute the mean and maximum error.
6. Interference experiment: have the whole class gather around the test point, measure the fingerprint of that same point again, and compare the change in RSSI with the change in positioning error.

Discussion Questions

1. What mean positioning error did you measure? Does it agree with the 3 to 8 meters stated in the text? Where is the reference point with the largest error, and why there?
2. By how much did the crowd increase the error? What does this imply for indoor navigation in a shopping mall at peak hours?
3. If the reference-point pitch were reduced from 2 meters to 1 meter, how much would accuracy improve? How much would the survey workload increase? Is the trade worth it?

4. (Undergraduate) If you were to fuse this system with the LiDAR SLAM of Chapter 14, which of the coupling approaches in Section 4.7 would you adopt, and why?

Extended Challenge (Undergraduate)

Write a WKNN positioning program in Python that reads the fingerprint database you built together with a live measurement and outputs the estimated coordinates and error statistics automatically; then compare accuracy for K = 1, 3, and 5, and try adding step detection from the phone's IMU to smooth the trajectory, observing whether the error falls.

Further Reading

1. P. Bahl and V. N. Padmanabhan, "RADAR: An In-Building RF-Based User Location and Tracking System," IEEE INFOCOM, 2000. (The founding work on Wi-Fi fingerprint positioning; essential reading.)
2. H. Liu et al., "Survey of Wireless Indoor Positioning Techniques and Systems," IEEE Transactions on Systems, Man, and Cybernetics, 2007. (A complete survey of indoor positioning technologies.)
3. IEEE Std 802.15.4z-2020, Enhanced Ultra Wideband (UWB) Physical Layers and Associated Ranging Techniques. (The current standard document for UWB ranging.)
4. E. Olson, "AprilTag: A Robust and Flexible Visual Fiducial System," ICRA, 2011. (The original AprilTag paper, explaining the error-correction design.)
5. S. Garrido-Jurado et al., "Automatic Generation and Detection of Highly Reliable Fiducial Markers Under Occlusion," Pattern Recognition, 2014. (The original ArUco paper.)

6. The ArUco module tutorial in the official OpenCV documentation (docs.opencv.org). (Includes complete code examples; the first reference for implementation.)
7. Bluetooth SIG, Bluetooth Core Specification 5.1, the Direction Finding sections. (The specification for AoA / AoD direction finding.)

From Indoor Positioning to Navigation Sensors: Where This Chapter Stands

The technologies introduced in this chapter share one feature: every one of them requires that something be installed in the environment beforehand—an AP, a beacon, a UWB anchor, an RFID tag, or a visual marker. That is the source of their strength (a drift-free absolute position) and also their fundamental limitation (they fail entirely in an environment where nothing has been installed).

In the five chapters that follow we turn to a different class of sensor: ones that require no installation and work from physical law or from information already present in the environment. The GNSS of Chapter 5 relies on satellites, the geomagnetic sensing of Chapter 6 on the Earth's magnetic field, the IMU of Chapter 7 on inertia, and the LiDAR and camera of Chapters 8 and 9 on the geometry and texture of the surroundings. The price is that none of them can answer directly how many meters into the building you are.

This is exactly why SLAM exists. With neither installed infrastructure nor a map, a robot can only draw the map as it goes and localize itself with the map it has just drawn. The mathematics of Part III and the algorithms from Part IV onward all address this problem. What this chapter asks students to remember is this: if the environment permits

installation, do not be a hero—a thirty-dollar AprilTag is sometimes more dependable than an entire SLAM system.

Summary

1. GPS fails indoors for three reasons: the signal is extremely weak and penetrates building materials poorly, multipath reflection distorts the range measurement, and the deteriorated geometry of the visible satellites amplifies the error.
2. Because indoor propagation is complex, Wi-Fi RSSI is unsuited to direct ranging, so the mainstream approach is fingerprint positioning—build a signal database offline and match with KNN online—with accuracy of about 3 to 8 meters, zero hardware cost, and the burden of maintaining the database.
3. BLE beacons can be placed as the application requires and support three modes: proximity detection, trilateration, and fingerprinting; the direction finding introduced in Bluetooth 5.1 raises accuracy substantially using AoA / AoD, at the cost of an array antenna.
4. UWB measures time of flight directly with nanosecond pulses and can separate the direct wave from reflections, making it the most accurate wireless positioning technology; TWR needs no synchronization but has limited capacity, while TDoA is power-efficient and high-capacity but requires precisely synchronized anchors.
5. RFID, NFC, and QR codes provide discrete but highly reliable landmark positioning at low cost and suit sites where the environment can be modified, with Amazon's warehouse robots as the representative case.
6. Visual markers such as AprilTag and ArUco can recover a complete 6-DoF pose from four corner points and the camera intrinsics (the PnP problem), which

makes them powerful tools for absolute pose correction, precise docking, and SLAM accuracy evaluation.

7. Indoor positioning and SLAM are exactly complementary: the former has bounded error but needs infrastructure, while the latter is self-sufficient but drifts; fusion comes at three levels—loosely coupled, tightly coupled, and aiding initialization and relocalization.

In the next chapter we widen the view from indoors to outdoors and take up radio and satellite navigation: how do GPS and GNSS actually work? Why can RTK compress meter-level error to the centimeter level? And how does a campus patrol robot switch seamlessly between indoor and outdoor localization modes?

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 5

Radio Navigation and Satellite Navigation

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 4 settled absolute localization indoors; this chapter handles the outdoors—and in fact anywhere outdoors on Earth. Satellite navigation is the most widely covering positioning infrastructure in human history, and the only sensor on a robot that can tell it where on Earth it is without anything having been installed.

What this chapter really means to establish, however, is not that GPS is useful but three deeper ideas. First, the mathematical skeleton of GNSS—one more unknown than there are observations (the clock error), hence the need for one more satellite—is the same habit of mind as the state estimation of Chapter 11 and the graph optimization of Chapter 12. Second, geometry determines accuracy: the notion of DOP reappears in another guise in the epipolar geometry of Chapter 21 and the point cloud registration of Chapter 17. Third, no single sensor is dependable and fusion is the answer—the triple GNSS / INS / SLAM architecture of Section 5.6 is a miniature of the second half of this book.

Figure 5-0 How Chapter 5 Connects to the Rest of the Book

[Building on the previous chapter]

Chapter 4 Absolute localization indoors (installation required) → this chapter: absolute localization outdoors (no installation)

The multipath, NLOS, and deteriorating geometry of Chapter 4 acquire quantitative names here: UERE and DOP

[This chapter is responsible for]

The three-dimensional version of range intersection + an unknown clock = pseudorange positioning

Differential corrections and carrier phase = advancing from the meter level to the

centimeter level

Judging signal quality = when a robot should trust GNSS

[Where it leads]

→ Chapters 6 and 7 Filling in when GNSS fails: the geomagnetic field supplies heading, the IMU supplies short-term dead reckoning

→ Chapter 11 This chapter's "many observations for many unknowns" formalized as state estimation

→ Chapter 12 GNSS observations as absolute constraint edges in a pose graph

→ Chapters 18 and 26 The role of GNSS in outdoor LiDAR SLAM and UAV navigation

→ Chapter 26 Autonomous navigation in GNSS-denied environments

Learning Objectives

In the previous chapter we examined indoor positioning; in this one we step outdoors and examine the globe-spanning radio and satellite navigation systems. These are the principal source of a robot's absolute localization capability, and the key lesson for understanding why self-driving cars and UAVs still need SLAM. After completing this chapter, you should be able to:

1. Describe the three segments of GNSS (space, control, and user) and the basic principle of satellite positioning, and explain why at least four satellites are required.
2. Distinguish triangulation, trilateration, and time-difference-of-arrival positioning in principle and in applicable setting.
3. Explain carrier phase measurement and the principle of RTK, and understand why RTK can compress meter-level error to the centimeter level.

4. Compare the potential and the limitations of 5G, LoRa, Wi-Fi RTT, and other terrestrial radio positioning technologies for robotic applications.
5. List and explain the principal error sources in radio positioning: the ionosphere, the troposphere, multipath, obstruction, geometry, and interference.
6. Explain how GNSS fuses with an IMU and with SLAM, and design an indoor–outdoor hybrid navigation strategy for a campus patrol robot.

Core Concepts

Chinese Term	English Term	Explanation
全球衛星導航系統	GNSS	The collective name for the satellite positioning systems of various nations, including GPS, GLONASS, Galileo, and BeiDou.
虛擬距離	Pseudorange	The distance obtained by multiplying signal propagation time by the speed of light; called "pseudo" because it includes the receiver clock error.
星曆	Ephemeris	The satellite's orbital parameters, used to compute its position precisely at any instant.
精度稀釋因子	DOP	Describes how far satellite geometry amplifies positioning error; the smaller the value the better.
載波相位	Carrier Phase	Measurement of the phase of the carrier waveform, far more accurate than pseudorange and the basis of RTK.
整週波未定值	Integer Ambiguity	The unknown whole number of cycles in a carrier phase measurement; resolving it is what achieves centimeter accuracy.
即時動態定位	RTK	A technique achieving centimeter positioning

Chinese Term	English Term	Explanation
		by differential correction from a base station.
差分 GNSS	DGNSS	Uses a base station of known coordinates to compute error corrections and broadcasts them for rovers to use.
星基增強系統	SBAS	A wide-area augmentation system broadcasting corrections from geostationary satellites, such as WAAS and EGNOS.
網路 RTK	NTRIP	The communication protocol for transmitting RTK correction data over the internet.
GNSS 拒止環境	GNSS-denied	An environment where satellite signals are obstructed or jammed and unusable, such as indoors, in tunnels, or in urban canyons.
都市峽谷效應	Urban Canyon	The phenomenon of tall buildings on both sides obstructing satellites and causing severe multipath reflection.

5.1 The Basic Principles of GPS and GNSS

GPS (Global Positioning System) is the satellite positioning system the United States began building in the 1970s and brought to full operation in the 1990s. Today it is no longer the only choice: Russia's GLONASS, the European Union's Galileo, and China's BeiDou have all achieved global coverage, and together with the two regional systems—Japan's QZSS and India's NavIC—they are collectively called the Global Navigation Satellite System (GNSS). As of March 2026 the four major systems had about 130 operational satellites in orbit, of which about 31 belong to GPS, distributed over 6 orbital planes in medium Earth orbit at an altitude of about twenty thousand kilometers. Most smartphones manufactured after 2020 support all four systems at once—which is exactly why phone positioning has become noticeably more accurate in recent years: more satellites are available, so the geometry naturally improves.

The Three Segments

- Space segment: the constellation in orbit. Each satellite carries a high-precision atomic clock and continuously broadcasts its own identifier, a time stamp, and its orbital parameters (the ephemeris).
- Control segment: the network of ground monitoring stations and the master control station. These continuously track the satellites, correct orbit and clock deviations, and upload the latest ephemeris to the satellites.
- User segment: the receiver. It only receives and never transmits—which is the key to understanding GNSS: the satellite does not know where you are, the position is computed entirely by your own receiver, and the number of users can therefore be unlimited.

The Basic Principle: Intersection of Range Measurements

GNSS positioning is, in essence, three-dimensional trilateration. The satellite stamps the signal with "I transmitted this at time t"; the receiver notes its own time on reception, the difference gives the propagation time, and multiplying by the speed of light gives the range. Knowing the satellite's position (computed from the ephemeris) and the range, the receiver lies on a sphere centered on that satellite. Three spheres from three satellites intersect, and in theory the position is determined.

Why a Fourth Satellite Is Needed

The trouble is the clock. The atomic clock aboard the satellite is expensive and accurate to the nanosecond, but the receiver holds only a cheap quartz oscillator that may at any moment differ from satellite time by several microseconds. At the speed of light 1 microsecond equals 300 meters of range error—an error large enough that the three spheres fail to intersect at all.

The remedy is elegant: treat the receiver's clock error as a fourth unknown and solve for it along with the rest. With four unknowns (x, y, z, Δt), four equations are needed—that is, four satellites. This is also why the measured range, inaccurate because of the receiver clock, is not called a true range but a pseudorange.

A side effect of this design is that in completing a position fix a GNSS receiver also corrects its own clock to atomic-clock accuracy. Telecommunication base stations, power grids, and financial trading systems worldwide rely on GNSS for time synchronization—the importance of satellite navigation to modern society goes far beyond helping you find your way.

5.2 Triangulation, Trilateration, and Time-Difference Positioning

The geometric methods of radio positioning fall into three classes, and students must keep them apart—the three terms are often used interchangeably in the Chinese-language literature, yet their principles are entirely different.

1. Triangulation: Using Angles

Measure the bearing of the target relative to two known reference points; the intersection of the two rays is the target's position. Classical surveying, radio direction finding (RDF) in navigation at sea, and the Bluetooth AoA direction finding mentioned in Chapter 4 all belong to this class. Its characteristic is that it needs a directional antenna or an antenna array but no precise time measurement.

2. Trilateration: Using Distances

Measure the distances from the target to several known points and find the intersection of the spheres (or circles). GNSS and UWB TWR belong to this class. Its characteristic is that it needs accurate time or signal strength measurement but no knowledge of direction. Chapter 4 already presented the formulation for the two-dimensional case; GNSS is the three-dimensional version with the clock unknown added.

3. Time Difference of Arrival (TDoA): Using Time Differences

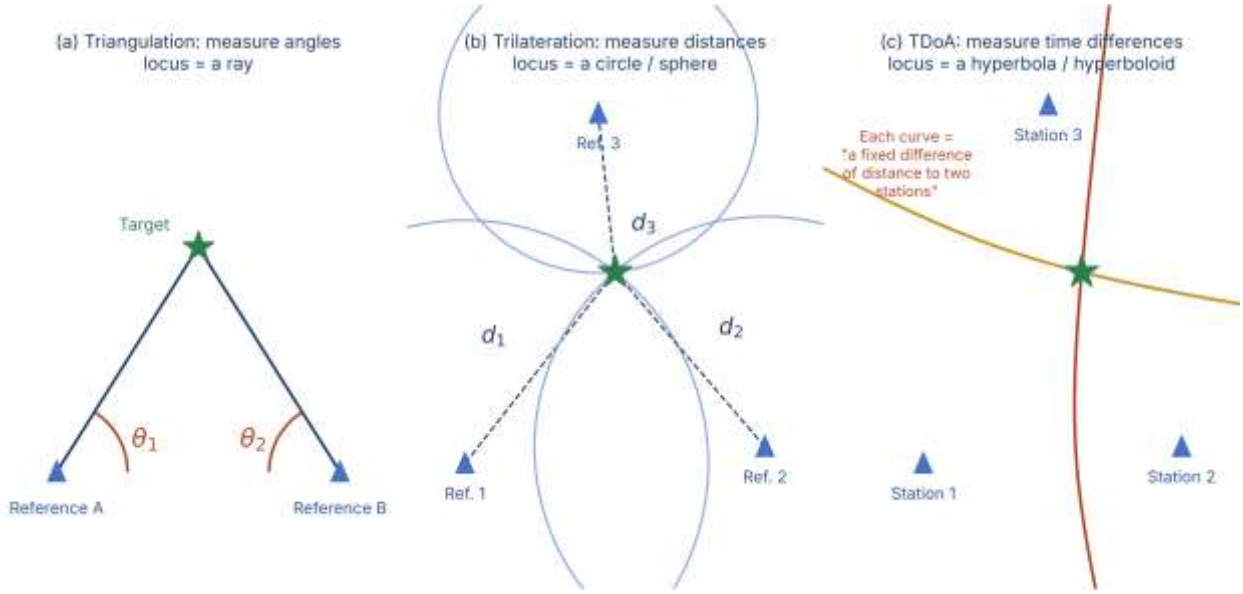
No absolute distance is measured, only the difference in the signal's arrival time at two base stations. The set of points with a fixed time difference forms, geometrically, a hyperbola (a hyperboloid in three dimensions), and the intersection of two or more such surfaces is the target's position—hence the alternative name hyperbolic positioning. The UWB TDoA architecture introduced in Chapter 4 and the LORAN long-wave navigation system of the Second World War era both use this method. Its characteristic is that the target end need only transmit, or only receive, which is

extremely power-efficient and gives large system capacity, at the price of requiring precise time synchronization among the base stations.

Method	Physical Quantity Measured	Geometric Locus	Requirements	Typical Application
Triangulation	Angle	Ray	A directional antenna	BLE AoA, radio direction finding
Trilateration	Absolute distance	Circle / sphere	Precise timing or known transmit power	GNSS, UWB TWR
TDoA	Distance difference	Hyperbola / hyperboloid	Precise synchronization among anchors	UWB TDoA, LORAN

Table 5-1 Comparison of Three Geometric Methods of Radio Positioning

Figure 5-1 Comparing the Loci of the Three Geometric Methods of Radio Positioning



This figure is the page of this section most worth remembering. The three methods differ not in technical detail but in what geometric shape a single measurement

confines the target to—a ray, a circle (sphere), or a hyperbola (hyperboloid). The essence of positioning is to intersect several such constraints.

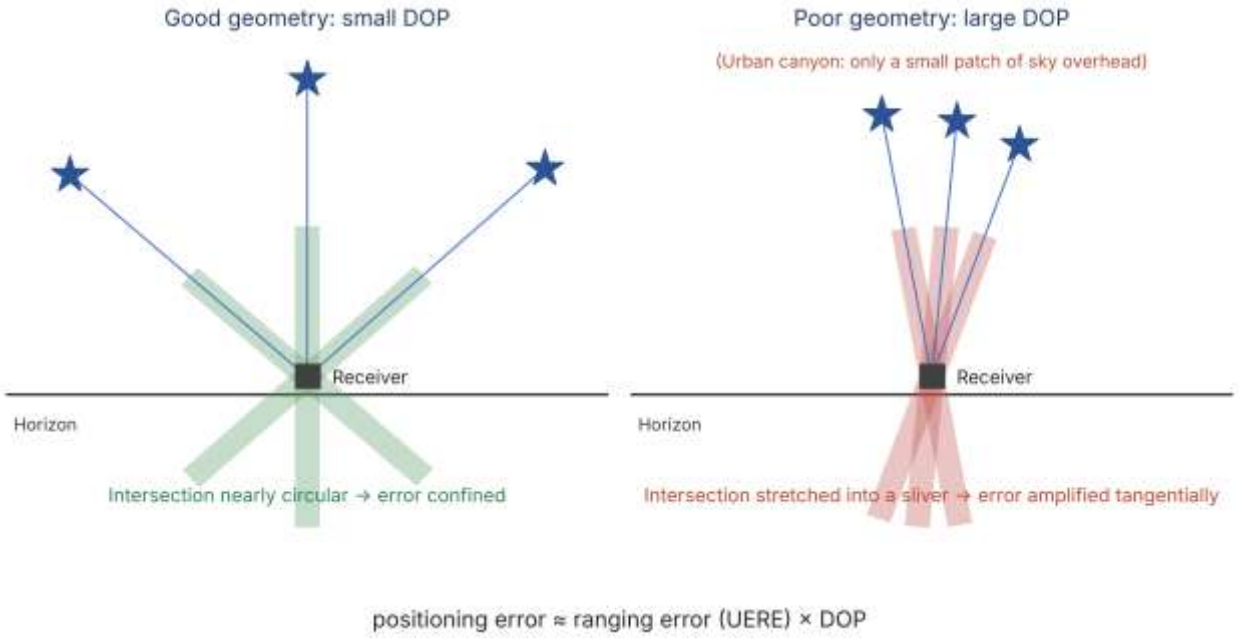
It is worth noting that this idea—an observation confines the state to some manifold, and the intersection is then sought—is precisely the geometric intuition behind state estimation in Chapter 11 and graph optimization in Chapter 12. The scan matching of Chapter 13 and the epipolar geometry of Chapter 21 (where one correspondence confines the camera motion to an epipolar line) are, in essence, doing the same thing. If you build this intuition here, the parts that follow will come much more easily.

Dilution of Precision (DOP)

Whichever method is used, the geometric distribution of the reference points (satellites or base stations) directly affects accuracy. Imagine two satellites in almost the same direction in the sky: their spheres cross at a very shallow angle, and a small change in measurement error slides the intersection a long way along the tangent. Conversely, when satellites are spread evenly in all directions the crossing angles approach perpendicular and the error is confined to a small region.

This amplification is quantified by the dilution of precision (DOP): the actual positioning error is roughly the ranging error times the DOP value. Under open sky the DOP is often below 2, while in an urban canyon or indoors it may climb above 10—which is the quantitative explanation of the "deteriorating geometry" described in Chapter 4.

Figure 5-2 The Effect of Satellite Geometry on Positioning Accuracy (DOP)



Each satellite in the figure draws a semi-transparent band of ranging uncertainty—the width of the band is the ranging error, and its direction is perpendicular to the line of sight. In the left panel the satellites are spread in all directions and the three bands intersect in a small, nearly circular region; in the right panel they are crowded into one small patch of sky overhead, the bands run almost parallel, and the region of intersection is drawn out into a long, narrow slanted sliver. For the same ranging error, positional uncertainty in the right panel is several times greater.

This explains several things seen in practice: why GNSS accuracy in the vertical (height) is always two to three times worse than in the horizontal—there are no satellites below the ground, so the vertical geometry is inherently one-sided; why in an urban canyon "many satellites but terrible accuracy" occurs—number is not geometry; and why a patrol robot should use HDOP rather than the satellite count as its primary quality indicator.

The DOP Family and Its Thresholds

GDOP (geometric), PDOP (position), HDOP (horizontal), VDOP (vertical), TDOP (time)—robot navigation most often uses HDOP.

Rules of thumb in practice: HDOP < 2 is excellent, 2–5 usable, 5–10 marginal, and > 10 should be treated as untrustworthy.

These thresholds are used directly in the patrol system design of Section 5.7 and in the survey sheet of Exercise 5-1.

5.3 RTK-GNSS and High-Accuracy Outdoor Positioning

Ordinary phone GNSS is accurate to about 3 to 10 meters, which is enough to find your way but nowhere near enough for self-driving cars, automated agricultural machinery, or UAV surveying. Compressing the error to the centimeter level requires two key techniques: differential correction and carrier phase measurement.

1. The Idea of Differencing

A large part of GNSS error is common to a region: ionospheric and tropospheric delay, and small satellite clock and orbit deviations, are almost identical at two receivers several kilometers apart.

Differential GNSS (DGNSS) exploits this. A base station is set up at a position whose coordinates have been precisely surveyed; comparing the position it computes with the known true position lets it infer the present error, which it then broadcasts as a correction to nearby rovers. With the common error removed, the rover's accuracy improves from the meter level to the decimeter level.

2. Carrier Phase: The Key to the Leap in Accuracy

Ordinary GNSS uses the ranging code modulated onto the carrier, whose chip

length is on the order of hundreds of meters, so measurement resolution is naturally limited. The carrier itself, however, has a wavelength of only about 19 centimeters (the L1 band), and if its phase can be measured the resolution reaches a hundredth of a wavelength—the millimeter level.

The price is a new unknown: the receiver can measure the fractional part of a cycle precisely but does not know how many whole wavelengths the signal has traveled in all—this is the integer ambiguity problem. The core technique of an RTK system is to resolve that integer quickly and reliably (called ambiguity resolution, or obtaining a "fixed solution"). Once the fix succeeds, accuracy reaches the centimeter level; when it fails, only a "float solution" is maintained and accuracy falls back to the decimeter level.

3. The Architecture of an RTK System

Figure 5-3 The Architecture of an RTK-GNSS System

Base station: coordinates known, receives satellite signals → computes error corrections

↓ (transmitted by radio, 4G / 5G, or internet NTRIP)

Rover: receives satellite signals + the base station corrections

↓

Double differencing → common errors canceled → integer ambiguity resolved

↓

Fixed solution: centimeter positioning / Float solution: decimeter positioning

4. Network RTK and Practical Considerations

Building one's own base station is costly and limited in coverage, so network RTK developed: a national agency or a commercial operator builds a network of base stations and broadcasts correction data over the internet using the NTRIP protocol. In Taiwan, the National Land Surveying and Mapping Center of the Ministry of the Interior operates the e-GNSS system, which provides real-time positioning service across the country.

- Baseline length limit: the farther the rover is from the base station, the less the common-error assumption holds; a separation within a few tens of kilometers is generally recommended.
- Initialization time: reaching a fixed solution from power-on usually takes tens of seconds, and a re-fix is needed after loss of lock in a tunnel or under trees.
- Open sky is still required: RTK removes atmospheric and satellite errors and can do nothing about obstruction or multipath—RTK under trees or beside a tall building is just as unreliable. This point is crucial for robotic applications.

Worth mentioning is a new generation of techniques such as Galileo's High Accuracy Service (HAS), which can broadcast correction data directly by satellite and reach accuracy of about 20 centimeters with no ground base station—a rather practical compromise for wide-area outdoor robotics.

5.4 5G, LoRa, and Wi-Fi RTT in Robot Localization

A satellite signal comes from twenty thousand kilometers away and is inherently fragile. The signal from a terrestrial base station is far stronger—can it be used for positioning? This is a popular direction in the recent convergence of communication

and positioning.

1. 5G Positioning

5G included positioning in its design from the outset and has three advantages: millimeter waves and wide bandwidth give high time resolution, massive MIMO antenna arrays can measure angle of arrival, and the dense deployment of small cells shortens distances. 5G positioning combines time difference of arrival, angle of arrival, and round-trip time, and under ideal conditions reaches the meter or even sub-meter level. For factory AGVs and campus self-driving vehicles this means that communication and positioning share one infrastructure, which is highly attractive. The limitations are that accuracy depends strongly on cell density and that the carrier must expose the relevant capabilities.

2. LoRa Positioning

LoRa is a long-range, low-power IoT communication technology in which a single gateway can cover several kilometers. Its positioning capability comes from TDoA—several gateways record the difference in arrival time of the same packet. Accuracy is only on the order of tens to hundreds of meters, far short of the technologies above, but it wins on coverage, extremely low power, and low cost. Its place is wide-area coarse positioning: tracking farm equipment, monitoring assets in mountainous terrain, or the hybrid architecture in the Chapter 4 case study of "precise UWB ranging plus long-range LoRa backhaul."

3. Wi-Fi RTT (Round-Trip Time)

The Fine Timing Measurement (FTM) mechanism introduced in IEEE 802.11mc lets a device exchange time stamps with an AP and measure the round-trip time directly to obtain distance—fundamentally different from guessing distance from RSSI in Chapter

4. Accuracy reaches 1 to 2 meters and no fingerprint database is required. The limitation is that both AP and device must support FTM, and adoption remains the main bottleneck today.

Technology	Coverage	Typical Accuracy	Infrastructure Required	Role in Robotic Applications
GNSS	Global	3–10 meters	None (the satellites are already up there)	The outdoor mainstay
RTK-GNSS	Within tens of km of a base station	Centimeter level	A base station or a network service	Agricultural machinery, surveying, self-driving
5G	The coverage of the base stations	Meter to sub-meter	Densely deployed base stations	Campuses, factories
LoRa	Several kilometers	Tens to hundreds of meters	Gateways	Wide-area coarse tracking
Wi-Fi RTT	Tens of meters indoors	1–2 meters	APs supporting FTM	Indoor assistance

Table 5-2 Comparison of Radio Positioning Technologies for Robotic Applications

5.5 Error Sources in Radio Positioning: Multipath, Obstruction, and Interference

Only by understanding the errors can one judge when GNSS may be trusted and when a switch to SLAM is required. GNSS errors fall into three classes.

1. Propagation Path Errors (Removable by Differencing)

- Ionospheric delay: the ionosphere, roughly 50 to 1,000 kilometers above the surface, contains a great many free electrons and delays the signal. This is the

largest error source for a single-frequency receiver, reaching several meters and varying sharply with solar activity. A dual-frequency receiver can remove it by exploiting the difference in delay between frequencies—which is why dual-frequency GNSS is more expensive but far more accurate.

- Tropospheric delay: the delay caused by water vapor and pressure in the lower atmosphere, amounting to tens of centimeters up to a few meters, correctable by model estimation.
- Satellite clock and orbit errors: continuously corrected by the control segment, with residual error below about a meter.

2. Local Environmental Errors (Not Removable by Differencing)

- Multipath: the signal arrives after reflecting off buildings, the ground, or water, and the longer path makes the range too large. This is the principal error source in urban and campus settings and can reach several meters or more. Mitigation includes choke-ring antennas and signal quality screening algorithms, but it cannot be eliminated.
- Obstruction: buildings, tree canopies, and tunnels block the signal outright, leaving too few usable satellites or a degraded geometry. The royal palm and banyan canopies common on Taiwanese campuses are enough to drop RTK from a fixed solution back to a float solution.
- The urban canyon effect: with tall buildings on both sides, only a narrow strip of sky remains, the visible satellites cluster in one direction and the DOP degrades, while the wall surfaces create severe multipath—obstruction and multipath striking together. This is the most troublesome localization problem for a self-driving car in a city center.

3. Deliberate Interference

- Jamming: a strong signal swamps the faint satellite signal and makes the receiver lose lock. A small illegal jammer available on the market can disable GNSS within several tens of meters around it.
- Spoofing: counterfeit satellite signals are transmitted so that the receiver computes a wrong but normal-looking position. This is more dangerous than jamming—the system raises no error and simply guides you quietly to the wrong place. For UAVs and self-driving cars this is a serious security issue.

The existence of these three classes of error is the fundamental reason a robot cannot rely on GNSS alone. A robot patrolling under the shade of campus trees may at any moment meet a signal dropout, a jump of several meters, or a loss of the RTK fix—and if the navigation system depends entirely on GNSS, the robot loses its bearings at those moments. The remedy is the fusion of the next section.

5.6 Fusing Radio Navigation with the IMU and SLAM

In Chapter 4 we saw how indoor positioning and SLAM complement each other; the GNSS with IMU and SLAM fusion of this section is the same idea unfolded outdoors, and one of the most mature applications of sensor fusion today.

GNSS and the IMU: A Match Made in Heaven

Their error characteristics are exactly opposite:

- IMU: a very high update rate (hundreds of Hz), extremely accurate over short intervals, and completely immune to obstruction and interference; but integration drift makes the error accumulate rapidly with time (treated in detail in Chapter 7).

- GNSS: a low update rate (1 to 10 Hz) and noisy single measurements; but the error is bounded and does not drift over time.

The logic of fusion is therefore quite natural: let the IMU supply a continuous high-rate trajectory, and let GNSS periodically pull the accumulated drift back. This combination is called GNSS / INS integrated navigation and is standard equipment in aviation, shipping, missiles, and self-driving cars. It is usually implemented with a Kalman filter (Chapter 11), which continuously estimates and compensates the IMU bias, so that even when GNSS is briefly lost (in a tunnel or under trees) the IMU maintains usable accuracy for seconds to tens of seconds—an interval known as the dead-reckoning capability of inertial navigation.

Adding SLAM: Triple Insurance

GNSS / INS still fails in a long tunnel or indoors, and SLAM must then take over. The complete triple architecture is as follows:

Figure 5-4 The Triple GNSS / INS / SLAM Fusion Architecture

IMU (hundreds of Hz): supplies high-rate continuous motion estimation, accurate in the short term

+

GNSS (1–10 Hz): supplies absolute position outdoors, eliminating long-term drift

+

SLAM (10–30 Hz): supplies relative constraints from environmental features; the mainstay when GNSS fails

↓

Fusion filter / graph optimization back end (Chapters 11 and 12)

↓

A continuous, drift-free pose estimate available in all conditions

A Key Engineering Problem: Unifying the Coordinate Systems

Before fusion, an easily overlooked problem must be solved: GNSS gives latitude and longitude (WGS84 geographic coordinates), while SLAM uses local rectangular coordinates with the starting point as origin. The two must first be brought into one frame—the usual approach is to convert latitude and longitude into east-north-up (ENU) local rectangular coordinates with the first GNSS fix as the origin. Time synchronization (aligning the time stamps of all sensors) and extrinsic calibration (the relative mounting positions of GNSS antenna, IMU, and LiDAR) must also be handled. These unremarkable engineering details are often what decides whether an implementation succeeds—the coordinate transforms and TF tree of Chapter 10 treat

this subject in full.

Switching Trust

A practical system also needs an arbiter to decide which source to believe at a given moment. Common criteria include the number of usable satellites, the DOP value, the RTK solution status (fixed, float, or single-point), and whether the discrepancy between the GNSS and SLAM estimates has grown too large. As the robot moves from outdoors to indoors, the system should shift weight smoothly from GNSS to SLAM rather than jumping—which is the central challenge of the campus scenario in the next section.

5.7 Indoor–Outdoor Hybrid Navigation for a Campus Patrol Robot

Putting together the techniques of this and the previous chapter, we can design the complete navigation system of a campus patrol robot. What makes this scenario a classic is precisely that it covers every difficult situation.

Site Analysis: Four Environments on One Patrol

- The open sports field and parking lot: the sky is open, GNSS and RTK perform at their best, and they can serve as the mainstay of localization.
- Tree-shaded walkways and the sides of buildings: obstruction and multipath occur together, and GNSS may jump by several meters or lose its fix. Here SLAM must be primary and GNSS secondary, and the system must be able to judge that GNSS is not to be trusted.
- Breezeways and arcades: semi-indoor, with GNSS coming and going; the transition zone where errors are most likely.

- Indoor corridors and classrooms: GNSS fails completely, so LiDAR or visual SLAM alone carries the load, with the AprilTags or BLE beacons of Chapter 4 supplying floor information and absolute correction.

System Design Strategy

1. Layered maps: outdoors, a global map referenced to GNSS coordinates; indoors, a local SLAM map for each building; the two joined at the doorway as an anchor, forming a topological network of maps (where the topological maps of Chapter 3 come into their own).
2. Sensor configuration: an RTK-GNSS antenna (the outdoor mainstay) + an IMU (the backbone of continuity) + a 3D or 2D LiDAR (the SLAM mainstay) + a camera (visual markers and semantic recognition).
3. Mode-switching mechanism: decide jointly from the GNSS quality indicators (satellite count, DOP, solution status) and the SLAM matching quality, adjusting weights progressively in the transition zone rather than switching hard.
4. Reinforcing the transition zones: post an AprilTag at the entrance of each building as a bridge between the outdoor and indoor coordinate frames—the moment the robot sees the marker it knows its position in both frames at once, and the handover is seamless.
5. Failure fallback: if every means of localization goes wrong at once (strong interference together with an obstructed LiDAR, say), the robot should stop and report rather than drive on with a wrong pose—a basic principle of safety design.

This architecture is actually realized in the capstone project of Chapter 31. For now you need only grasp one central idea: navigation for a real-world robot is never a matter of choosing the single best technology but of letting several technologies run a

relay, each on the leg it runs best. That is exactly the purpose of introducing the six major sensors one at a time in Part II of this book—you must know the strengths and weaknesses of every team member to be a good coach.

Formulations and Procedures

1. The Pseudorange Equation

Let the receiver be at (x, y, z) with clock bias Δt, let the known position of the i-th satellite be (x_i, y_i, z_i), and let the measured pseudorange be ρ_i. Then:

$$\rho_i = \sqrt{(x - x_i)^2 + (y - y_i)^2 + (z - z_i)^2} + c \cdot \Delta t$$

where c is the speed of light. There are four unknowns—x, y, z, and Δt—so at least four satellites are needed to form four equations. Because the equations are nonlinear, in practice they are linearized by Taylor expansion and solved iteratively, or handled by least squares when there are redundant measurements from more than four satellites—with 10 satellites visible there are 10 equations for 4 unknowns, and the surplus information serves precisely to suppress noise.

2. Positioning Error and DOP

Positioning error can be approximately decomposed into the product of two factors:

$$\textit{positioning error} \approx \textit{UERE} \times \textit{DOP}$$

UERE (user equivalent range error) is the equivalent ranging error of a single satellite (including ionosphere, troposphere, multipath, and so on), while DOP is the geometric amplification factor. This compact relation points to two routes of improvement: reduce the ranging error (dual frequency, differential correction) or

improve the geometry (more constellations, a more open location). You can observe this directly with a GNSS viewer app on a phone: step beside a tall building and the satellite count falls, the DOP rises, and the error circle immediately grows.

3. The Carrier Phase Observation Equation

The carrier phase observable φ (in cycles) relates to the true range r as:

$$\lambda \cdot \varphi = r + c \cdot \Delta t + \lambda \cdot N + \text{error terms}$$

where λ is the carrier wavelength (about 19 centimeters for L1) and N is the integer ambiguity—an unknown integer. The whole essence of RTK lies in using double differencing between base station and rover to cancel most of the error terms and then resolving N as an integer quickly and reliably. Once N is fixed, λ · φ becomes a range measurement accurate to the millimeter, and that is the source of centimeter-level positioning.

4. A Procedure for Judging the Trustworthiness of a GNSS Fix

Figure 5-5 A Decision Procedure by Which a Robot Judges Whether to Trust GNSS

① Are there ≥ 6 visible satellites? No → do not trust; switch to SLAM as the lead

↓ Yes

② Is HDOP < 2.0? No → reduce the weight

↓ Yes

③ Is the RTK status a fixed solution? Float → medium weight; single-point → low weight

↓ Fixed

④ Is the discrepancy from the IMU / SLAM estimate below threshold? No → judge it a multipath jump and reject this measurement

↓ Yes

⑤ Accept as a high-weight absolute position measurement and correct the accumulated drift

5.8 The Contemporary Upheaval in Satellite Navigation: Stronger Signals and Stronger Interference

The GNSS architecture described in the first seven sections has been fairly stable since it took shape in the 1990s. Between 2025 and 2026, however, two opposing forces appeared in this field at once—one making positioning more accurate and more robust, the other making it less trustworthy. Both bear directly on robotic system design, and students should know them.

1. High Accuracy for Free: The Galileo High Accuracy Service

The RTK of Section 5.3 requires a base station and a communication link, which is the greatest obstacle to its spread. The European Union's Galileo High Accuracy Service (HAS) offers another route: precise ephemerides and clock corrections are broadcast free of charge in the satellite signal itself, so a receiver needs neither a network connection nor a subscription to reach about twenty centimeters of horizontal accuracy by precise point positioning (PPP).

This is good news for robotics teaching—it moves centimeter-class positioning a long way from professional equipment requiring a base station toward consumer-grade hardware. The price is that PPP takes longer to converge (usually several minutes) and does not fix instantly as RTK does, so it suits patrol or agricultural machinery running for long stretches rather than stop-and-go settings.

2. Low Earth Orbit Navigation Constellations: A Generational Gap in Signal Strength

A more fundamental change comes from low Earth orbit positioning, navigation, and timing (LEO-PNT). Traditional GNSS satellites sit in medium orbit at twenty thousand kilometers, and their signal reaches the ground at only about −130 dBm—weaker, as Section 5.1 noted, than the ambient noise. A LEO satellite is only about five hundred kilometers up, forty times closer, and the received power is accordingly far greater.

Take one representative commercial system: its planned maximum received power is about −136 dBW, which the operator claims to be a hundred times the strength of the traditional GPS L1 C/A signal; after an in-orbit software update the ranging accuracy of a single satellite improved from 4.2 centimeters to about 1.5 centimeters, and once the full constellation is deployed the target under open sky is about two centimeters

horizontally and four centimeters vertically. The constellation is planned at two hundred fifty-eight satellites, more than ten times the twenty-four-satellite baseline constellation of GPS; the first production satellites are expected to launch in October 2026, with service beginning in 2027.

For robots the significance of signal strength is not only greater accuracy but reception in more places—under tree shade, at the edge of a building, indoors near a window: the settings listed as failure zones in Section 5.5 may all become localizable again.

3. The Other Force: Jamming and Spoofing Are Becoming Routine

In the same period, however, the scale of deliberate interference with GNSS has far exceeded anything in the past. Observations by the experimental LEO satellite just described, from an altitude of five hundred kilometers, revealed widespread signal interference from Europe through the Middle East—from France in the west to the Pakistani border in the east; in the worst-affected regions GPS signal strength fell from a normal level of about forty decibels to only about ten. As the research team put it: "We expected to see some interference, but far more than we imagined."

This turns the "deliberate interference" of Section 5.5 from a short paragraph in a textbook into a routine condition that system design must confront head-on. Jamming leaves the receiver unable to acquire a signal, so the robot at least knows it has gone blind; spoofing is more dangerous—it sends a plausible false signal and lets the robot go confidently to the wrong place.

It is worth noting that the strong signal of a LEO constellation is itself a countermeasure: measurements indicate that its stronger signal can shrink the area a jammer is able to disable by as much as ninety-five percent. The operator also concedes, however, that a

nearby or high-power jammer may still succeed under unfavorable conditions—no single system is immune.

4. The Terrestrial Network Fills In: 5G Positioning

The 5G positioning introduced in Section 5.4 is also advancing. From Release 17 onward the 3rd Generation Partnership Project (3GPP) brought high-accuracy positioning formally into the specification, and Release 18 further strengthened carrier phase positioning and sidelink positioning—the latter letting devices range against each other directly without routing everything through a base station. This is a development worth noting for the multi-robot collaboration of Chapter 27: a group of robots can range against one another and form a local relative-positioning network.

5. The Conclusion for a Robotics Engineer

Taking these four developments together yields a clear conclusion: GNSS is becoming both stronger and more fragile at once. Stronger, because signal power, correction services, and multi-system multi-band reception are all improving; more fragile, because the electromagnetic environment it inhabits is no longer simple.

And the engineering response is exactly what Section 5.6 of this chapter has already set out: never let any single sensor bear the burden of localization alone. However good GNSS becomes, a robot should be able to detect when it cannot be trusted and at that moment switch gracefully to IMU dead reckoning and SLAM.

A Practical Criterion: Three Lines of Defense Against Spoofing

1. Plausibility check: does the position jump instantaneously? Does the speed exceed the robot's physical limit? (The IMU of Chapter 7 provides exactly the

independent motion reference needed.)

2. Consistency check: does the discrepancy between the GNSS position and the SLAM estimate persistently exceed a threshold? (This is the heart of the decision procedure of Figure 5-5.)

3. Signal-level check: is the carrier-to-noise ratio (C/N_0) anomalously uniform? Real satellites at differing elevations should differ noticeably, while a spoofed signal is often too consistent.

Case Studies

Case 5-1 Self-Driving Farm Machinery: RTK's Most Successful Commercial Application

In the years when self-driving cars were still being tested, RTK-guided tractors were already in large-scale commercial operation in farm fields worldwide. Why agriculture became the perfect stage for RTK is well worth considering: fields are open and unobstructed, so satellite signal quality is excellent; the working path consists of regular straight passes back and forth and needs no complex decision-making; and yet accuracy requirements are very high—a few centimeters of error in row spacing causes overlapping or missed sowing and accumulates into considerable losses in yield and cost.

RTK lets a tractor drive its planned route automatically to centimeter accuracy, with the operator intervening only to turn at the headland (newer systems automate the turns as well). This case shows an important engineering judgment: when the environment is extremely favorable to a given technology, use that technology to the full rather than bolting on sensors you do not need for the sake of technical

completeness. Conversely, drive the same tractor under an orchard canopy and RTK fails at once—only then is LiDAR or visual SLAM needed to take over. Technology selection is always decided by the site.

Case 5-2 Self-Driving Car Localization in an Urban Canyon

A self-driving car in a city center faces the harshest test of GNSS: tall buildings on both sides cut the sky into a narrow slit, the visible satellites drop from more than ten to three or four clustered in one direction, and the DOP degrades several-fold; worse still are the mirror reflections of glass-curtain towers—the receiver may be receiving a signal bounced back from the building opposite, the computed position may be off by tens of meters, and the system usually does not know it is wrong.

The mainstream solution in self-driving cars is to reduce dependence on GNSS drastically: a pre-surveyed high-definition map serves as the reference, the onboard LiDAR and cameras localize by matching map features in real time (lane lines, curbs, sign poles, building outlines), and GNSS serves only for initialization and coarse regional determination. This is exactly where the commercial value of the 3D LiDAR mapping techniques of Chapters 17 and 18 lies—when the signal overhead is unreliable, the structure of the ground around you is the most faithful reference.

Case 5-3 UAV Flight in GPS-Denied Conditions

A consumer UAV can hover stably on GNSS in the open, but as soon as it flies beneath a bridge, into an indoor plant, or into a forest it enters what is called a GNSS-denied environment. Early UAVs would drift out of control in such conditions, or even trigger a return-to-home that flew them into an obstacle.

The modern solution is visual-inertial odometry (VIO): a downward- or forward-

facing camera tracks texture features on the ground or in the surroundings, motion is estimated together with the IMU, and no external signal is relied upon at all (treated in detail in Chapter 26). This also explains why research on UAV visual localization is concentrated at ETH and UZH (Chapter 2)—their central question is precisely how a robot can fly on its own eyes when satellites cannot help. For the reader of this book this is a clear thread: this chapter is about using the signal well when there is one, and Parts VI and VII are about what to do when there is none.

Hands-On Activities

Exercise 5-1 A GNSS Signal Quality Survey of the Campus

Purpose: to measure satellite signal quality at various points on campus with a phone, verify first-hand the effect of obstruction, multipath, and DOP on positioning accuracy, and produce a GNSS trustworthiness map to serve as the basis for route planning in the patrol robot project of Chapter 31.

Equipment

- A smartphone with an app that displays satellite information (search for tools of the "GNSS viewer" or "GPS Test" kind; it must show the satellite count, signal strength, and DOP or an accuracy estimate).
- A campus floor plan, a recording sheet, and a tape measure (optional).

Procedure

1. Choosing points: select at least 8 representative measurement points across the campus, covering four environments—an open sports field, a tree-shaded walkway, the north wall of a building, a breezeway or arcade, and an indoor corridor, with at least one point each.

2. Measuring: hold the phone still for 60 seconds at each point, then record the total number of visible satellites, the number actually used in the fix, the distribution of signal strengths, the HDOP value (or the accuracy estimate the app shows), and the latitude and longitude displayed.
3. Repeatability test: measure again at the same point after an interval of 5 minutes and compare the difference in latitude and longitude between the two—that difference is the repeatability error of the fix.
4. Drawing the trustworthiness map: classify each point into three grades by satellite count and DOP (green: trustworthy, GNSS may lead; yellow: downweight, with SLAM assisting; red: not trustworthy, switch to SLAM) and mark them on the campus plan.
5. Observing multipath: stand for 2 minutes beside a glass-curtain tower and for 2 minutes in the open, compare how far the computed position jumps, and record whether the distribution of satellite signal strengths looks anomalous in each case.
6. Summary report: using the trustworthiness map, plan a patrol route for a campus patrol robot, indicating which localization method should lead on each stretch and where AprilTag correction points should be placed.

Discussion Questions

1. At which point was the repeatability error largest? What were the satellite count and DOP value there? Which class of error source from Section 5.5 of this chapter does it correspond to?
2. Which has the greater effect on positioning, shade from trees or the side of a building? What do you think is the reason (hint: foliage is partly transmissive, while a wall blocks completely but reflects)?

3. If this patrol robot carried GNSS alone and no SLAM, on which stretches of your planned route would it run into trouble, and with what consequences?
4. (Undergraduate) Following the decision procedure of Figure 5-5, set the thresholds for your own campus (satellite count, HDOP, positional discrepancy) and justify each choice.

Extended Challenge (Undergraduate)

Write a program in Python that parses the NMEA sentences (GGA, GSA, GSV) output by a phone or a USB GNSS receiver, records satellite count, HDOP, and the fix quality indicator automatically, and plots time series and trajectory. Then try computing the standard deviation of the position scatter at a static point to quantify the difference in accuracy among environments. This program can be carried over directly into the patrol project of Chapter 31.

Further Reading

1. P. Misra and P. Enge, Global Positioning System: Signals, Measurements, and Performance, 2nd ed., Ganga-Jamuna Press, 2011. (The standard textbook on GNSS principles.)
2. B. Hofmann-Wellenhof, H. Lichtenegger, and E. Wasle, GNSS – Global Navigation Satellite Systems, Springer, 2008. (A complete reference covering the four major systems and RTK.)
3. P. D. Groves, Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems, 2nd ed., Artech House, 2013. (The authoritative work on GNSS / INS integrated navigation; the principal reference for Section 5.6.)

4. The e-GNSS real-time kinematic positioning system website of the National Land Surveying and Mapping Center, Ministry of the Interior. (Official information and usage instructions for Taiwan's network RTK service.)
5. The RTKLIB open-source project (rtklib.com). (Free GNSS and RTK processing software, usable for teaching experiments and post-processing analysis.)
6. European Union Agency for the Space Programme, the official Galileo High Accuracy Service (HAS) documentation. (The new generation of satellite-broadcast correction service.)
7. 3GPP TS 38.305, NG-RAN Stage 2 Functional Specification of User Equipment Positioning. (The standard specification for 5G positioning.)

From Satellites to Sensors: Where This Chapter Stands

Chapter 4 and this chapter together answer one question: how does a robot know where in the world it is? The answer is that if the environment allows it—infrastructure installed indoors, open sky outdoors—let external infrastructure tell it. That is the least troublesome approach and the one least prone to drift.

But these two chapters also mark out the boundary. Where there is no installed infrastructure indoors and no sky outdoors—tunnels, underground garages, the depths of a warehouse, forests, buildings after a disaster, the surface of the Moon—a robot must rely entirely on itself. Chapters 6 through 9 introduce the internal senses available to it one at a time: the magnetic compass, the inertial measurement unit, LiDAR, and the camera. None requires any external installation, and all share one fatal weakness: they drift.

And the only way to eliminate drift is for the robot to recognize that it has returned to

a place it has been. That is loop closure detection, and it is the reason SLAM is possible at all. The mathematics of Part III and the algorithms from Part IV onward all prepare the way for it. The central judgment this chapter leaves with the student is this: when GNSS is available, use it; when it is not, know that it is not—the latter is the truly difficult and truly important engineering capability.

Summary

1. GNSS comprises three segments—space, control, and user; the receiver only receives and never transmits, performs three-dimensional trilateration with pseudoranges, and needs at least four satellites because the clock bias must be solved for at the same time. As of 2026 the four major systems had about 130 satellites in orbit in all.
2. Radio positioning has three geometric methods: triangulation uses angles, trilateration uses distances, and TDoA uses distance differences; the geometric distribution of the reference points is quantified by DOP, and the positioning error is roughly the ranging error times the DOP.
3. RTK combines differential correction with carrier phase measurement to achieve centimeter-level positioning, the key being resolution of the integer ambiguity; a fixed solution is centimeter-level and a float solution decimeter-level, and neither can do anything about obstruction and multipath.
4. Each terrestrial radio positioning technology has its place: 5G suits campuses and factories, LoRa suits wide-area coarse tracking, and Wi-Fi RTT can provide meter-level indoor ranging—but all are limited by how widely the infrastructure has been adopted.

5. GNSS errors divide into propagation errors removable by differencing (ionosphere, troposphere, satellite clock and orbit), local errors that cannot be removed (multipath, obstruction, urban canyons), and deliberate interference (jamming and spoofing).
6. GNSS and the IMU have complementary error characteristics and together form a mature integrated navigation system; adding SLAM yields a triple architecture that maintains continuous, drift-free localization outdoors, in transition zones, and indoors alike.
7. A campus patrol robot must adopt a different primary means of localization in each zone of the environment and use visual markers as the bridge between the indoor and outdoor coordinate frames; the central idea of the system design is to let several technologies run a relay, each on the leg it runs best.

In the next chapter we take up one of the oldest means of navigation still in service—the Earth's magnetic field. From the mariner's compass to the phone's compass app, from correcting ferromagnetic interference to indoor geomagnetic fingerprint positioning, we will see how the planet's own field supplies a robot with an absolute directional reference.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 6

Geomagnetic Navigation and the Electronic Compass

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapters 4 and 5 answered the question "where am I"; from this chapter Part II turns to another question, equally basic and more easily overlooked: "which way am I facing?"

Position and orientation are two different things. Even if a robot knows it is standing in the middle of a corridor, it still cannot move a step without knowing which way it faces—it can neither translate "go forward three meters" into the right action nor correctly mark on the map a sensor reading of "a wall two meters ahead." Chapter 10 formalizes this as pose: position plus orientation, six degrees of freedom of which none may be missing.

The magnetometer of this chapter is the only sensor on a robot that supplies an absolute heading—it needs no map, needs no installation, and does not drift. The price is that it is extremely easy to disturb, and the source of the disturbance is often the robot itself. This character of being uniquely absolute yet easily contaminated gives it a very particular role in sensor fusion: it contributes little most of the time but is irreplaceable at the critical moment.

Figure 6-0 How Chapter 6 Connects to the Rest of the Book

[Building on the previous chapters]

Chapter 4 Wi-Fi fingerprint positioning → Section 6.3 of this chapter: the same method with magnetic field strength substituted

Chapter 5 GNSS gives position but cannot give heading at rest → this chapter supplies that piece

[This chapter is responsible for]

The three geomagnetic parameters (F, D, I) → tilt compensation → magnetic heading

→ true heading

Hard-iron and soft-iron calibration: correcting a deformed point cloud back to an ideal sphere

Turning "interference" into a "fingerprint": indoor geomagnetic positioning

[Where it leads]

→ Chapter 7 The IMU: the gyroscope is accurate in the short term and the magnetometer in the long term, and together they form an AHRS

→ Chapter 10 Heading angle becomes formally part of the pose

→ Chapter 14 Geomagnetic fingerprint matching ≈ the observation model of a particle filter

→ Chapter 26 The consequences of a failed UAV compass calibration (Case 6-1)

→ Chapter 29 Mounting location and interference troubleshooting for the IMU and magnetometer on a physical vehicle

Learning Objectives

The compass is one of humanity's oldest navigational instruments, and yet it remains active in every phone and every UAV. This chapter examines how the Earth's magnetic field supplies a robot with an absolute directional reference, and why indoors it turns from a clean direction sensor into a distinctive positional fingerprint. After completing this chapter, you should be able to:

1. Explain the origin of the Earth's magnetic field and the three parameters that describe it: declination, inclination, and field intensity.
2. Explain how a magnetometer measures the field, and compute a heading angle from a three-axis magnetometer together with tilt compensation.

3. Explain the principle of indoor geomagnetic fingerprint positioning, and account for why "interference" becomes instead a source of positional information.
4. Distinguish the causes of hard-iron and soft-iron error, and carry out practical calibration procedures such as the figure-eight calibration.
5. Describe the role geomagnetic sensing plays in phones, UAVs, and quadruped robots, and its known limitations.
6. Analyze the error characteristics of geomagnetic sensing, the IMU, and visual SLAM, and explain the logic of fusing them complementarily.

Core Concepts

Chinese Term	English Term	Explanation
地磁場	Geomagnetic Field	The field produced by convection of liquid metal in the Earth's outer core, with an intensity of about 25 to 65 microtesla.
磁偏角	Declination	The horizontal angle between magnetic north and true north, varying with place and time.
磁傾角	Inclination / Dip	The angle between the field lines and the horizontal plane; near 0 degrees at the equator and near 90 at the poles.
磁力計	Magnetometer	A sensor measuring field strength and direction, mostly by the Hall effect or magnetoresistance.
電子羅盤	Electronic Compass	A module combining a magnetometer and an accelerometer to output a heading angle.
航向角	Heading / Yaw	The angle between the robot's direction of travel and a reference north.

Chinese Term	English Term	Explanation
傾斜補償	Tilt Compensation	Using the attitude measured by the accelerometer to project the magnetometer readings back into the horizontal plane.
硬鐵誤差	Hard-iron Error	A fixed offset caused by permanently magnetized material on the platform, displacing the center of the measurement sphere from the origin.
軟鐵誤差	Soft-iron Error	Ferromagnetic material distorts the external field, deforming the measurement sphere into an ellipsoid.
地磁指紋	Magnetic Fingerprint	The local field distortion caused by a building's steel structure, usable as a positional signature.
世界地磁模型	WMM	The global field model maintained jointly by NOAA and BGS, updated every five years.
磁暴	Geomagnetic Storm	A violent field disturbance driven by solar activity, which temporarily degrades compass accuracy.

6.1 The Earth's Magnetic Field and the Magnetometer

The Earth is an enormous magnet. The field comes not from some solid lodestone but from the convective motion of hot liquid iron and nickel in the planet's outer core—the searing conductive fluid forms circulating currents under the combined action of rotation and thermal convection and, like a giant dynamo, continuously generates a magnetic field. This mechanism is called the geodynamo.

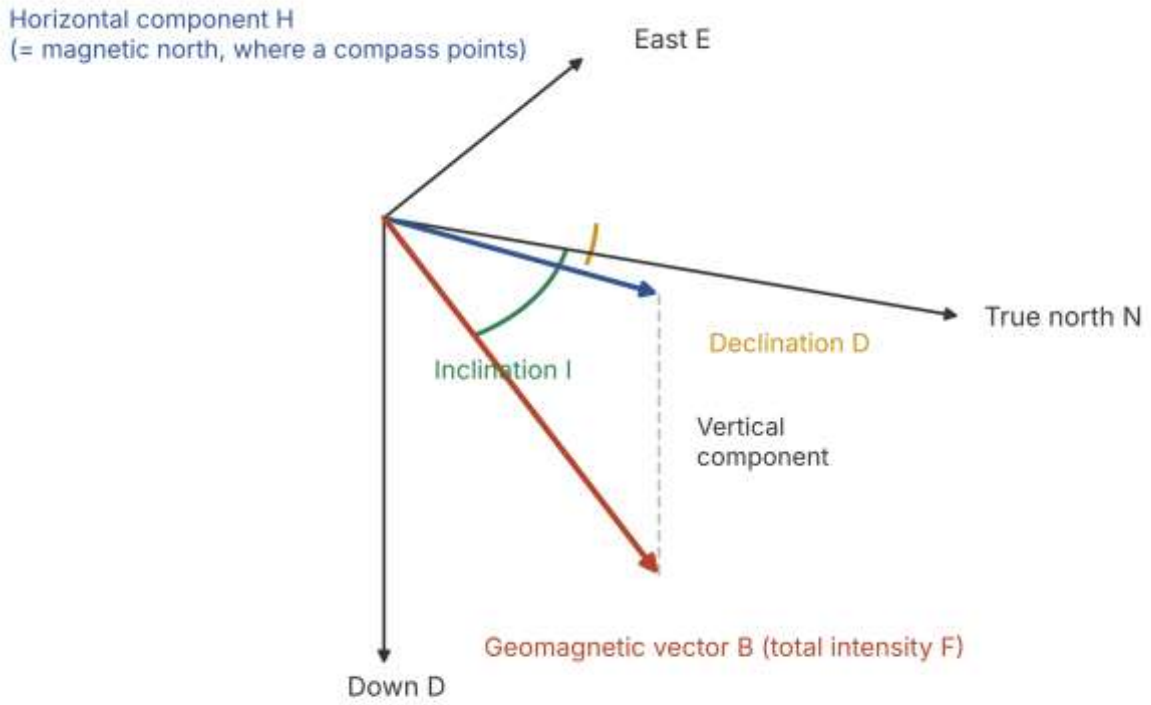
The Three Parameters That Describe the Geomagnetic Field

At any point on the surface the geomagnetic field is a three-dimensional vector, usually described by three parameters:

- Total intensity: between about 25 and 65 microtesla (μT), weaker near the equator and stronger at high latitudes. In Taiwan it is around 45 μT. By comparison, the surface field of a refrigerator magnet can reach several thousand microtesla—the geomagnetic field is in fact extremely weak, which is exactly why it is so easily disturbed.
- Declination: the horizontal angle between magnetic north and true north. Maps and navigation software use true north while a compass points to magnetic north, and the difference is the declination. In Taiwan the declination is about 4 degrees west (varying slightly with place and year), meaning the "north" a compass indicates lies about 4 degrees west of true north.
- Inclination: the angle between the field lines and the horizontal plane. Near the magnetic equator it approaches 0 degrees (field lines parallel to the ground) and near the magnetic poles 90 degrees (field lines entering the ground vertically). In Taiwan it is about 35 degrees—a parameter that matters greatly to a robot, because it means the geomagnetic vector has a substantial vertical

component, and tilt compensation is required before the heading can be computed correctly.

Figure 6-1 The Geomagnetic Vector and Its Three Descriptive Parameters



This figure lays out the geometric relation among the three parameters at a stroke. The geomagnetic vector B points from the origin obliquely downward (in the northern hemisphere), and its projection H onto the horizontal plane is the magnetic north a compass indicates; the horizontal angle between H and true north is the declination D, and the angle between B and the horizontal plane is the inclination I.

Note in particular that the inclination in Taiwan is about 35 degrees—which means the geomagnetic vector has a substantial vertical component. Tilt the robot even slightly and that vertical component "leaks" into the readings of the horizontal axes and the heading comes out wrong. This is exactly why the tilt compensation of Section 6.2 exists, and why an electronic compass can never work without an accompanying

accelerometer (Chapter 7). Near the magnetic equator, where I approaches zero, tilt compensation matters far less; at high latitudes, where I approaches 90 degrees, the horizontal component H becomes too small and the compass all but fails—the magnetic blackout zone mentioned in Section 6.1.

The Field Drifts: The World Magnetic Model

Because the motion of the outer-core fluid is not steady, the geomagnetic field changes continuously and slowly, and the north magnetic pole in particular keeps moving—over recent decades it has drifted from northern Canada toward Siberia, and noticeably faster since the late 1990s. Precise navigation therefore requires a continuously updated model.

The World Magnetic Model (WMM), maintained jointly by the U.S. NOAA National Centers for Environmental Information (NCEI) and the British Geological Survey (BGS), is that standard. It is updated every five years; the current edition, WMM2025, was released in December 2024 and is valid through the end of 2029, and a high-resolution edition, WMMHR2025, is published alongside it. The model is used by NATO, the U.S. Department of Defense, aviation and marine systems, and the compass app on every phone—the "true north" shown on your phone is this model performing the declination correction behind the scenes.

The model also marks the magnetic blackout zones near the poles, where the horizontal field intensity is too low for compass navigation to be reliable. In addition, magnetic storms driven by solar activity make the declination depart temporarily from the model prediction—an effect especially pronounced for aviation operations at high latitudes.

How a Magnetometer Measures

- The Hall effect: when current flows through a conductor subjected to a perpendicular magnetic field, the carriers deflect to one side and a transverse voltage appears whose magnitude is proportional to the field strength. The principle is intuitive and the cost low, but sensitivity is relatively poor.
- Magnetoresistance: the resistance of certain materials changes with the magnetic field. Anisotropic magnetoresistance (AMR) and tunneling magnetoresistance (TMR) offer high sensitivity and low power consumption and are the schemes commonly used in phones and robots today.

A modern MEMS magnetometer integrates three-axis sensing elements in a chip a few millimeters square, often packaged together with a three-axis accelerometer and a three-axis gyroscope to form what is called a nine-axis IMU module. Note that the raw output of a magnetometer is the three field components in the sensor frame, which is still a considerable amount of processing away from a heading angle—the subject of the next section.

6.2 The Electronic Compass and the Heading Angle

The needle of a traditional compass is a freely pivoting magnetized bar, held level by gravity and pointed to magnetic north by magnetic force. An electronic compass has no moving parts and must accomplish the same thing with mathematics—and a pitfall many beginners fall into is hidden here.

The Simplest Case: Perfectly Level

If the magnetometer happens to be held level, the heading needs only the two horizontal components:

$$\psi = atan2(-m_Y, m_x)$$

where m_x and m_Y are the two horizontal-axis readings of the magnetometer and ψ is the magnetic heading. atan2 is used rather than atan so that all four quadrants are resolved correctly.

The Real Problem: The Robot Tilts

The trouble is that a robot can hardly stay level forever—a vehicle crossing a slope, a UAV pitching forward to accelerate, a quadruped climbing stairs: the sensor tilts with the platform. Because the inclination in Taiwan is about 35 degrees, the geomagnetic vector has a substantial vertical component, and once the sensor tilts that component leaks into the two horizontal axes and badly biases the heading. In practice a tilt of 10 degrees can produce a heading error of several to more than ten degrees.

The Remedy: Tilt Compensation

The remedy is to measure the direction of gravity with an accelerometer first, deduce the roll angle φ and pitch angle θ, and then rotate the magnetometer readings back into the horizontal plane:

$$m_x' = m_x \cdot \cos\theta + m_z \cdot \sin\theta$$

$$m_Y' = m_x \cdot \sin\varphi \cdot \sin\theta + m_Y \cdot \cos\varphi - m_z \cdot \sin\varphi \cdot \cos\theta$$

$$\psi = atan2(-m_Y', m_x')$$

This is why an electronic compass module must always include an accelerometer—a magnetometer alone cannot work correctly while tilted. It also explains something familiar from daily life: a phone compass app usually asks you to hold the phone level, because although tilt compensation corrects for attitude, an accelerometer on a moving platform measures acceleration together with gravity, which spoils the attitude estimate and in turn makes the heading unstable.

From Magnetic Heading to True Heading

The last step is to apply the declination correction:

$$\textit{true heading} = \textit{magnetic heading} + \textit{declination}$$

In Taiwan the declination is about 4 degrees west (recorded by convention as a negative value), so the true heading is roughly the magnetic heading minus 4 degrees. The precise declination can be looked up from the WMM by latitude, longitude, and date—a small example of GNSS and geomagnetic sensing working together: you must know roughly where you are (GNSS) before you can look up the right declination correction.

6.3 Indoor Geomagnetic Fingerprint Positioning

The previous two sections treated interference as the enemy—the rebar in a building distorts the geomagnetic field and throws the compass off. But consider it from another angle: since the pattern of distortion differs at every position and is quite stable, are those distortions not themselves an excellent positional signature? This inversion is the central insight of indoor geomagnetic positioning.

The Principle: Turning a Bug into a Feature

The rebar, steel beams, elevator shafts, and heavy machinery of a modern building are all made of ferromagnetic material and distort the geomagnetic field locally. These distortions have three properties highly favorable to positioning:

- Spatial distinctiveness: the field vector (magnitude and direction) differs from position to position, forming a fingerprint-like signature.

- Temporal stability: building structure does not move, so the field distribution is unchanged for months or even years—far more stable than a Wi-Fi signal (APs get relocated, and crowds cause fluctuations).
- Zero infrastructure: no beacon or base station need be installed; only a database must be built. This is its greatest advantage over UWB and BLE.

The Workflow: Directly Descended from Wi-Fi Fingerprinting

The workflow is almost identical to the Wi-Fi fingerprint positioning of Chapter 4: in the offline stage, field signatures are collected along corridors or at grid points to build the database; in the online stage, a live measurement is matched against the database to find the most similar position.

One Crucial Difference: Too Few Dimensions of Information

Geomagnetic fingerprints have an inherent weakness, however. A Wi-Fi fingerprint at one point may contain a dozen APs, forming a feature vector of a dozen dimensions with strong discriminating power; a geomagnetic fingerprint has only three components (and often only the one dimension of total intensity), so its discriminating power is comparatively thin—it is not at all unusual for two points ten meters apart in a corridor to have exactly the same field strength.

For this reason practical geomagnetic positioning rarely matches single points and instead matches trajectories: the user or robot records a sequence of field readings while walking, and that whole stretch of "field curve" is matched against the database. A curve is far more distinctive than a single value, much as a phrase of melody is easier to recognize than one note. Common methods include the particle filter (Chapter 11) and dynamic time warping (DTW).

Practical Accuracy and Limitations

- Accuracy: combined with trajectory matching and pedestrian dead reckoning (PDR), indoor accuracy reaches about 1 to 3 meters, comparable to or better than Wi-Fi fingerprinting.
- Survey cost: as with Wi-Fi fingerprinting, collection requires walking the site by hand, which is the greatest obstacle to deployment.
- Environmental change: relocating heavy metal equipment or remodeling the building invalidates part of the database; elevator movement and the starting and stopping of large motors also cause momentary disturbance.
- Magnetic storms: although solar activity affects the geomagnetic field, indoor positioning matches a local relative pattern rather than an absolute value, so the effect is comparatively slight.

6.4 Ferromagnetic Interference and Hard-Iron / Soft-Iron Calibration

The magnetometer is one of the sensors most in need of calibration, for a simple reason: the geomagnetic field is only about 45 μT, while the fields produced by the robot's own motors, battery, steel screws, and metal chassis are often far larger. Without calibration the compass reads the robot's own field rather than the Earth's.

The Ideal Case: A Perfect Sphere

First establish a key geometric intuition. Rotate a magnetometer slowly through every direction, collect all three-axis readings, and plot them in three-dimensional space. Since the magnitude of the geomagnetic field is fixed and only its direction changes, those points should in theory lie on a sphere centered at the origin with radius equal to the geomagnetic intensity. The whole task of calibration is to correct the

deformed cloud of measured points back onto that ideal sphere.

Hard-Iron Error: A Displaced Center

Hard-iron error comes from objects on the platform that produce a fixed magnetic field—permanent magnets, magnetized steel parts, motors. They superimpose a constant field vector in the sensor frame, and the effect is to translate the whole sphere away from the origin.

Correction is comparatively simple: find the maximum and minimum of the measured cloud on each axis and take the midpoint as the offset:

$$b = (m_max + m_min) / 2$$

$$m_corrected = m_raw - b$$

Soft-Iron Error: The Sphere Becomes an Ellipsoid

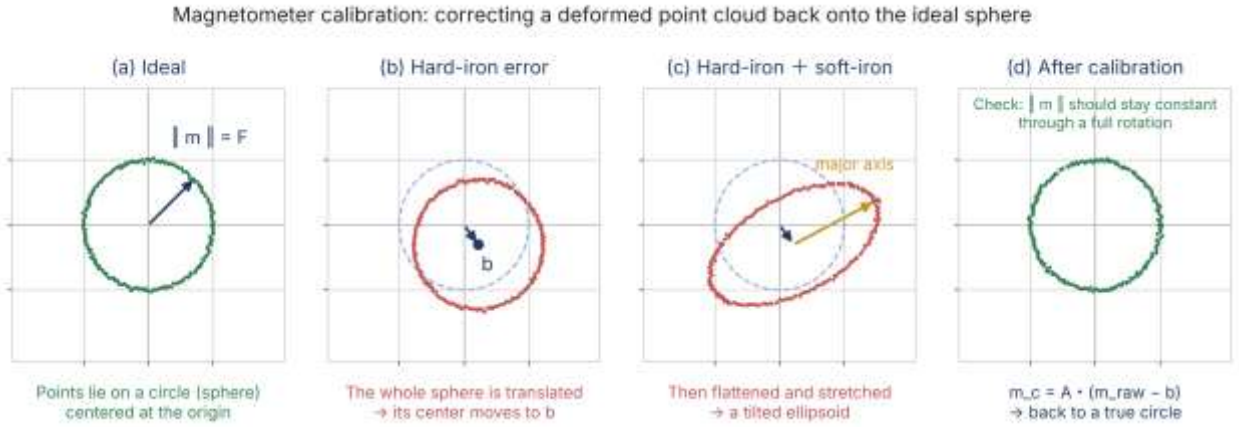
Soft-iron error comes from ferromagnetic material that produces no field of its own but distorts an external one—iron plate, steel beams, the chassis. Such material concentrates field lines in some directions and spreads them in others, and the effect is to flatten or stretch the sphere into an ellipsoid, possibly tilted as well.

Correction requires a 3×3 transformation matrix to pull the ellipsoid back into a true sphere:

$$m_corrected = A \cdot (m_raw - b)$$

The matrix A is usually estimated from the measured point cloud by an ellipsoid fitting algorithm. This is also why full magnetometer calibration requires data from every direction—turning only in the horizontal plane cannot constrain deformation in the vertical.

Figure 6-2 The Geometric Meaning of Hard-Iron and Soft-Iron Error



These four panels contain the entire spirit of this section. (a) Ideally, turning the magnetometer through every direction puts all readings on a sphere centered at the origin with radius equal to the total geomagnetic intensity. (b) Hard-iron error translates the whole sphere—its shape is unchanged, only its center has moved to b. (c) Soft-iron error further flattens, stretches, and rotates the sphere into a tilted ellipsoid. (d) Calibration subtracts the offset and then applies a matrix that pulls the ellipsoid back into a true sphere.

Once this figure is understood, three practical matters become self-evident: why turning only in the horizontal plane is not enough (that constrains only one cross-section of the ellipsoid and leaves vertical deformation entirely unobserved, hence the figure-eight with flips); why calibration must be done in the final installed configuration (b and A are properties of the whole robot, and calibrating a sensor removed from it measures a different set of parameters); and why "turn through a full rotation and see whether the total intensity stays constant" is the best verification (successful calibration is equivalent to the cloud returning to a true sphere, and a true sphere is by definition one whose radius is everywhere the same).

A Common Misconception

Many students think calibration means pointing north. It does not—calibration requires no knowledge whatever of which way north lies.

Calibration does one thing only: it corrects the shape of the measured point cloud back into a sphere. Which direction on that sphere corresponds to true north is the work of the tilt compensation of Section 6.2 and the declination correction of Section 6.1.

This division of labor—correct the sensor's own systematic error first, then speak of its relation to the world—reappears with exactly the same structure in the camera calibration of Chapter 9 (intrinsics versus extrinsics).

Practical Calibration Procedure

1. Figure-eight calibration: trace a figure eight in the air with the device while rotating it, so that the sensor covers as many three-dimensional attitudes as possible. This is exactly what a phone app is asking for when it says to wave the phone.
2. Calibrate in the final installed configuration: the magnetometer must be mounted on the robot, with all wiring connected and the battery in place, before calibration—because hard-iron error is created by the whole robot, and calibrating the sensor on its own is meaningless.
3. Stay away from environmental sources: calibrate away from reinforced walls, computers, and large metal tables, or the soft-iron effect of the environment will be recorded as if it were error of the platform itself.
4. Verification: turn through a full rotation after calibration and check whether the total field intensity stays steady (it should be close to the local geomagnetic

intensity with little variation); this is the simplest indicator of whether calibration has succeeded.

Dynamic Interference: The Part Calibration Cannot Save

One class of interference cannot be dealt with by calibration beforehand: the field produced by changing motor current. As the robot accelerates the current rises and the field changes with it—a dynamic error tied to the state of motion. Mitigations include mounting the magnetometer far from the motors and power leads (which is why UAVs often place the compass on an extended arm or atop the GPS antenna), building a compensation model from the current reading, or reducing the magnetometer's weight in the filter during high-current intervals.

6.5 The Role of Geomagnetic Navigation in Phones, UAVs, and Quadruped Robots

Phones

The magnetometer in a phone carries three duties: showing direction in the compass app, setting the view heading in a map app (so the map rotates as you turn), and establishing the initial bearing in AR applications. It also supplies a drift-free heading reference in sensor fusion—on the gyroscope alone a phone's orientation estimate would depart noticeably within minutes.

UAVs

The UAV is the platform that depends most on the compass and is most troubled by its problems. The flight controller needs heading to interpret GNSS position changes correctly and to carry out directional control; once the compass is wrong, the classic circular flight known as the toilet bowl effect appears—the UAV believes it is flying

forward while it actually circles outward in ever-widening loops.

- Pre-flight calibration is standard procedure and must be performed near the actual launch point.
- Never calibrate or take off near reinforced structures, iron fencing, an underground garage, or a large metal roof.
- Professional models often carry two compasses that check each other, and switch automatically to a GNSS heading (inferred from the direction of motion) when anomalous readings are detected.

Quadrupeds and Ground Robots

A quadruped or wheeled robot changes attitude sharply, carries many metal parts, and has high-power motors, so a magnetometer works in a rather hostile environment. Indoors, therefore, most systems deliberately reduce the magnetometer's weight or dispense with it altogether—heading is supplied instead by the scan matching result of LiDAR or visual SLAM, because in a structured indoor environment the direction of the walls is far more dependable than the geomagnetic field.

Conversely, in open outdoor settings with little metal (farmland, grassland, large outdoor sites), the geomagnetic field remains the most economical and dependable source of heading. This confirms once again an idea this book stresses repeatedly: there is no sensor that is absolutely better or worse, only settings it suits and settings it does not.

6.6 How the Geomagnetic Field, the IMU, and Visual SLAM Complement One Another

Placed within the localization system as a whole, the role of geomagnetic sensing

can be summed up in one sentence: the only low-cost passive sensor that supplies an absolute heading.

Sensor	Information Provided	Error Characteristics	Principal Weakness
Gyroscope	Angular rate (integrated for change of angle)	Very accurate in the short term; drifts in the long	Accumulation of bias drift
Accelerometer	Direction of gravity (roll, pitch)	No long-term drift	Contamination by motional acceleration
Magnetometer	Direction of magnetic north (heading)	No long-term drift	Ferromagnetic interference; needs calibration
Visual / LiDAR SLAM	Relative pose and environmental structure	Accurate in the short term; drifts in the long	Depends on environmental features
GNSS	Absolute position (heading inferable)	Bounded error	Fails indoors; low update rate

Table 6-1 The Role of Each Sensor in Attitude and Heading Estimation

Three-Way Complementarity: The Logic of an AHRS

The attitude and heading reference system (AHRS) is precisely the classic design that combines the first three:

- The gyroscope supplies smooth, high-rate angular change and is the backbone of attitude estimation.

- The accelerometer corrects roll and pitch drift using the direction of gravity—but it constrains heading (rotation about the vertical axis) not at all, because rotating about the gravity axis does not change the direction of gravity.
- The magnetometer fills exactly that gap: it corrects heading drift using the direction of magnetic north.

The division of labor can be remembered this way: the gyroscope handles everything in the short term, the accelerometer handles up and down in the long term, and the magnetometer handles the compass points in the long term. The complementary filter and the extended Kalman filter (Chapters 7 and 11) are the mathematical tools that realize this division.

After SLAM Is Added

Visual or LiDAR SLAM can also supply heading, and in a structured indoor environment it is often more dependable than a magnetometer. A common strategy in modern indoor robots is therefore:

5. At start-up, let the magnetometer supply a coarse initial heading so that SLAM has a sensible starting point (keeping the map from being built in an entirely random direction).
6. During operation, use the SLAM scan matching result as the primary source of heading, with the magnetometer downweighted or used only to monitor for anomalies.
7. On entering an outdoor or unstructured open area (where the risk of SLAM failure is high), raise the magnetometer's weight again.

8. If the magnetometer reading and the SLAM estimate disagree substantially and persistently, raise an alert—this usually means either that a large metal object is nearby or that SLAM has already lost track.

This design pattern of adjusting trust by situation is exactly the same as the GNSS trustworthiness decision of Figure 5-5 in Chapter 5. By this point you should be able to feel that although the chapters of Part II appear to treat different sensors, they repeatedly treat the same subject: know the error characteristics of each sensor, then let each contribute the right information at the right moment.

Formulations and Procedures

1. The Component Representation of the Geomagnetic Vector

The geomagnetic vector B can be decomposed in the local horizontal frame (north N, east E, down D) as:

$$B = (B_N, B_E, B_D)$$

from which the three commonly used parameters follow:

$$\text{horizontal component} \quad H = \sqrt{(B_N^2 + B_E^2)}$$

$$\text{declination} \quad D = \operatorname{atan2}(B_E, B_N)$$

$$\text{inclination} \quad I = \operatorname{atan2}(B_D, H)$$

$$\text{total intensity} \quad F = \sqrt{(B_N^2 + B_E^2 + B_D^2)}$$

Taking Taiwan as the example, with total intensity F of about 45 μT and inclination I of about 35 degrees, substitution gives a horizontal component $H \approx F \cdot \cos I \approx 37$ μT and a vertical component of about 26 μT. The vertical component is a considerable fraction of the whole, which is the quantitative reason Section 6.2 insists that tilt compensation cannot be omitted.

2. The Magnetometer Calibration Model

The complete calibration model handles hard-iron and soft-iron error at once:

$$m_corrected = A \cdot (m_raw - b)$$

where b is the 3×1 hard-iron offset vector (the displacement of the sphere's center) and A the 3×3 soft-iron correction matrix (restoring the ellipsoid to a true sphere). The indicator of calibration quality is this: after calibration, the total field intensity should stay steady in any attitude:

$$||m_corrected|| \approx F \quad \textit{(the local total geomagnetic intensity, a constant)}$$

In practice one computes the standard deviation of ||m|| over a stretch of rotation data, and the smaller the value the more successful the calibration. This is a practical indicator students can compute, and judge, immediately in the laboratory.

3. The Complete Processing Chain of an Electronic Compass

Figure 6-3 The Processing Chain of an Electronic Compass from Raw Readings to True Heading

① Raw three-axis magnetometer readings m_raw

↓

② Hard-iron and soft-iron calibration: $m = A \cdot (m_{raw} - b)$

↓

③ Attitude from the accelerometer: roll φ, pitch θ

↓

④ Tilt compensation: project m back into the horizontal plane to obtain m_x', m_Y'

↓

⑤ Compute the magnetic heading: $\psi = \text{atan2}(-m_Y', m_x')$

↓

⑥ Apply the declination correction (looked up from the WMM by latitude and longitude) → true heading

↓

⑦ Fuse with the gyroscope (complementary filter or EKF) → a smooth, drift-free heading output

6.7 Contemporary Developments in Geomagnetic Navigation: From Manual to Automatic Calibration

Geomagnetic sensing is an old technology—the compass has a history of nearly a thousand years. Its application in robotics, however, has taken two notable turns in

recent years: the resolution of the model has improved greatly, and calibration has moved from a manual ritual to something done automatically.

1. A Finer Geomagnetic Model: WMMHR

The World Magnetic Model (WMM) mentioned in Section 6.1 was released in its WMM2025 edition in December 2024. Published alongside it was the high-resolution edition WMMHR2025, whose spatial resolution near the equator is about three hundred kilometers—roughly a tenfold improvement over the standard edition's three thousand three hundred kilometers.

This difference matters little for ordinary navigation, but for work requiring a high-accuracy heading—surveying, aviation, and long-range autonomous navigation aided by the geomagnetic field—its meaning is clear: the finer the model, the more accurate the local declination correction and the smaller the error in true heading. NOAA recommends that users move to the high-resolution edition.

It is worth remarking that this model is itself a kind of map—only what it draws is not the position of walls but the distribution of a magnetic field. You may regard it as a global-scale prior map maintained by national institutions; the indoor geomagnetic fingerprint map of Section 6.3 is the same concept shrunk to the scale of a building and built by the user. The occupancy grid map to be built from Chapter 13 onward is a map of yet another scale and content. The question of what a map is turns out to be much broader than intuition suggests.

2. No More Waving the Phone: Calibrating Automatically During SLAM

The figure-eight calibration described in Section 6.4 is the greatest practical obstacle to using a magnetometer—it demands an unnatural motion of the user, and in many

settings it simply cannot be done. A sensor mounted on a train, in an elevator, or on a high-speed platform cannot be picked up and waved; nor will a cleaning robot roll itself over.

The remedy proposed in recent research is to bring calibration into SLAM itself. A representative 2024 work proposes a "simultaneous localization, mapping, and calibration" architecture in which a factorized particle filter estimates the magnetometer bias along with the pose and the magnetic field map—concretely, each particle maintains its own Kalman filter to track the bias, and loop closures are determined by spatial neighborhood analysis. Experiments on mobile robots in a shopping mall, an office, and an apartment produced bias estimates comparable to the calibration method built into the operating system, with no calibration motion required of the user at all.

The structure of this method repays close study: its skeleton is precisely the Rao-Blackwellized particle filter used by Gmapping in Chapter 14—the pose is represented by particles, and the remaining quantities (the map in Gmapping, the map plus the bias here) are maintained analytically for each particle. In other words, this is the algorithm you will implement with your own hands in Part IV, applied to a different object. It confirms once again a point this book stresses repeatedly: the mathematical skeleton of SLAM is general, and replacing the sensor and the observation model lets it solve an entirely new problem.

3. Deep Learning Enters Geomagnetic Fingerprint Positioning

Section 6.3 noted that the fundamental difficulty of geomagnetic fingerprinting is having too few dimensions of information—only three numbers at each position (and in practice often only the single number of total intensity), so different positions collide

very easily. Recent work turns to sequence models: rather than a single reading, it considers the shape of the whole curve of field variation over the robot's motion and matches it with a neural network.

Such methods have advanced from early convolutional networks to architectures combining attention mechanisms with long short-term memory networks, and show better robustness than traditional KNN matching in complex indoor environments. Their central insight agrees with the visual bag of words in Chapter 20 and the ORB-SLAM2 loop closure detection in Chapter 22: a single observation is unreliable, but a sequence of observations is highly distinctive.

Threading the Three Together

A finer model (WMMHR) → a more accurate absolute heading; automatic calibration (built into SLAM) → no more manual ritual; sequence learning → localization even from low-dimensional observations.

Their common direction is this: hand the steps that once required human intervention to algorithms that resolve them while running.

That is exactly the spirit of the implementation chapters in Part VIII of this book—a good robotic system should leave the user responsible only for saying where to go, and everything else to the system.

Case Studies

Case 6-1 The UAV "Toilet Bowl Effect": A Textbook Case of Compass Error

A vivid term circulates among UAV pilots: the toilet bowl effect—in position-hold

mode the aircraft not only fails to hold station but circles outward in ever-widening loops, like water going down a bowl. This is almost always a compass problem.

The cause can be understood as follows. The flight controller believes the nose points north while it actually points northeast. When GNSS shows the aircraft drifting east, the controller commands a correction to the west—but what it takes for west is actually northwest. Every correction therefore carries a fixed angular bias, and instead of being eliminated the position error is pushed around tangentially, tracing a circle.

Common causes include calibrating on a reinforced balcony or a metal car roof, rebar or buried utilities beneath the launch point, and magnetic accessories fitted to the aircraft (some camera gimbals and speakers contain permanent magnets). The teaching value of this case is that it shows clearly how heading error and position error couple—get the heading wrong and every position-based control decision is biased systematically to the same side. This idea reappears in the coordinate transforms of Chapter 10 and the navigation implementation of Chapter 29.

Case 6-2 From Interference to Information: Commercial Attempts at Indoor Geomagnetic Positioning

In the mid-2010s companies such as Finland's IndoorAtlas proposed commercial indoor positioning based on geomagnetic fingerprints, briefly regarded as the ideal "no hardware at all" solution—no beacons to install, no reliance on Wi-Fi, nothing but the magnetometer already in the phone. In guidance applications for shopping malls, airports, and exhibition halls the idea did demonstrate meter-level positioning.

Commercialization also exposed its practical limits, however: the fingerprint database had to be built by walking the site, and a large mall required tens of hours of

survey work; once the mall remodeled its units or replaced large metal display racks, part of the database had to be resurveyed; and the discriminating power of the geomagnetic field alone was insufficient, so in practice Wi-Fi, Bluetooth, and phone PDR still had to be fused for stable operation.

This case offers students two lessons. First, zero hardware cost is not zero deployment cost, and data collection and maintenance are often the real barrier—exactly the conclusion reached for Wi-Fi fingerprint positioning in Chapter 4. Second, in a real product it is rare for any single technology to carry the requirement alone, and fusion is very nearly the inevitable answer.

Case 6-3 A Magnetic Field Map of the Classroom: Drawing the Rebar

A demonstration experiment highly popular in both university and high school classrooms: have students carry a phone with a magnetometer app slowly across the classroom floor and along the walls, watching the field strength reading change.

The result tends to impress. In the open center of the room the reading holds steady near 45 μT; step close to a column or a load-bearing wall and it may jump instantly above 60 μT; hold the phone against the metal legs of a computer desk or the iron frame of a blackboard and the reading may double or plunge. Plotting the measured values by position as a color map makes an invisible map of the rebar emerge—the skeleton of the building developed by the magnetic field.

This one experiment accomplishes three teaching aims at once: it lets students feel directly how weak and easily disturbed the geomagnetic field is (Section 6.1), it makes clear why an indoor compass cannot be trusted (Section 6.4), and at the same time it lets them witness the spatial stability and distinctiveness of those very disturbances,

which is the basis of geomagnetic fingerprint positioning (Section 6.3). One set of data explains both why the field cannot be used and why it can—which is the central conceptual turn of this chapter.

Hands-On Activities

Exercise 6-1 Magnetometer Calibration and a Geomagnetic Fingerprint Map of the Classroom

Purpose: to carry out a complete magnetometer calibration and measure the geomagnetic distribution of a classroom, experiencing at the same time the dual identity of the geomagnetic field as a heading sensor and as a positional fingerprint. This exercise can be done with a phone alone; undergraduates may extend it to Arduino or ESP32 hardware.

Equipment

- A smartphone with a sensor app that displays raw three-axis magnetometer values (search for tools of the "magnetometer" or "sensor logger" kind).
- A classroom floor plan, a recording sheet, and colored pencils or spreadsheet software.
- (Optional, undergraduate) An ESP32 or Arduino with a nine-axis module such as the HMC5883L, QMC5883L, or MPU-9250.

Part One: Calibration and Heading Verification

1. Uncalibrated baseline test: hold the phone in the center of the classroom, rotate slowly in place through one full turn, and record the maximum and minimum of the total field strength ||m||. In theory the total strength should

stay constant through the rotation, so the measured range reflects the magnitude of the uncalibrated error.

2. Figure-eight calibration: following the app or system instructions, trace a figure eight in the air with the phone while flipping it through various angles for about 20 seconds, so that the sensor covers as many attitudes as possible.
3. Post-calibration verification: repeat step 1 and compare the range of total strength before and after (the standard deviation may be computed) to verify the calibration.
4. Tilt experiment: hold the phone level facing the blackboard and record the heading; then pitch the phone forward about 30 degrees while keeping the same facing, and observe how much the heading reading changes. Discuss the relation to tilt compensation in Section 6.2.
5. Interference experiment: bring the phone near a laptop, a metal chair leg, a fire hydrant, and a power outlet in turn, recording how much the total field strength and the heading change.

Part Two: A Geomagnetic Fingerprint Map

6. Mark grid points on the classroom floor at a 1.5- to 2-meter pitch (about 15 to 20 points) and number them on the floor plan.
7. Pause for 5 seconds at each grid point in a fixed attitude (phone level, facing the same direction), record the total field strength and the three axis components, and take three measurements per point and average them.
8. Draw the color map: fill in the floor plan using total field strength as the color value (blue for low, red for high) and observe whether the outlines of columns, load-bearing walls, or heavy metal equipment emerge.

9. Blind-test verification: one student is led with eyes closed to a grid point, measures the field, and the others match it against the database to infer the position; repeat 6 times and tabulate the success rate and mean error.
10. Advanced trajectory matching: have one student walk one circuit along the edge of the classroom while recording the field sequence continuously, and observe whether this "field curve" makes it easier to identify which stretch of the route was walked than a single-point value does.

Discussion Questions

1. What was the range of total field strength through one rotation before and after calibration? What does the difference tell you?
2. How large was the heading error produced by a 30-degree tilt? If a robot climbed a 15-degree slope without tilt compensation, what would go wrong in its navigation?
3. What was your blind-test success rate? At which grid points did most of the failures occur? Which limitation mentioned in Section 6.3 does this confirm?
4. Compare the discriminating power of single-point matching with that of trajectory matching, and explain why practical systems prefer the latter.
5. (Undergraduate) Judging from your classroom's magnetic field map, is this room suited to using a magnetometer to supply a robot's heading? If not, what method would you use instead (hint: see Section 6.6)?

Extended Challenge (Undergraduate)

Using an ESP32 with a nine-axis module, write a program that does the following: read the raw three-axis magnetometer and accelerometer values; implement hard-iron calibration (the min-max method) and soft-iron calibration by ellipsoid fitting;

implement tilt compensation and output the true heading (including the declination correction for Taiwan); and finally fuse gyroscope data with a complementary filter, comparing the smoothness and long-term stability of the heading output before and after fusion. This program can serve directly as the basis for IMU attitude estimation in Chapter 7.

Further Reading

1. NOAA NCEI, the official World Magnetic Model 2025 (WMM2025) website and technical report. (Includes an online declination calculator for looking up values anywhere in Taiwan.)
2. British Geological Survey, Geomagnetism teaching resources (geomag.bgs.ac.uk). (Fundamentals of geomagnetism and an account of the model.)
3. M. Kok, J. D. Hol, and T. B. Schön, "Using Inertial Sensors for Position and Orientation Estimation," Foundations and Trends in Signal Processing, 2017. (A complete tutorial on inertial and magnetic sensor fusion, freely available.)
4. B. Li et al., "How Feasible Is the Use of Magnetic Field Alone for Indoor Positioning?," IPIN, 2012. (A representative study of the feasibility of geomagnetic indoor positioning.)
5. J. Haverinen and A. Kemppainen, "Global Indoor Self-localization Based on the Ambient Magnetic Field," Robotics and Autonomous Systems, 2009. (An early classic paper on geomagnetic fingerprint positioning.)
6. J. F. Vasconcelos et al., "Geometric Approach to Strapdown Magnetometer Calibration in Sensor Frame," IEEE Transactions on Aerospace and Electronic

Systems, 2011. (A rigorous mathematical treatment of magnetometer calibration.)

7. The official Compass Calibration documentation of the PX4 and ArduPilot open-source flight stacks. (Practical guidance and troubleshooting for UAV compass calibration.)

From Direction to Inertia: Where This Chapter Stands

This chapter has given the robot an absolute sense of direction and also exposed its fragility: a reinforced wall, a running motor, even the change in current as the robot itself accelerates, is enough to make it point the wrong way. The magnetometer's update rate is also low, so it cannot capture rapid rotation.

The IMU of the next chapter fills exactly these two gaps. A gyroscope updates hundreds of times per second and is extremely sensitive to attitude change over short intervals, and it is wholly immune to magnetic disturbance; but its output must be integrated, and the error accumulates and drifts with time. Hence an elegant division of labor: the gyroscope handles the short term and the magnetometer the long term, and together with the direction of gravity from the accelerometer they form the attitude and heading reference system (AHRS)—exactly the arrangement set out in Table 6-1 of Section 6.6, and the problem that the attitude filtering of Chapter 7 treats formally.

By this point you should have caught the rhythm of Part II: each chapter introduces one sensor, says what it can give and under what conditions it lies, and then points out how the sensor of the next chapter fills that gap. When this chain ends at Chapter 9, every sense on the robot will be in place—and what Part III then teaches is the mathematics of fusing these mutually contradictory readings into the single most reasonable answer.

Summary

1. The geomagnetic field arises from the convection of liquid metal in the Earth's outer core; at only about 25 to 65 μT it is far weaker than the fields produced by everyday metal objects and is therefore extremely easy to disturb; it is described by three parameters—declination, inclination, and total intensity.
2. The north magnetic pole continues to drift toward Siberia, so precise navigation must rely on the World Magnetic Model (WMM); the current edition, WMM2025, was released in December 2024 and is valid through the end of 2029, with a high-resolution edition WMMHR2025 published alongside it.
3. An electronic compass must combine an accelerometer for tilt compensation—the inclination in Taiwan is about 35 degrees, so the vertical component is substantial and uncompensated tilt produces significant heading error; the declination correction is applied last to obtain the true heading.
4. The field distortion caused by indoor steel structure is interference for navigation yet a stable and distinctive positional fingerprint for localization; because a single point carries few dimensions of information, practice favors trajectory matching with a particle filter, giving accuracy of about 1 to 3 meters.
5. The geometric essence of magnetometer calibration is to restore a deformed point cloud to a true sphere: hard-iron error displaces the center (corrected by the offset vector b) and soft-iron error turns the sphere into an ellipsoid (corrected by the matrix A); calibration must be performed in the final installed configuration and away from environmental disturbance.
6. The UAV depends on the compass most and suffers most from it, and a compass error leads to the toilet bowl effect; indoor robots commonly

downweight the magnetometer and take heading from SLAM scan matching instead.

7. In the AHRS architecture the gyroscope handles everything in the short term, the accelerometer handles up and down in the long term, and the magnetometer handles the compass points in the long term; once SLAM is added, the magnetometer often retreats to initialization and anomaly monitoring.

In the next chapter we go deeper into the central sensor of Part II—IMU inertial navigation. How do an accelerometer and a gyroscope measure motion? Why does integration produce drift? And what are attitude filtering, IMU preintegration, and tightly coupled visual-inertial estimation? These will be important groundwork for the later chapters on visual SLAM and UAV navigation.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 7

IMU Inertial Navigation

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

The magnetometer of Chapter 6 gave the robot an absolute direction, but it is slow and fragile. The inertial measurement unit of this chapter supplies the opposite character: extremely fast and wholly immune to outside disturbance, yet badly wrong as soon as any length of time passes.

The IMU holds a peculiar place in this book. It is one of the sensors of Part II and also one of the protagonists of the visual-inertial SLAM of Part VI. More importantly, it is the first sensor in the book that can only be used by integration—which brings out the central contradiction of SLAM: dead reckoning is fast and continuous yet must drift, while observation is slow and discrete yet does not drift. The whole book, from the filtering of Chapter 11 and the graph optimization of Chapter 12 to the loop closure detection from Chapter 20 onward, addresses different faces of this one contradiction.

What this chapter asks you to attend to, then, is not the physics of MEMS but the law of error growth in Section 7.3 and the idea of preintegration in Section 7.5. The former explains why the IMU cannot be used alone, and the latter is the key engineering technique that lets modern visual-inertial systems run in real time.

Figure 7-0 How Chapter 7 Connects to the Rest of the Book

[Building on the previous chapter]

Chapter 6 The magnetometer: absolute and drift-free, but slow and easily disturbed

This chapter The IMU: fast and undisturbed, but bound to drift → the two complement each other as an AHRS

[This chapter is responsible for]

The measurement principles of specific force and the Coriolis force → attitude solution

→ integration drift (t^2 and t^3)

Attitude filtering: complementary → Kalman → EKF

Preintegration: letting a high-rate IMU enter a low-rate optimization problem

[Where it leads]

→ Chapter 11 This chapter's complementary filter and EKF formally subsumed into the Bayesian filtering framework

→ Chapter 12 Preintegration becomes an edge in a pose graph (foreshadowed in Section 7.5)

→ Chapter 18 LIO: LiDAR-inertial odometry

→ Chapter 23 The tightly coupled visual-inertial estimation of ORB-SLAM3 (the full version of Section 7.7)

→ Chapter 26 UAVs: the IMU is the heart of the flight controller, and its failure means a crash

Learning Objectives

The IMU is the busiest sensor on a robot—it reports the state of motion hundreds of times per second—and also the one most apt to lie. This chapter goes deep into the principles, limitations, and modern remedies of inertial navigation, material that directly supports the visual-inertial SLAM of Part VI and the UAV navigation of Part VII. After completing this chapter, you should be able to:

1. Explain the MEMS measurement principles of the accelerometer and the gyroscope, and explain why an accelerometer still reads about 9.8 at rest.
2. Define the three attitude angles roll, pitch, and yaw, and compute roll and pitch from an accelerometer.

3. Explain the cause of integration drift, and explain why position error grows with the square or even the cube of time.
4. Explain the basic ideas of the complementary filter and the Kalman filter, and compare the settings each suits.
5. Explain what problem IMU preintegration solves, and why it is the key technique in modern visual-inertial SLAM.
6. State the four major contributions of the IMU in visual-inertial SLAM, and explain the difference between loose and tight coupling.
7. Explain how ORB-SLAM3 completes IMU initialization by maximum a posteriori estimation, and why initialization requires motion excitation.

Core Concepts

Chinese Term	English Term	Explanation
慣性量測單元	IMU	A sensor module integrating an accelerometer and a gyroscope (and sometimes a magnetometer).
比力	Specific Force	What an accelerometer actually measures: motional acceleration minus gravitational acceleration.
科氏力	Coriolis Force	The physical effect by which a MEMS gyroscope measures angular rate.
姿態角	Roll / Pitch / Yaw	Roll, pitch, and yaw, describing a rigid body's orientation in three-dimensional space.
零偏	Bias	The fixed output error a sensor shows with zero input, which varies with temperature and time.

Chinese Term	English Term	Explanation
隨機游走	Random Walk	The random drift produced by integrating white noise, with error growing as the square root of time.
積分漂移	Integration Drift	Tiny errors accumulating continuously through integration, so the estimate departs gradually from the truth.
互補濾波器	Complementary Filter	Fuses gyroscope and accelerometer by complementary high-pass and low-pass filtering.
卡爾曼濾波器	Kalman Filter	The optimal linear recursive estimator under the assumption of Gaussian noise.
擴展卡爾曼濾波	EKF	A method applying the Kalman filter after linearizing a nonlinear system.
IMU 預積分	IMU Preintegration	Integrating the IMU measurements between two keyframes in advance into a single constraint, avoiding repeated computation.
緊耦合	Tightly Coupled	Solving the raw IMU measurements and the visual measurements within one optimization problem.
最大後驗估測	MAP	Finding the most probable state estimate given the observations; the theoretical basis of ORB-SLAM3's initialization.
尺度不確定性	Scale Ambiguity	A monocular camera cannot determine true size, so scale must be supplied by an IMU or an object of known size.

7.1 The Accelerometer, the Gyroscope, and the Magnetometer

At the core of an IMU (inertial measurement unit) are two kinds of sensor: an accelerometer measures linear acceleration and a gyroscope measures angular rate. The magnetometer introduced in Chapter 6 is often packaged with them to form what is called a nine-axis module. The division of labor among the three is exactly the one given in Section 6.6: the gyroscope handles everything in the short term, the accelerometer handles up and down in the long term, and the magnetometer handles the compass points in the long term.

1. The Accelerometer: A Mass on a Spring

The principle of a MEMS accelerometer can be understood from a very simple model: a tiny proof mass is suspended within the chip on elastic beams. When the chip accelerates, inertia displaces the mass relative to the frame, that displacement changes the gap between capacitor plates, and measuring the change in capacitance with a circuit converts it into acceleration. The whole structure is etched into a few square millimeters of silicon, at a cost as low as a few tens of NT dollars.

There is a key idea here that almost every beginner finds puzzling: set an accelerometer at rest on a desk and it reads not 0 but about 9.8 m/s² along the vertical. Why should something at rest have an acceleration?

A Clarification: An Accelerometer Measures Specific Force, Not Acceleration

The physical quantity an accelerometer actually measures is specific force, defined as:

a_measured = a_true - g

where a_true is the platform's true acceleration relative to an inertial frame and g is the gravitational acceleration vector.

Intuitively: an accelerometer measures how hard the spring is pushing the proof mass. At rest the spring must exert force against gravity to hold the mass up, so the reading is about 9.8;

in free fall the mass descends together with the housing and the spring bears no force at all, so the accelerometer reads 0 instead—which is the physical essence of weightlessness aboard a space station.

Conclusion: before using an accelerometer you must know the direction of gravity and subtract it to obtain the true motional acceleration. And the direction of gravity must in turn be determined from the attitude—

this circle, in which attitude and acceleration depend on each other, is one of the roots of the difficulty of inertial navigation.

2. The Gyroscope: Vibration and the Coriolis Force

A MEMS gyroscope measures angular rate. Its principle uses the Coriolis force: a proof mass inside the chip is driven in high-frequency reciprocating vibration, and when the whole assembly rotates the vibrating mass experiences a Coriolis force perpendicular to the direction of vibration, producing a detectable lateral displacement whose magnitude is proportional to the angular rate.

Note in particular that a gyroscope outputs angular rate (degrees turned per

second), not angle. To obtain an angle it must be integrated over time—and that is precisely the source of the drift problem (Section 7.3).

3. Grades and Specifications

IMUs divide by accuracy into several grades, with prices differing by factors of thousands:

Grade	Typical Bias Stability	Price Range	Typical Application
Consumer MEMS	Tens of degrees per hour	Tens to hundreds of NT dollars	Phones, toys, teaching modules
Industrial MEMS	A few degrees per hour	Thousands to tens of thousands	UAVs, robots, AGVs
Tactical grade	0.1 to 1 degree per hour	Hundreds of thousands	Guidance, surveying, self-driving cars
Navigation grade	Below 0.001 degree per hour	Millions and above	Aviation, submarines, spaceflight

Table 7-1 IMU Accuracy Grades and Applications

The MPU-6050, MPU-9250, ICM-20948, and BNO055 modules used in the teaching experiments of this book fall between consumer and industrial grade; UAV flight controllers (the Pixhawk family) mostly use industrial grade and often carry several units that check each other. Understanding the difference between grades matters: the same algorithm can dead-reckon independently for minutes on a tactical-grade IMU and may lose accuracy in seconds on a consumer module—which determines how much trust the system design may place in the IMU.

7.2 Attitude Estimation: Roll, Pitch, and Yaw

Attitude describes the orientation of a rigid body in space. The Euler angle representation customary in aviation has three angles, which you may remember by thinking of an aircraft:

- Roll (φ): rotation about the fore-and-aft axis—the wings rock up and down and the fuselage banks.
- Pitch (θ): rotation about the lateral axis—the nose rises or drops.
- Yaw (ψ): rotation about the vertical axis—the nose turns left or right, which is the heading angle discussed in Chapter 6.

Obtaining Roll and Pitch from the Accelerometer

When the platform is at rest or in uniform motion, what the accelerometer measures is the components of gravity on its three axes. Since gravity necessarily points toward the center of the Earth (straight down), comparing the three readings reveals how far the sensor is tilted:

$$\varphi = \operatorname{atan2}(a_y, a_z)$$

$$\theta = \operatorname{atan2}(-a_x, \sqrt{a_y^2 + a_z^2})$$

A Key Limitation: The Accelerometer Cannot Reach Yaw

There is a limitation that must be kept firmly in mind: an accelerometer cannot measure yaw. The reason is intuitive—lay the device flat on a desk and rotate it in place, and the direction of gravity never changes, so the three readings never change and the accelerometer is entirely insensible of the rotation. This is exactly why the magnetometer of Chapter 6 is indispensable, and why purely visual SLAM needs some other source for an initial heading.

A Second Limitation: Acceleration Contamination

The formulas above presuppose that the accelerometer measures gravity alone. But as soon as the robot accelerates, turns, or vibrates, the reading is the superposition of gravity and motional acceleration, and the attitude angles computed from it are badly distorted. A vehicle accelerating hard is misjudged by these formulas as nose-up—because a forward acceleration is, to the sensor, indistinguishable from the component of gravity after a tilt.

This limitation yields an important conclusion: an accelerometer is suited to supplying a long-term average attitude reference but cannot be used for instantaneous attitude estimation. The gyroscope is exactly the opposite—accurate in the short term, drifting in the long. Their complementarity is the entire motivation for the attitude filtering of Section 7.4.

Gimbal Lock in Euler Angles

Euler angles are intuitive but have a famous defect: when the pitch angle approaches ±90 degrees (nose straight up or straight down), the roll and yaw axes coincide and a degree of freedom is lost—gimbal lock. For UAV aerobatics or a robot arm this is fatal. Practical systems therefore mostly represent attitude internally with quaternions, which have no singularity and are computationally efficient, if harder to picture. A full account of quaternions is left to Chapter 10; for this chapter it is enough to build intuition with Euler angles.

7.3 The Problem of Integration Drift

The beautiful promise of inertial navigation is that an IMU alone lets a robot know its pose entirely on its own—no satellites, no landmarks, no external signal of any kind. The promise holds perfectly in theory and is almost impossible to realize alone in

practice. The reason is one word: integration.

Double Amplification by Integration

Deriving a pose from IMU readings requires integration:

$$\text{angular rate } \omega \xrightarrow{\text{(integrate once)}} \text{attitude angle}$$

$$\text{acceleration } a \xrightarrow{\text{(integrate once)}} \text{velocity} \xrightarrow{\text{(integrate again)}} \text{position}$$

Integration is an operation that only takes in and never lets out: every tiny error at every instant is added permanently into the result and never cancels itself. Worse, position requires two integrations, so the error is amplified twice over.

The Mathematics of Error Growth

Suppose the accelerometer has a constant bias error b (0.01 m/s², say, which is quite common on a consumer module). Then after a time t:

$$\text{velocity error} = b \cdot t$$

$$\text{position error} = (1/2) \cdot b \cdot t^2$$

Put numbers in to get a feel for it: with $b = 0.01$ m/s², the position error after 10 seconds is about 0.5 meters, after 60 seconds about 18 meters, and after 300 seconds as much as 450 meters. And this is only the accelerometer bias; gyroscope bias makes attitude error grow linearly with time, and once the attitude is wrong gravity is not fully removed, so the residual gravity component is taken for motional acceleration and integrated twice—a path whose position error grows as t^3, and the dominant error source in practice.

An Intuitive Account: How Attitude Error Poisons Position Estimation

Suppose the attitude estimate is off by just 1 degree. What the system takes

for vertical then differs from true vertical by 1 degree,

and after gravity is removed a horizontal component remains: $9.8 \times \sin(1°) \approx 0.17$ m/s².

This nonexistent acceleration, integrated twice, produces about 8.5 meters of position error in 10 seconds.

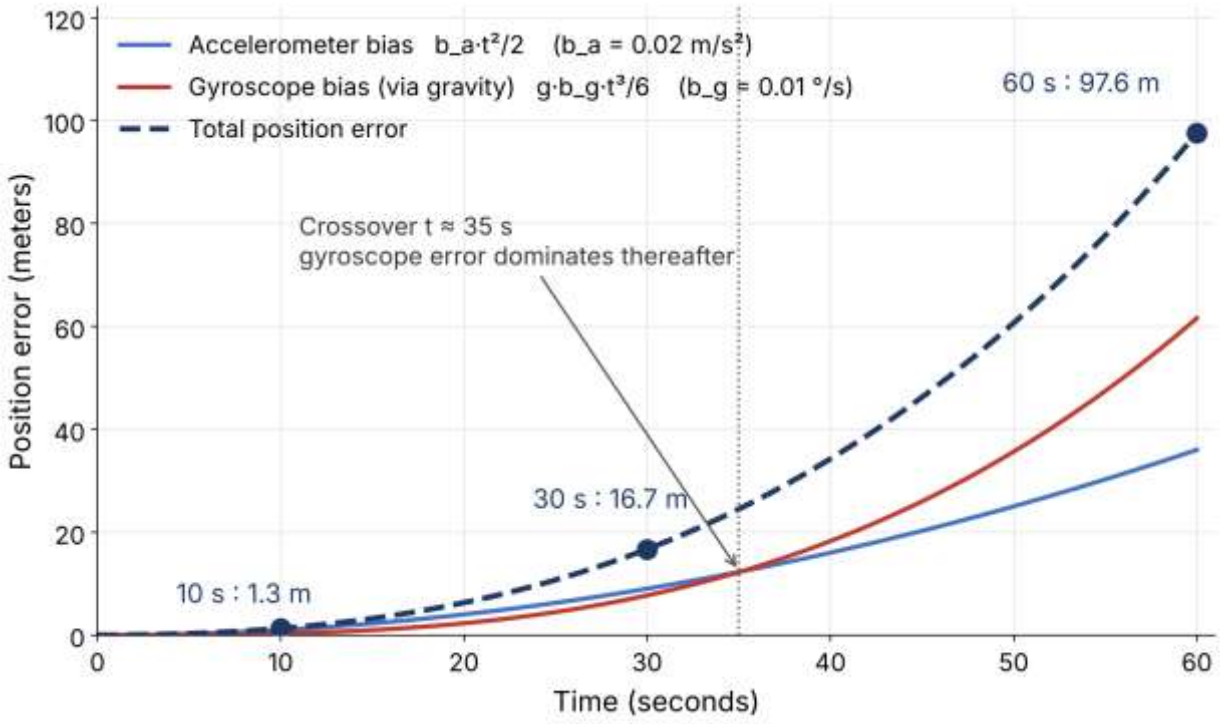
Conclusion: in inertial navigation, attitude accuracy matters more than acceleration accuracy. This is exactly why the attitude filtering of Section 7.4 is so important,

and why every practical system must bring in external information (magnetometer, GNSS, vision) to constrain the attitude.

Three Sources of Error

- Bias: a constant output offset with zero input. It is not a constant but varies with temperature (especially noticeably during warm-up) and wanders slowly with time. It is the principal error source and the quantity every filter and optimization algorithm sets out to estimate online.
- Noise: random high-frequency disturbance. After integration it becomes a random walk, with error growing as the square root of time—compared with the linear or quadratic growth of bias, this term is in fact not the main problem.
- Scale factor and axis non-orthogonality errors: departure of the actual sensitivity from its nominal value, and three axes not exactly perpendicular. Both can be reduced substantially by calibration beforehand.

Figure 7-1 Position Error Growth in Pure Inertial Dead Reckoning with a Consumer-Grade MEMS IMU



This figure makes the whole point of Section 7.3 with concrete numbers. It uses bias levels typical of consumer MEMS: an accelerometer bias of 0.02 meters per second squared (about 2 mg) and a gyroscope bias of 0.01 degrees per second. The two error curves have entirely different shapes—the error from accelerometer bias grows as t^2, while gyroscope bias produces t^3 growth by the route of an attitude error leaving gravity imperfectly removed.

Note the crossing point: at about thirty-five seconds the gyroscope-induced error overtakes the accelerometer's and dominates completely thereafter. The total error is about 1.3 meters at ten seconds, about 16.7 meters at thirty, and close to a hundred meters by sixty. One minute, one hundred meters—which is why nobody navigates on a consumer-grade IMU alone.

The figure also explains two things of great engineering importance. First, why nearly

all the cost of a high-end IMU goes into the gyroscope: because the t^3 term dominates in the end, reducing gyroscope bias pays far more than reducing accelerometer bias. Second, why the key parameter of a visual-inertial system is how often a visual observation arrives: pull the IMU back on course while the error is still small (within a keyframe interval of a fraction of a second, say) and this curve never reaches its right-hand half. That strategy is exactly how ORB-SLAM3 in Chapter 23 attains centimeter accuracy on a UAV.

An Order of Magnitude Worth Remembering

Attitude error $\propto t$ (linear), velocity error $\propto t^2$, position error $\propto t^3$—each further integration raises the power of time by one.

This law of integration amplifying error is not the IMU's alone. The wheel odometry of Chapter 13 and the visual odometry of Chapter 20 are all dead-reckoning sensors, and none escapes accumulated drift.

They differ only in rate of growth: wheel odometry roughly linear to quadratic, visual odometry roughly linear, and the IMU cubic—so the IMU drifts most fiercely and most needs frequent correction.

The Practical Conclusion

The right use of an IMU is not to navigate with it alone but to fill the gaps of other sensors. The usable dead-reckoning interval of a consumer-grade IMU is on the order of seconds, an industrial-grade unit reaches tens of seconds, and only tactical grade and above lasts for minutes. The central question of system design is therefore always this: before the IMU drifts beyond acceptable bounds, when will the next external correction arrive? Whether that correction comes from GNSS (Chapter 5), LiDAR scan

matching (Chapter 13), or visual features (Chapter 20), the reasoning is the same.

7.4 Attitude Filtering: The Complementary Filter, the Kalman Filter, and the EKF

Since the gyroscope is accurate in the short term and drifts in the long, while the accelerometer is accurate in the long term and contaminated in the short, the answer suggests itself: fuse the two and take the best of each. That is the task of attitude filtering.

1. The Complementary Filter: Simplest and Effective

The idea of the complementary filter is wonderfully direct: high-pass the integrated gyroscope result (trust its rapid changes), low-pass the angle computed from the accelerometer (trust its long-term average), and add the two with weights:

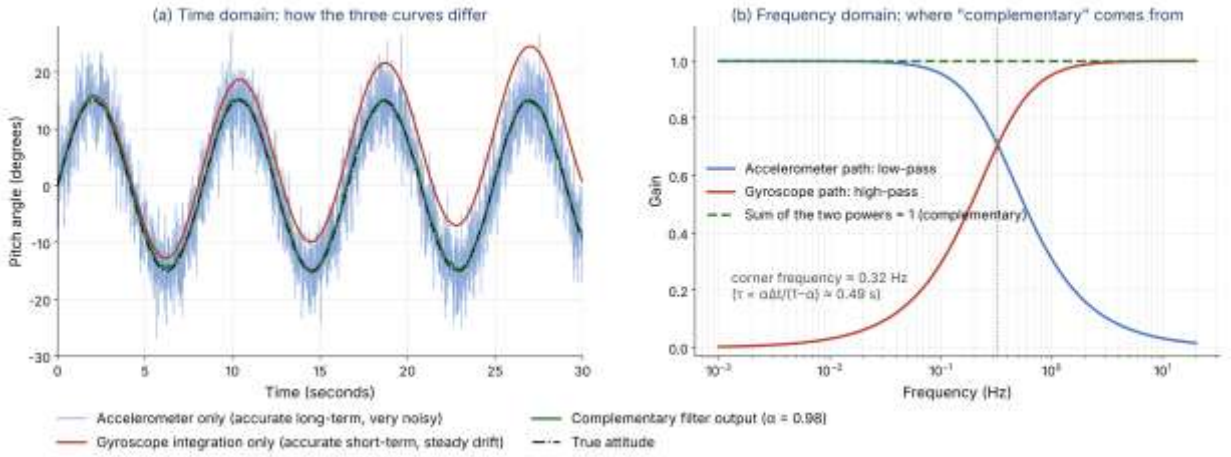
$$\theta = \alpha \cdot (\theta_{prev} + \omega \cdot \Delta t) + (1 - \alpha) \cdot \theta_{accel}$$

where α is usually taken between 0.95 and 0.99. A larger α means greater trust in the gyroscope, with a smoother output but weaker resistance to drift; a smaller α follows the accelerometer more quickly but is more easily disturbed by vibration.

- Advantages: a few lines of code, almost no computational cost, and a single parameter with a clear physical meaning—very well suited to Arduino-class microcontrollers and teaching experiments.
- Limitations: no statistical model of error, no ability to estimate bias, and no information about how uncertain the estimate is.

Figure 7-2 The Time-Domain Behavior and Frequency-Domain Meaning of the

Complementary Filter



The left panel is what you will plot with your own hands in Exercise 7-1. The light blue trace is the attitude computed from the accelerometer alone: centered on the truth over the long run but extremely noisy; the red trace is pure gyroscope integration: a beautifully smooth curve that departs steadily upward; and the green trace is the complementary filter output, nearly coincident with the black dashed line of true attitude.

The right panel explains where the word "complementary" comes from. The complementary filter in fact performs a division of labor in frequency: the accelerometer signal passes a low-pass filter (keeping only slow changes and rejecting vibration noise), the gyroscope signal passes a high-pass filter (keeping only fast changes and rejecting slow drift), and the responses of the two paths are exactly complementary, summing to unity at every frequency.

The corner frequency is set by α and the sampling period: the time constant is $\tau = \alpha\Delta t / (1 - \alpha)$. With $\alpha = 0.98$ and $\Delta t = 0.01$ second, τ is about 0.49 second and the corner frequency about 0.32 hertz. This gives you a physical basis for tuning: attitude changes slower than that are judged by the accelerometer, faster ones by the gyroscope. If the robot moves violently and acceleration contamination is severe, raise α (trust the

gyroscope longer); if the gyroscope is of poor quality and drifts quickly, lower α.

This idea of allocating trust according to signal characteristics recurs in more elaborate forms throughout the rest of the book: the Kalman gain of Chapter 11 is an α computed automatically from the uncertainties, the information matrix on each edge in the graph optimization of Chapter 12 is the matrix version of the same thing, and the judgment in the multi-robot systems of Chapter 27 of whether to trust oneself or a teammate is, in essence, the same question.

2. The Kalman Filter: The Optimal Linear Estimator

The Kalman filter raises the problem to the level of probability. It estimates not only what the attitude is but also how uncertain that estimate is (represented by a covariance matrix), and at each step it does two things:

1. Predict: propagate the attitude to the next instant using the gyroscope reading, while letting the uncertainty grow—because integration accumulates error.
2. Update: correct the prediction with the accelerometer observation. The strength of the correction is set by the Kalman gain: if the prediction is very uncertain and the observation noise small, the observation is trusted more; otherwise the prediction is.

This predict-update cycle is a miniature of the standard SLAM pipeline of Chapter 1 and the core mathematics that Chapter 11 unfolds in depth. A great advantage of the Kalman filter is that bias can be included in the state vector and estimated along with everything else—the system learns automatically how far this particular gyroscope is off today, which the complementary filter cannot do.

3. The Extended Kalman Filter (EKF): Handling Nonlinearity

The standard Kalman filter requires a linear system, but attitude kinematics is full of trigonometric functions and quaternion products and is nonlinear in essence. The EKF's remedy is to linearize the system by a first-order Taylor expansion about the current estimate (computing a Jacobian) and then apply the Kalman filter equations.

The EKF is the most widely used method of attitude estimation in robotics: UAV flight stacks (EKF2 and EKF3 in PX4 and ArduPilot), the robot_localization package in ROS, and sensor fusion in phones are nearly all EKFs behind the scenes. Its limitations are that linearization may be inaccurate when errors are large, and that it needs a sensible initial estimate to converge.

Method	Computational Demand	Estimates Bias	Provides Uncertainty	Suited To
Complementary filter	Very low	No	No	Teaching, microcontrollers, simple platforms
Kalman filter	Medium	Yes	Yes	Linear systems, simplified attitude models
EKF	Medium to high	Yes	Yes	UAV flight control; the mainstream choice in robotics
Mahony / Madgwick	Low	Partly	No	Common in open-source flight stacks and AHRS modules

Table 7-2 Comparison of Common Attitude Filtering Methods

A note in addition: the Mahony and Madgwick filters are practical schemes between the complementary filter and the Kalman filter, representing attitude by

quaternion and correcting by gradient descent or proportional-integral control; they are computationally light yet perform respectably, are extremely common in open-source flight stacks and AHRS modules, and make a good first stop for more advanced implementation.

7.5 The Concept of IMU Preintegration

This section enters the key technique of modern visual-inertial SLAM. The material is comparatively advanced; a high school course may take away only the intuition of what problem it solves, while undergraduates should understand its design logic.

The Starting Point: A Mismatch of Rates

A camera in visual SLAM runs at about 20 to 30 frames per second, while an IMU runs at 200 to 1,000 samples per second. That means several tens to over a hundred IMU measurements lie between two consecutive images. How is this mass of high-rate data to be turned into a single constraint the back-end optimizer can use?

The Fatal Flaw of the Naive Approach

The most intuitive approach is to integrate the IMU measurements over that interval directly, obtain the change of pose from frame i to frame j, and add it as a constraint in the optimization.

But there is a serious problem here. Integrating acceleration requires first removing gravity, removing gravity requires knowing the attitude, and the attitude is tied to the pose estimate of frame i. During optimization the pose estimate of frame i is updated repeatedly—and every update requires recomputing the entire IMU integral. With over a hundred measurements and dozens of iterations, the computation becomes completely unmanageable in real time.

The Central Insight of Preintegration

The ingenuity of preintegration lies in expressing the integral in a form independent of the initial pose.

Concretely, instead of integrating the change of pose in the world frame, one integrates the relative motion in the body frame of frame i. These relative quantities depend only on the raw IMU readings and the bias estimates and are entirely independent of frame i's position and orientation in the world. However the pose of frame i is updated during optimization, the preintegration result needs no recomputation—only one simple coordinate transformation at the outermost level.

Figure 7-3 The Problem Preintegration Solves

[The naive approach]

The pose of frame i is updated → the direction of gravity changes → the whole IMU interval must be reintegrated → the computation explodes

[The preintegration approach]

Preintegrate the relative motion (independent of the initial pose) → computed once only

↓

However the pose of frame i changes during optimization, only one outer coordinate transformation is needed

↓

The computation falls sharply and real-time execution becomes possible

What Preintegration Produces

Preintegration between two keyframes produces three relative quantities and one uncertainty:

- Relative rotation ΔR: how far the body turned over the interval.
- Relative velocity change Δv: how much the velocity changed.
- Relative position change Δp: how far it moved in the initial body frame.
- The covariance matrix: how trustworthy this constraint is, which sets its weight in the optimization.

Handling the Bias

One more detail: the bias must be removed before integration, yet the bias is itself an unknown to be estimated. If the bias estimate changes during optimization, must the preintegration be recomputed after all? The remedy in preintegration theory is to compute at the same time the first-order partial derivatives (the Jacobians) of the preintegration result with respect to the bias, so that a small change in bias needs only a quick linear correction rather than reintegration.

Preintegration theory was introduced by Forster et al. in 2015 with a rigorous treatment of the manifold structure of the rotation group, and it is now a standard component of every mainstream visual-inertial system—VINS-Mono, OKVIS, and ORB-SLAM3 without exception. This also answers why the MIT VNAV course of Chapter 2 devotes so much space to optimization on manifolds: a rotation is not an ordinary vector, and handling it requires dedicated mathematical tools.

7.6 The Role of the IMU in Visual-Inertial SLAM

Why must visual SLAM add an IMU? The answer is not as simple as one more sensor being one more insurance policy—what the IMU supplies is several things that purely visual methods cannot do even in principle.

Four Major Contributions of the IMU

3. It supplies absolute scale: a monocular camera has an inherent scale ambiguity—the same image sequence could be large motion in a large room or small motion in a doll's house, and vision alone cannot tell. The IMU measures acceleration in physical units, and integrating it supplies true scale. This is the single most important value of an IMU in a monocular visual-inertial system.

4. It supplies the direction of gravity: the accelerometer's long-term average points along gravity, letting the system establish which way is down. This makes roll and pitch observable and lets the map be set upright—crucial for UAV control and AR applications.
5. It fills the gaps when vision fails: when rapid rotation blurs the image, the lens is briefly obscured, or the robot enters a white-walled or dark area, the visual front end loses track. The IMU can still dead-reckon usefully for a few seconds, carrying the system through the blank and back onto the track—which raises robustness substantially.
6. It supplies high-rate output: 30 frames per second is too slow for flight control. The IMU lets the system output pose at hundreds of Hz, meeting the needs of a real-time control loop.

What Vision Gives Back to the IMU

The partnership runs both ways. Vision supplies exactly what the IMU lacks most: drift-free observations of position and attitude. Through visual constraints the system can continuously estimate and compensate the IMU bias, bringing the drift nightmare of Section 7.3 under control. Their relationship can be summed up thus: the IMU gives vision scale and gravity, and vision gives the IMU calibration and constraint.

Loose Coupling and Tight Coupling

- Loosely coupled: the visual and inertial systems each compute a pose independently and the two results are then fused with weights. The advantages are modularity, ease of implementation, and the ability to reuse an existing visual SLAM system directly; the drawback is that information has already been compressed within each stage, so the raw measurements cannot be revisited at fusion time and accuracy is limited.

- Tightly coupled: the visual reprojection errors and the IMU preintegration constraints are placed in one optimization problem and pose, velocity, bias, and map points are solved together. The advantage is that all information is handled consistently within one probabilistic framework, with markedly better accuracy; the drawbacks are implementation complexity and computational cost.

Modern high-accuracy systems use tight coupling without exception. This again echoes the fusion discussion in Section 4.7 of Chapter 4 and in Chapter 5—by now you will have noticed that "tight coupling beats loose coupling" is a general principle running through this whole book: the later information is compressed, the better the optimum that can be found.

7.7 ORB-SLAM3 and Tight Coupling with the IMU

ORB-SLAM3 is the most complete open-source visual-inertial SLAM system available, developed by a team at the University of Zaragoza in Spain. It was the first system to support purely visual, visual-inertial, and multi-map SLAM at once while accommodating monocular, stereo, and RGB-D cameras and both pinhole and fisheye lens models. This section builds the overall picture; a full dissection of the system is left to Chapter 23.

The Core Innovation: Maximum a Posteriori Estimation Throughout

The most important contribution of ORB-SLAM3 is a tightly coupled visual-inertial system built entirely on maximum a posteriori (MAP) estimation—the IMU initialization stage included. This lets the system run robustly in real time indoors and out and in scenes large and small, with accuracy about two to ten times better than earlier methods.

Why IMU Initialization Is So Difficult

At start-up a cluster of unknowns must be determined at once: the true scale of the map, the direction of gravity, the accelerometer and gyroscope biases, and the initial velocity. These quantities are coupled to one another, and not all of them are immediately observable.

The initialization procedure of earlier systems (such as ORB-SLAM-VI) took about 15 seconds to complete, during which accuracy was poor, badly affecting robustness and the user's experience. ORB-SLAM3 reformulates initialization as an optimization problem—solving within the MAP framework for the most probable values of scale, gravity direction, bias, and velocity at once—which makes initialization far faster and more reliable.

Motion Excitation: A Necessary Condition for Initialization

There is a point here that must be known in practice: IMU initialization requires sufficient motion excitation. The reason is that if the platform moves at constant velocity in a straight line, or stands still, the accelerometer reads essentially nothing but gravity and the system cannot separate scale from bias—mathematically these quantities are said to be unobservable.

In practice, therefore, after starting visual-inertial SLAM one should first perform a stretch of motion with accelerations, decelerations, and rotations (hold the camera and move it a little forward and back and side to side while rotating it, for a few seconds) before the system can complete initialization. This is one of the most frequent problems students meet while implementing ORB-SLAM3 in Chapter 30: "why does initialization keep failing?" The answer is usually that there was not enough motion.

The IMU within the Three-Thread Architecture

- Tracking thread: extract ORB features from each frame and estimate the camera pose by minimizing reprojection and inertial error; in visual-inertial mode it also estimates velocity and the IMU biases.
- Local mapping thread: insert keyframes, cull redundant information, and run local bundle adjustment—which in inertial mode is visual-inertial BA, completing and continuously refining the IMU parameters by MAP estimation.
- Loop closing and map merging thread: detect loop closures and map overlap and perform global correction.

The Multi-Map System: Losing Track Is No Longer a Disaster

The second innovation of ORB-SLAM3 is its multi-map architecture (the Atlas). When the system loses track because visual information has been poor for a long stretch, it does not collapse but opens a new map and continues; later, on returning to a region it has visited, it merges the new and old maps seamlessly. This is extremely practical for a robot running for long periods—Chapter 23 discusses the mechanism in full.

Accuracy

On public datasets, the stereo-inertial mode of ORB-SLAM3 attains an average accuracy of about 3.5 centimeters on the EuRoC UAV dataset; on the TUM-VI room sequences, recorded by hand with rapid motion to represent AR and VR scenarios, accuracy reaches about 9 millimeters. The order of magnitude speaks for itself: a cheap camera plus a cheap IMU, under the right algorithm, can attain surprising accuracy—and that is exactly what this book hopes you will verify with your own hands.

Formulations and Procedures

1. The Basic Kinematic Equations of Inertial Navigation

Let the position in the world frame be p, the velocity v, and the attitude rotation matrix R; let the IMU measure angular rate ω and specific force a, with gravity vector g. The continuous-time update equations are:

$$\dot{R} = R \cdot [\omega]_{\times}$$

$$\dot{v} = R \cdot a + g$$

$$\dot{p} = v$$

where $[\omega]_{\times}$ denotes the skew-symmetric matrix of the angular rate. These three equations are the whole of inertial navigation—and precisely because they must be integrated twice, the drift problem of Section 7.3 arises. In discretized form for implementation:

$$R(k+1) = R(k) \cdot Exp(\omega \cdot \Delta t)$$

$$v(k+1) = v(k) + (R(k) \cdot a + g) \cdot \Delta t$$

$$p(k+1) = p(k) + v(k) \cdot \Delta t + (1/2) \cdot (R(k) \cdot a + g) \cdot \Delta t^2$$

2. Error Growth Caused by Bias

With accelerometer bias b_a and gyroscope bias b_g, the principal error terms grow as:

$$attitude\ error \approx b_g \cdot t \quad (linear)$$

$$velocity\ error \approx b_a \cdot t + g \cdot b_g \cdot t^2 / 2$$

$$position\ error \approx b_a \cdot t^2 / 2 + g \cdot b_g \cdot t^3 / 6 \quad (the\ cubic\ term\ dominates)$$

Note the t^3 term containing g in the third expression: gyroscope bias becomes, by the route of attitude error → gravity imperfectly removed → spurious acceleration, the

dominant error source in long-duration inertial navigation. This expression explains why nearly all the cost of a high-end IMU goes into the gyroscope.

3. The Complete Implementation Procedure of a Complementary Filter

Figure 7-4 The Attitude Estimation Flow of a Complementary Filter

① Read the IMU: three-axis acceleration a, three-axis angular rate ω

↓

② Compute attitude from the accelerometer: $\varphi_{acc} = \operatorname{atan2}(a_y, a_z)$, $\theta_{acc} = \operatorname{atan2}(-a_x, \sqrt{a_y^2 + a_z^2})$

↓

③ Integrate the gyroscope: $\varphi_{gyro} = \varphi_{prev} + \omega_x \cdot \Delta t$ (likewise for pitch)

↓

④ Complementary fusion: $\varphi = \alpha \cdot \varphi_{gyro} + (1-\alpha) \cdot \varphi_{acc}$ (α about 0.95 to 0.99)

↓

⑤ Yaw ψ: supplied by the magnetometer (Chapter 6), or integrated from the gyroscope with its drift accepted

↓

⑥ Output the attitude (φ, θ, ψ) and update φ_{prev} and θ_{prev} for the next round

4. The Cost Function of Tightly Coupled Visual-Inertial Estimation

A tightly coupled system places all information in one optimization problem,

whose cost function has the structure:

$$\min \Sigma \| \text{visual reprojection error} \|^2 + \Sigma \| \text{IMU preintegration error} \|^2 + \| \text{prior error} \|^2$$

The first term requires the projected position of a map point in the image to be as close as possible to the feature position actually observed; the second requires the change of pose between adjacent keyframes to agree as closely as possible with what IMU preintegration predicts; and the third retains the information of earlier optimizations. The variables to be solved for include the poses, velocities, and IMU biases of all keyframes together with the coordinates of the map points.

This is exactly the full form of the graph optimization of Chapter 12—for now you need understand only one thing: tight coupling means letting the error terms of different sensors contend with one another within a single expression, so that the answer found is the most reasonable one for all the observations together.

7.8 Contemporary Developments in Inertial Sensing: When the IMU Meets Deep Learning

The IMU is a field mature to the point of dullness—the physics of MEMS has not changed in decades, and the mathematics of preintegration largely settled between 2015 and 2017. Between 2024 and 2026, however, a new line of work appeared that students should know: treating IMU error by learning.

1. The Crux of the Problem: Bias Is Not a Constant

Recall from Section 7.3 that the principal source of IMU error is bias. The traditional approach treats bias as a slowly varying random walk and estimates it with a filter or an optimizer—which is what both the EKF of Section 7.4 and ORB-SLAM3 of Section 7.7

do.

Real bias behavior is more complex, however: it varies with temperature, supply voltage, and mechanical stress, and it is entangled with the sensor's own nonlinearities. Describing it with a simple random walk model is in essence a compromise. That is exactly where deep learning enters—if the variation of bias has structure, why not let a model learn it?

2. Learning Bias Dynamics: Modeling with Neural Differential Equations

A representative 2025 work proposes modeling the continuous-time dynamics of bias directly with a neural ordinary differential equation (neural ODE) rather than treating it as a random walk. The design contains several ingenuities worth noting.

First, training requires no ground truth for the bias—which cannot be measured directly at all; the researchers instead build the loss function from pose ground truth and let the model learn the bias indirectly. Second, because IMU kinematics is defined on a matrix Lie group (a concept formally introduced in Chapter 10), the authors extend the neural differential equation into a Lie-algebraic form so that the model output satisfies the geometric constraints of rotation by construction. Third, the discrete IMU measurements are interpolated into a continuous control input by cubic Hermite splines, and a hierarchical training strategy is adopted that takes the gyroscope first and the accelerometer second.

On the EuRoC and TUM-VI datasets (the two you will meet in Section 7.6 and in the exercises) and in physical experiments, the method clearly improved the error of pure IMU integration and the overall performance of visual-inertial odometry, and showed the ability to generalize across datasets.

3. A Foundation Model for Inertial Localization

Learning-based IMU methods have long shared one weakness: a model trained on a particular dataset fails on another platform. A 2025 CVPR work attempts to resolve this along foundation-model lines—first pretraining on more than a hundred hours of multi-platform data to acquire general knowledge of motion; then fine-tuning for a new task with low-rank adaptation (LoRA), with only about 1.1 million trainable parameters; and finally performing online test-time adaptation at deployment so the model keeps learning as it runs.

The result is about a thirty-six percent improvement in absolute trajectory error over models specialized to each case, while running in real time at two hundred frames per second on a robot. This three-stage structure of pretraining, fine-tuning, and online adaptation is just like the vision-language-action models to be introduced in Chapters 33 and 34, which suggests that the subfields of robotics are converging on a single methodology.

4. How This Line of Work Should Be Regarded

This book's position is pragmatic: learning-based methods are, at present, a way to improve the quality of the IMU's input rather than a replacement for the preintegration of Section 7.5 or the tightly coupled optimization of Section 7.7. The most valuable use in practice is to place the learned model at the very front—denoise the raw readings and estimate the bias more accurately first, then feed the existing geometric system.

This division of labor has two merits: the interpretability and mathematical guarantees of the geometric system are preserved, while the learned model is responsible only for what it does best, finding in data the regularities that are hard to model. As you will see in Chapter 32, this is currently the most mature pattern for integrating AI with SLAM.

A Reminder for Students

However powerful the model, the physical fact of Section 7.3 does not change: integration must amplify error, and the t^3 term will dominate in the end.

Learned denoising can lower the bias by an order of magnitude, which means shifting the curve of Figure 7-1 to the right—not flattening it.

What can actually flatten the curve is only a drift-free external observation: vision, LiDAR, GNSS, loop closure. That is the fundamental reason SLAM exists and the entire content of the last six parts of this book.

Case Studies

Case 7-1 Submarines and Missiles: The Original Battlefield of Inertial Navigation

The first large-scale applications of inertial navigation appeared aboard the submarines and intercontinental missiles of the Cold War. A submerged submarine receives no external signal whatsoever and must hold its course for weeks on pure inertial navigation; a missile in flight cannot rely on an external signal that may be jammed and likewise can only dead-reckon on its own.

These systems used navigation-grade components such as mechanical or ring laser gyroscopes—the size of a briefcase, weighing tens of kilograms, costing millions of dollars, with bias stability a million times better than a phone IMU. Even so, a submarine must still surface periodically or correct by other means—because the mathematics of Section 7.3 is merciless: however small an error, multiplied by a long enough time it becomes a large one.

This historical comparison is highly instructive: the MPU-6050 in your hand, costing a few tens of dollars, works on exactly the same physical principles as those astronomically expensive systems, differing only in accuracy grade. And so the emphasis of this book is not on buying a better IMU but on using cleverer algorithms and sensor fusion to extract the greatest value from a cheap one—which is the greatest contribution of the past two decades of SLAM research.

Case 7-2 The Phone's Pedometer and Automatic Screen Rotation

The phone in your pocket demonstrates both the applications and the limitations of inertial sensing every day. Automatic screen rotation relies on the accelerometer to determine the direction of gravity—which is why, lying in bed with the phone held flat, it so often rotates the wrong way: gravity is then perpendicular to the screen and the roll angle is mathematically unobservable (the limitation of Section 7.2 appearing in daily life).

The pedometer shows another line of thinking: it does not integrate acceleration to obtain position (which, as Section 7.3 explains, would quickly lose accuracy) but detects periodic peaks in the acceleration signal to count steps, then multiplies by an estimated stride length to get distance. The method neatly sidesteps integration drift—rather than integrate, recognize a pattern. This is the central idea of pedestrian dead reckoning (PDR) and is often paired with the Wi-Fi fingerprint positioning of Chapter 4 and the geomagnetic fingerprint positioning of Chapter 6.

The lesson for students is important: faced with a sensor's inherent limitation, there is a route besides using a better filter—use the sensor differently, and do not ask it to do what it cannot.

Case 7-3 The EuRoC Dataset: A Common Examination Hall for Visual-Inertial SLAM

The EuRoC MAV dataset, released by a team at ETH Zürich, was recorded with a micro aerial vehicle carrying a stereo camera and an IMU through a machine hall and an industrial plant, with high-accuracy ground truth from a motion capture system and laser scanning. It has become the standard benchmark of visual-inertial SLAM—nearly every paper in the field reports its score on EuRoC.

The dataset deliberately includes sequences from easy to hard: from slow, steady flight to rapid, aggressive maneuvers with visibly blurred images. This design lets researchers see clearly under what conditions an algorithm begins to fail. The average accuracy of about 3.5 centimeters attained by ORB-SLAM3's stereo-inertial mode is its score on this benchmark.

For students the value of a public dataset is this: with no UAV and no motion capture system, a single computer is enough to run the same experiments as the world's leading research groups and compare the results. The exercises of Chapter 30 use the EuRoC dataset directly—the most concrete practice of standing on the shoulders of giants.

Hands-On Activities

Exercise 7-1 Witnessing Integration Drift First-Hand

Purpose: to experience the drift of inertial navigation through actual measurement and computation, and to implement a complementary filter and compare the result before and after fusion. This is the single most important experiment in Part II of this book—it will give you a visceral sense of why SLAM is needed.

Equipment

- A smartphone (with an app that can record and export raw sensor data), or an ESP32 or Arduino with an MPU-6050 or MPU-9250 module.
- A computer (with a Python environment including numpy and matplotlib), a tape measure, and a flat desktop.

Part One: Static Drift Test

1. Lay the device completely still and flat on the desk, record 60 seconds of raw three-axis acceleration and angular rate, and export the data.
2. Compute the bias: take the mean of the first 10 seconds. In theory, at rest the angular rate should be 0 and the acceleration should be gravity alone, so the measured departure is the bias. Record the three-axis gyroscope bias (converted to degrees per second).
3. Integration experiment (bias not removed): integrate the 60 seconds of angular rate directly into angle. The device did not move at all—by how many degrees does the plotted angle depart?
4. Integration experiment (bias removed): subtract the bias computed in step 2 and integrate again, then compare the two curves. By how much did the error fall? How much remains? What causes what remains (hint: the three sources of error in Section 7.3)?
5. Position drift: remove gravity and integrate the acceleration twice to obtain position. With the device at rest for 60 seconds, how far did the computed "position" travel? Does it agree with the theoretical estimate of Section 7.3?

Part Two: Motion Test and Filtering

6. Push the device along the desk in a straight line for 1 meter and stop, recording data throughout. Compute the distance traveled by double integration, compare it with the 1 meter measured by tape, and compute the percentage error.
7. Tilt the device slowly to about 30 degrees and return it to level, repeating several times and recording the data.
8. In Python, compute three attitude curves separately—pure gyroscope integration, pure accelerometer computation, and complementary filter fusion ($\alpha = 0.98$)—and plot them on the same axes for comparison.
9. Vibration test: tap the desk lightly to create vibration and observe how the three curves differ in response—which one jumps violently? Which is smoothest? Which is most dependable in the long run?
10. Parameter experiment: set α to 0.90, 0.95, 0.98, and 0.999 in turn, compare the smoothness and the responsiveness of the output, and identify the value you consider best, with reasons.

Discussion Questions

1. After 60 seconds at rest, by how many meters did your computed "position" drift? What would happen to a robot navigating on that result?
2. The error fell markedly once the bias was removed, which shows that bias is the principal error source. But bias varies with temperature and time—what does this imply for the practice of calibrating once at power-on and leaving it at that?
3. In the 1-meter straight-line experiment, what was your percentage error? Did the error come mainly from the acceleration phase or the deceleration phase, and why?

4. Comparing the three attitude curves, explain how the respective strengths and weaknesses of the gyroscope and the accelerometer appear concretely in the plot.
5. (Undergraduate) Using the bias values you measured, apply the formulas of Section 7.3 to estimate how long it takes for the position error to reach 1 meter. What does that interval mean for robotic system design?

Extended Challenge (Undergraduate)

First: implement the Madgwick or Mahony filter (public reference code exists for both), represent attitude by quaternion, and compare its long-run stability with the complementary filter. Second: download one sequence of the EuRoC dataset, read its IMU data and ground-truth trajectory, implement pure inertial dead reckoning and compare it with the truth, and plot the growth of error with time to verify the t^2 and t^3 theory of Section 7.3. This experiment leads directly into the ORB-SLAM3 implementation of Chapter 30.

Further Reading

1. P. D. Groves, Principles of GNSS, Inertial, and Multisensor Integrated Navigation Systems, 2nd ed., Artech House, 2013. (The authoritative textbook on inertial navigation; Chapters 4 and 5 are the standard reference for IMU error models.)
2. M. Kok, J. D. Hol, and T. B. Schön, "Using Inertial Sensors for Position and Orientation Estimation," Foundations and Trends in Signal Processing, 2017. (A complete tutorial, freely available and with code examples; strongly recommended for undergraduates.)

3. C. Forster et al., "On-Manifold Preintegration for Real-Time Visual-Inertial Odometry," IEEE Transactions on Robotics, 2017. (The definitive paper on IMU preintegration theory and the principal basis of Section 7.5.)
4. C. Campos et al., "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM," IEEE Transactions on Robotics, 2021. (The principal basis of Section 7.7, dissected in full in Chapter 23.)
5. S. Madgwick, "An Efficient Orientation Filter for Inertial and Inertial/Magnetic Sensor Arrays," Technical Report, University of Bristol, 2010. (The original Madgwick filter document, with complete code.)
6. M. Burri et al., "The EuRoC Micro Aerial Vehicle Datasets," International Journal of Robotics Research, 2016. (The paper describing the EuRoC dataset.)
7. T. Qin, P. Li, and S. Shen, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," IEEE Transactions on Robotics, 2018. (Another classic tightly coupled visual-inertial system, worth reading alongside ORB-SLAM3.)

From Inertia to the Outside World: Where This Chapter Stands

The magnetometer and IMU introduced in Chapter 6 and this chapter are together called proprioceptive sensors—what they perceive is the robot's own state: which way I face, how fast I am turning, how great an acceleration I feel. They need no environment and can therefore work in any environment.

But they cannot see the world. The IMU does not know there is a wall ahead, and the magnetometer does not know whether this corridor has been walked before. The LiDAR of Chapter 8 and the camera of Chapter 9 belong to the other class—exteroceptive sensors, which measure the geometry and appearance of the

environment directly. The division of labor between these two classes is the basic architecture of any robotic perception system: proprioceptive sensors supply a continuous, high-rate but drifting estimate, and exteroceptive sensors supply a low-rate but stable correction.

In the next chapter we turn to LiDAR. Rather than guessing how far it has moved as an IMU does, it measures with a laser directly that the wall is three meters away. From the time-of-flight principle and the difference between 2D and 3D LiDAR to its role in cleaning robots, warehouse AGVs, and self-driving cars, Chapter 8 raises the curtain on the 2D SLAM exercises of Part IV.

Summary

1. An accelerometer measures specific force (motional acceleration minus gravity), so it reads about 9.8 at rest and 0 in free fall; a gyroscope measures angular rate using the Coriolis force, and its output must be integrated to obtain an angle.
2. An accelerometer can resolve roll and pitch from the direction of gravity but is entirely insensible of yaw, and it is contaminated by motional acceleration when the platform accelerates; Euler angles suffer gimbal lock as pitch approaches ±90 degrees, so practice mostly uses quaternions instead.
3. Integration drift is the fundamental limitation of inertial navigation: accelerometer bias makes position error grow as t^2, while gyroscope bias produces t^3 growth by the route of an attitude error leaving gravity imperfectly removed; attitude accuracy matters more than acceleration accuracy.
4. Attitude filtering fuses the gyroscope (accurate in the short term) with the accelerometer (accurate in the long term): the complementary filter is simple

and efficient but cannot estimate bias; the Kalman filter and EKF estimate bias and uncertainty together and are the mainstream choice in UAV flight control and robotics.

5. IMU preintegration integrates the high-rate measurements between two keyframes into relative quantities independent of the initial pose, so that nothing need be recomputed during optimization—the key technique that lets modern visual-inertial SLAM run in real time.
6. The IMU makes four major contributions in visual-inertial SLAM: absolute scale, the direction of gravity, filling gaps when vision fails, and high-rate output; vision gives back drift-free observations that correct the IMU bias. Tight coupling is more accurate than loose coupling.
7. ORB-SLAM3 performs tightly coupled visual-inertial SLAM on a maximum a posteriori basis throughout, IMU initialization included, with accuracy about two to ten times better than earlier methods; initialization requires sufficient motion excitation, without which scale and bias are unobservable.

In the next chapter we turn to a quite different sensor—LiDAR. Rather than guessing how far it has moved as an IMU does, it measures with a laser directly that the wall is three meters away. From the time-of-flight principle and the difference between 2D and 3D LiDAR to its role in cleaning robots, warehouse AGVs, and self-driving cars, Chapter 8 raises the curtain on the 2D SLAM exercises of Part IV.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 8

An Introduction to LiDAR

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

The magnetometer and IMU of Chapters 6 and 7 are proprioceptive sensors—they perceive the robot's own state but cannot see the world. From this chapter Part II turns to exteroceptive sensors: those that measure the geometry and appearance of the environment directly.

LiDAR is the most direct of them. It does not dead-reckon, integrate, or guess; it measures with a laser that there is something in that direction, three meters away. This character of measuring geometry directly makes it the protagonist of Parts IV and V of this book—from the occupancy grid map of Chapter 13 and Gmapping of Chapter 14 to Cartographer of Chapter 15 and the 3D point clouds and LOAM of Chapters 17 and 18, all built on the data introduced here.

The role of this chapter, then, is to lay a foundation of specifications and intuition. Having read it, you should be able to look at a LiDAR datasheet and judge at once whether that unit can see a chair leg, how fine the resulting map will be, and whether it will come to grief in a glass-curtain lobby. These judgments will be in daily use when you build maps with your own hands in Chapter 29.

Figure 8-0 How Chapter 8 Connects to the Rest of the Book

[Building on the previous chapters]

Chapters 6 and 7 Proprioceptive sensors: fast and continuous, but drifting

This chapter Exteroceptive sensors: slow and discrete, but drift-free → only together

can they eliminate accumulated error

[This chapter is responsible for]

Time-of-flight ranging → scanning → the LaserScan and PointCloud data formats

The physical meaning of the specifications: measurement range, angular resolution, scan rate, reflectivity

[Where it leads]

→ Chapter 13 LaserScan + pose → occupancy grid map (the first use of this chapter's data)

→ Chapters 14 and 15 Gmapping and Cartographer: the two mainstays of 2D LiDAR SLAM

→ Chapter 16 Cleaning robots: the full discussion of the height blind spot of Case 8-1 in this chapter

→ Chapters 17 and 18 3D point cloud registration and LiDAR odometry

→ Chapter 29 Mapping with a physical vehicle: where this chapter's judgment of specifications becomes practice

Learning Objectives

Every sensor in the previous chapters has been guessing—odometry guesses displacement, the IMU guesses motion, Wi-Fi guesses distance. The LiDAR of this chapter is different: it measures with laser light, directly, that there is something in that direction at that distance. This precise geometric measurement makes LiDAR one of the most important sensors in both 2D and 3D SLAM. After completing this chapter, you should be able to:

1. Explain the time-of-flight (ToF) ranging principle and why LiDAR can provide precise range measurement.

2. Distinguish the structural differences between 2D LiDAR and 3D multi-beam LiDAR, and the merits and drawbacks of mechanical and solid-state scanning mechanisms.
3. Describe the content of and difference between the LaserScan and PointCloud data formats.
4. Explain how angular resolution, measurement range, accuracy, and reflectivity affect LiDAR's performance in practice.
5. Give examples of the role of LiDAR in indoor mapping, warehouse AGVs, and self-driving cars.
6. Compare the strengths and weaknesses of LiDAR against the camera and the IMU, and explain why practical systems so often fuse all three.
7. Know the low-cost LiDAR platforms suited to teaching, in preparation for the 2D SLAM exercises of Part IV.

Core Concepts

Chinese Term	English Term	Explanation
光達	LiDAR	A sensor measuring distance with laser light; the name stands for light detection and ranging.
飛行時間	Time of Flight, ToF	Distance computed by multiplying the time from emission to the return of the reflection by the speed of light.
脈衝式測距	Pulsed / dToF	Emits a short pulse and measures the round-trip time directly; suited to long ranges.
調頻連續波	FMCW	Modulates the laser frequency continuously and can measure range and velocity at once.

Chinese Term	English Term	Explanation
雷射掃描	LaserScan	The data format of one revolution of range values output by a 2D LiDAR.
點雲	Point Cloud	The set of three-dimensional points output by a 3D LiDAR.
線數	Channels / Lines	The number of laser beams in the vertical direction of a 3D LiDAR, such as 16, 32, 64, or 128.
視野角	Field of View, FoV	The angular range a LiDAR can scan, horizontal and vertical.
角解析度	Angular Resolution	The angular interval between adjacent measurement points; the smaller, the denser.
反射率	Reflectivity	The proportion of laser light a surface reflects, affecting detectable range and signal strength.
固態光達	Solid-State LiDAR	A LiDAR without a large rotating mechanism, including MEMS, flash, and OPA architectures.
多普勒效應	Doppler Effect	The physical phenomenon by which FMCW LiDAR measures a target's radial velocity directly.

8.1 The Basic Principle of LiDAR: Time of Flight and Scanning

LiDAR stands for light detection and ranging. Its central idea descends directly from the GNSS of Chapter 5 and the UWB of Chapter 4: transmit a signal, measure the round-trip time, and multiply by the signal's speed to obtain distance. The only differences are that LiDAR uses laser light, over far shorter distances and with far greater accuracy.

1. Pulsed Time of Flight (dToF)

The most intuitive ranging method is direct time of flight (dToF): emit an extremely short laser pulse, time the interval t from emission to the return of the reflection, and the distance is:

$$d = (c \times t) / 2$$

where c is the speed of light and the division by 2 accounts for the light having traveled out and back. The formula is simple but the demand on timing accuracy is extreme—at the speed of light 1 nanosecond equals 30 centimeters, so centimeter accuracy requires timing to about 0.07 nanosecond. This is precisely the technical barrier of a LiDAR chip and why it is more expensive than ultrasonic or infrared ranging. The advantages of dToF are that energy is concentrated in a short pulse and long ranges can be measured, which makes it the mainstream for long-range automotive LiDAR.

2. Frequency-Modulated Continuous Wave (FMCW)

Another increasingly important method is frequency-modulated continuous wave (FMCW): rather than a pulse, a laser is emitted continuously whose frequency varies linearly with time (a chirp), and the returning signal is mixed with the signal being

emitted at that moment to measure the frequency difference (the beat frequency). That difference encodes distance and velocity at once—the most fascinating property of FMCW.

The greatest advantage of FMCW is that the Doppler effect lets it measure the radial velocity of every point directly, with no need to compare successive measurements as a pulsed system must; and because it uses coherent detection it can reject ambient light and other LiDARs almost entirely. The price is a narrow-linewidth laser source, which is difficult to manufacture and comparatively expensive, so FMCW is used mainly in premium automotive LiDAR and in research today. Recent advances in silicon photonics have realized FMCW systems with about 120 meters of detection range and 0.3 meters of accuracy on a micron-scale platform, and these are regarded as an important direction for the next generation.

3. Why Scanning Is Necessary

A single laser beam measures the distance in only one direction at a time—that is merely a point. To build the outline or the three-dimensional shape of an environment, the beam must be swept across different directions, which is the task of the scanning mechanism. The manner of scanning determines a LiDAR's form and price and is the subject of Section 8.2.

A Clarification: The Differences Among LiDAR, Radar, and Ultrasound

All three work on the same principle (emit a signal, measure the round-trip time) and differ in the wave used:

• LiDAR: laser light, short in wavelength and excellent in directionality, reaching

centimeter to millimeter accuracy and high resolution; its weaknesses are glass, mirrors, rain, and fog.

• Radar: radio waves, highly penetrating, unaffected by rain and fog, and able to measure speed; its weaknesses are low resolution and difficulty in resolving an object's shape.

• Ultrasound: sound waves, extremely cheap and dependable at short range; its weaknesses are small range, low accuracy, and susceptibility to wind and sound-absorbing materials.

Self-driving cars often use all three together, precisely for their complementarity—entirely consistent with the sensor fusion idea this book stresses repeatedly.

8.2 2D LiDAR, 3D LiDAR, and Multi-Beam Lasers

By scanning dimension, LiDAR divides into 2D and 3D; by scanning mechanism, into mechanical and solid-state. These two axes of classification determine a unit's capability, form, and price.

1. 2D LiDAR: Scanning One Plane

A 2D LiDAR rotates a single beam through one turn in a horizontal plane (commonly a 360-degree or 270-degree field of view), measures the distance in every direction in that plane, and outputs one revolution of range values. It is like a laser ruler sweeping horizontally across space at a fixed height.

The advantages of 2D LiDAR are a small data volume, light computation, and an affordable price (teaching models cost a few thousand to ten-odd thousand NT dollars),

which make it standard equipment on cleaning robots, teaching vehicles, and indoor AGVs, and the protagonist of the Gmapping and Cartographer exercises of Part IV of this book. Its inherent limitation is that it sees only one horizontal slice—mounted to scan at a height of 20 centimeters, it cannot see a tabletop at 30 centimeters and cannot detect an overhead bar or a low threshold. This limitation recurs throughout Section 8.5 and the discussion of cleaning robots in Chapter 16.

2. 3D LiDAR: Multiple Beams and Scanning

Obtaining three-dimensional information requires measuring in the vertical direction as well. 3D LiDAR achieves this chiefly in two ways:

- Multi-beam: several sets of laser emitters and receivers are stacked vertically, each set covering one elevation angle, and all rotate and scan together. Common specifications are 16, 32, 64, and 128 beams—the more beams, the denser the vertical sampling and the finer the three-dimensional reconstruction, and the higher the price. This is the mainstream for self-driving cars and autonomous patrol robots.
- Single-beam vertical scanning: an oscillating mirror or rotating prism makes a single beam (or a few beams) sweep vertically as well, covering the vertical field of view with fewer laser components and so reducing cost.

3. Mechanical versus Solid-State

Traditional LiDAR rotates the whole optical assembly or a mirror with a motor and is called mechanical. Its field of view is large and the technology mature, but its drawbacks are clear: bulk, rotating parts subject to wear, limited life in a vibrating environment, and high cost.

Solid-state LiDAR seeks to reduce or eliminate large rotating mechanisms, chiefly

in three architectures:

- MEMS: a micromirror on a chip deflects the laser beam to scan line by line. Strictly speaking this is semi-solid-state, since the tiny mirror still moves; it is currently the solid-state approach with the largest shipments.
- Flash: like a camera flash, the whole scene is illuminated at once and an array of SPAD or APD sensors receives all the returns, with no moving parts at all; the advantage is the absence of scanning latency, and the limitation is that spreading the illumination energy shortens the range.
- OPA (optical phased array): adjusting the phase of many waveguide emitters steers the beam purely electronically within microseconds, with no mechanical motion whatever; it is the most advanced technically, though its readiness for volume production is still advancing.

The overall trend in solid-state LiDAR is smaller size, greater vibration tolerance, and lower cost, and it is moving steadily from premium automotive use into robotics and consumer applications. For the reader of this book, understanding this progression from a mechanical present to a solid-state future helps in reading the bewildering array of products on the market.

8.3 LiDAR Data Formats: LaserScan and PointCloud

What a sensor measures must finally be handed to a SLAM algorithm in a particular format. LiDAR has two standard formats in the ROS ecosystem, corresponding to 2D and 3D.

1. LaserScan: The Data Format of 2D LiDAR

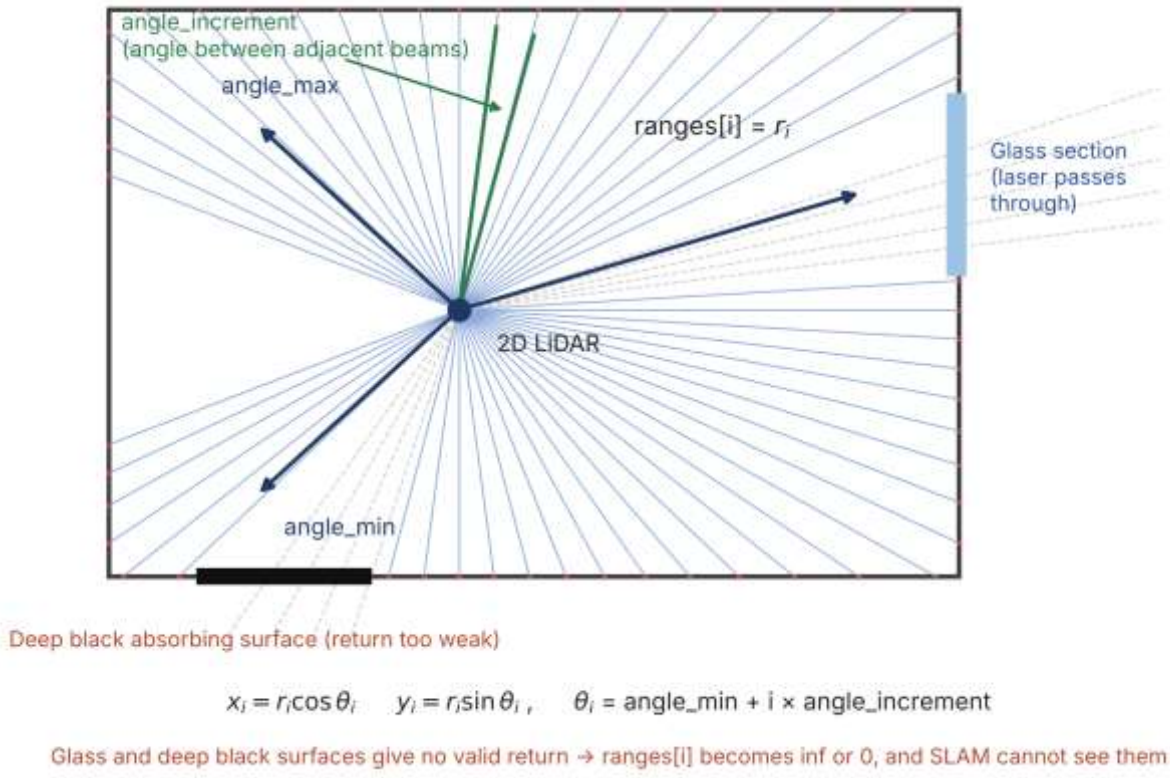
A 2D LiDAR outputs one revolution of range values, represented in ROS by the

sensor_msgs/LaserScan message. At its core is an array of ranges together with a few parameters describing the scan:

- angle_min, angle_max: the starting and ending angles of the scan.
- angle_increment: the angular interval between adjacent measurement points (that is, the angular resolution).
- range_min, range_max: the nearest and farthest trustworthy measurement distances.
- ranges[]: the array of range values—the body of the data, each element being the distance measured at the corresponding angle.
- intensities[]: the array of return intensities (optional).

Because the angle increases regularly, LaserScan need not store an angle for every point: the range array plus a starting angle and an angular interval suffices to recover each point's direction. This compact representation makes 2D data very light—one revolution is usually only a few hundred to a thousand range values. To plot a point in a coordinate frame requires only the simple conversion from polar to rectangular coordinates (the formulation in Section 8.5).

Figure 8-1 The Scan Geometry of 2D LiDAR and the LaserScan Data Structure



This figure puts every field of the LaserScan message back into its geometric meaning. The LiDAR sits at the origin and emits a beam starting at angle_min and every angle_increment thereafter until angle_max; each beam returns one range, stored in order in the ranges array. So the number ranges[i] carries no directional information in itself—the direction must be recovered from the index i: θ_i = angle_min + i × angle_increment.

This is one of the most frequent beginner's mistakes: plot the ranges array directly and you get a contorted curve rather than the shape of the room. The two conversion formulas at the bottom of the figure must be applied to turn polar (r, θ) into rectangular (x, y) before the point cloud looks like a room. You will do this with your own hands in Exercise 8-1, and the act of marking these points into cells in Chapter 13 is the first step of mapping.

The figure also deliberately shows two failure modes. The glass section of the right-hand wall lets the laser pass straight through with no return; the deep black light-absorbing material on the lower left wall absorbs nearly all the light and the return is too weak to read. The result is the same in both cases—the range value in that direction becomes infinite or zero, and in SLAM's eyes there is nothing there. An opening that should not exist appears in the map, and the robot may walk straight into a glass door.

Three Practical Countermeasures to the Glass Problem

1. Add an ultrasonic or infrared proximity sensor dedicated to detecting glass—the standard practice in cleaning robots and service robots.

2. Apply visible marking tape or an AprilTag to the glass (Section 4.6), solving recognition for people and robots at once.

3. Annotate a virtual wall on the map by hand (Nav2's keepout zone, implemented in Chapter 29) so that path planning avoids it outright.

Remember one principle: what the sensor cannot see, no algorithm however powerful can recover—an iron law that still holds when AI-assisted navigation is discussed in Chapter 32.

2. PointCloud: The Data Format of 3D LiDAR

The data of a 3D LiDAR is a great many points in three-dimensional space, represented in ROS by the sensor_msgs/PointCloud2 message. Unlike LaserScan, every point stores its own three-dimensional coordinates (x, y, z) directly, with optional additional fields for return intensity, beam index, time stamp, and the like.

The data volume of a point cloud is far larger than that of a LaserScan: a 64-beam LiDAR can produce more than a million points per second. This raises two engineering

issues—heavy transmission and storage requirements, and the downsampling usually needed before further processing (the voxel grid of Chapter 17). Point cloud processing has a dedicated open-source library, PCL (the Point Cloud Library), used extensively in Part V.

Characteristic	LaserScan (2D)	PointCloud2 (3D)
Data dimension	One revolution of range values (planar)	A set of three-dimensional points (volumetric)
Volume per message	Hundreds to a thousand points	Tens of thousands to millions of points
Point representation	Range + regular angle	x, y, z stored directly
Computational burden	Light	Heavy; usually needs downsampling
Typical source	A rotating 2D LiDAR	A multi-beam or solid-state 3D LiDAR
Use in this book	2D SLAM in Chapters 13 through 16	3D SLAM in Chapters 17 and 18

Table 8-1 Comparison of the LaserScan and PointCloud2 Data Formats

8.4 LiDAR Resolution, Angular Resolution, Measurement Range, and Reflectivity

Selecting or evaluating a LiDAR requires reading its datasheet. This section explains the four most critical parameters, which directly determine the quality of a SLAM map.

1. Measurement Range and Accuracy

- Measurement range: the nearest and farthest measurable distances. Teaching-grade 2D LiDARs commonly span 0.15 to 12 meters, while long-range automotive units reach beyond 200 meters.
- Accuracy: how close a measurement is to the true distance, commonly specified as ±2 to ±3 centimeters.
- Precision: the consistency of repeated measurements of the same target. Accuracy and precision are two different things—a ruler that always reads 5 centimeters long is inaccurate yet may be very precise, and a systematic bias can be calibrated out.

2. Angular Resolution: How Fine the Map Can Be

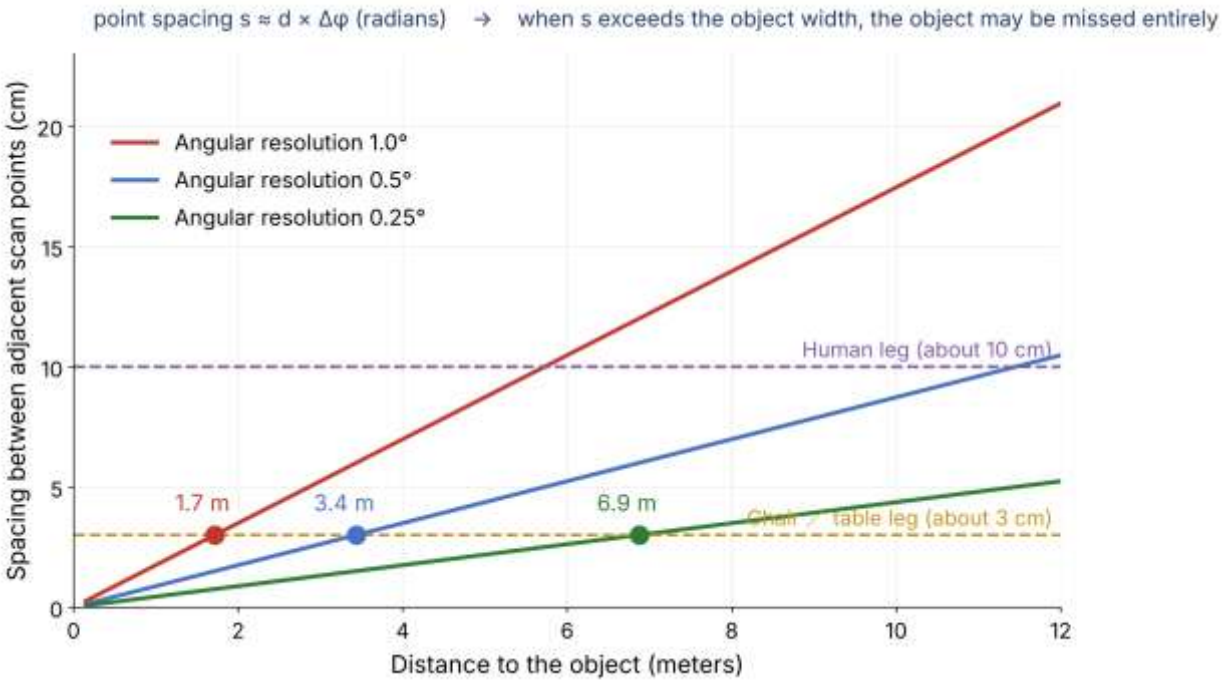
Angular resolution is the angular interval between adjacent measurement points, and it determines how fine a structure the map can represent. A key geometric fact lies here: for the same angular resolution, the farther away, the greater the actual spacing between adjacent points.

$$\textit{spacing between adjacent points} \approx \textit{distance} \times \textit{angular resolution (in radians)}$$

For example: at an angular resolution of 1 degree and a distance of 5 meters, adjacent points are about 8.7 centimeters apart—which means a chair leg 5 centimeters wide at 5 meters may fall entirely between two beams and go completely undetected. This explains why small distant objects are easily missed by LiDAR and is the key to the discussion in Section 8.5 and in Chapter 16 of why a cleaning robot runs into table legs.

Figure 8-2 The Relation Among Angular Resolution, Distance, and Detectable

Object Size



Angular resolution is the number on a datasheet most easily skipped over and most decisive of map quality. Its effect is simple: the spacing between adjacent scan points grows linearly with distance, $s \approx d \times \Delta\varphi$. But put concrete numbers in and the conclusion is striking.

Take a chair leg 3 centimeters wide. An entry-level LiDAR with 1-degree angular resolution is guaranteed to catch at least one point on it only within 1.7 meters; 0.5 degree extends that to 3.4 meters; and 0.25 degree to 6.9 meters. In other words, in the same classroom with the same chair, changing the LiDAR may turn four clear points on the map into nothing at all.

This explains several things seen in practice: why maps built with a cheap LiDAR have straight walls but fragmented furniture; why cleaning robots so often run into chair legs (the height blind spot of Case 8-1 is only one reason, and angular resolution is another); and why the occupancy grid map of Chapter 13 needs accumulated observations—a small object missed in one scan is recovered by moving closer and scanning again.

When purchasing or reading a datasheet you may treat this figure as a lookup table: decide first how small an object you must detect and at what distance, then work back to the angular resolution required. This habit of inferring specifications from requirements is the same engineering discipline as the indoor positioning selection procedure of Section 4.7.

3. Scan Rate

The scan rate (10 Hz means ten revolutions per second) affects the quality of mapping while the robot moves. If the rate is too low and the robot moves quickly, the robot has traveled some distance between the start and the end of one revolution, producing motion distortion—a problem that must be handled explicitly in the 3D LiDAR SLAM of Chapter 18.

4. Reflectivity: An Overlooked but Critical Variable

LiDAR depends on the target to reflect the laser back, so the target's reflectivity is crucial. A white wall is highly reflective and easy to detect; a black object absorbs most of the laser and reflects weakly, which shortens the detectable range sharply—a LiDAR rated at 12 meters may manage only a few meters against a black sofa. More troublesome still:

- Mirrors and glass: the laser is reflected elsewhere rather than back, so the LiDAR sees through the glass or measures a virtual image in the mirror, producing errors in the map—one of the most common causes of failure in indoor LiDAR SLAM.
- Highly reflective objects: metal and retroreflective signs may produce excessively strong returns or multiple reflections.

- Liquid surfaces: water and polished floors may reflect specularly and cause missed measurements.

The maximum range on a LiDAR datasheet is therefore usually quoted under a particular reflectivity (10% or 90%, say)—be sure to note that premise when reading specifications, or actual performance will fall well short of expectation.

8.5 LiDAR in Indoor Mapping, Warehouse AGVs, and Self-Driving Cars

1. Indoor Mapping and Service Robots

2D LiDAR is the mainstay of indoor mapping. Cleaning robots, food-delivery robots, and guide robots mostly build occupancy grid maps with one horizontal 2D LiDAR and Gmapping or Cartographer (Part IV). Indoor environments are clearly structured (walls, doors, corridors), and LiDAR's precise geometric measurement builds crisp, stable maps—something visual methods can hardly match against white walls or in dim light. But the glass partitions, mirrors, and single-height slice described above also defeat pure 2D LiDAR repeatedly in modern glass-curtain buildings, so in practice ultrasonic or infrared sensors are often added to fill the gap.

2. Warehouse AGVs and Factory Automation

The warehouse automated guided vehicle (AGV) is one of LiDAR's most mature industrial applications. Besides SLAM navigation, LiDAR doubles as a safety guard—many industrial units carry safety certification and can trigger slowing or stopping the instant a person enters a hazardous area. A factory environment is comparatively controlled (level floors, steady lighting, a known layout), which lets LiDAR achieve its greatest reliability. This echoes the idea of the Amazon case in Chapter 4: the more

controllable the environment, the simpler the technology selection can be.

3. Self-Driving Cars

3D LiDAR is the core sensor of most self-driving cars (Tesla's purely visual approach being the famous exception). It provides precise three-dimensional structure for obstacle detection, determination of drivable area, and localization by matching against a high-definition map (the urban canyon case of Chapter 5). The demands a self-driving car places on LiDAR are the most severe of all: long range (to see far ahead at highway speed), high resolution (to tell a pedestrian from a utility pole), a high update rate, and strict automotive-grade reliability. That is why automotive LiDAR still carries a high unit price (a few hundred US dollars each) and why it is evolving aggressively toward solid-state and FMCW designs.

4. Other Emerging Applications

- Surveying and spatial scanning: backpack or handheld LiDAR scans buildings, historic sites, and construction sites to produce high-accuracy 3D models (Chapters 15 and 17).
- Agriculture and forestry: UAV-borne LiDAR penetrates the canopy to measure tree height and terrain.
- Quadrupeds and patrol: 3D LiDAR tightly coupled with an IMU for autonomous patrol indoors and out without GNSS (Chapter 18).

8.6 Comparing LiDAR with the Camera and the IMU

Placing LiDAR in the panorama of Part II's sensors shows its position more clearly. Table 1-1 of Chapter 1 already gave an overview; here we make a deeper comparison for the tasks of mapping and localization.

Item of Comparison	LiDAR	Camera	IMU
Measures directly	Distance (geometry)	Brightness / color	Acceleration / angular rate
Depth information	Direct and precise	Must be inferred (no scale in monocular)	None
Affected by light	Almost not at all (active light source)	Severely (depends on ambient light)	Not at all
Semantic information	None (shape only)	Rich (objects recognizable)	None
Data volume	Medium to large	Large	Small
Cost	Medium to high	Low	Low to medium
Principal weakness	Glass and mirrors, rain and fog, no semantics	Light, no scale, no depth	Drift

Table 8-2 LiDAR, Camera, and IMU Compared on Mapping and Localization Tasks

The Logic of Their Complementarity

The table shows clearly why the three are naturally complementary:

- LiDAR gives precise geometry and depth but cannot tell what a thing is, and it fears glass and mirrors.
- The camera gives rich semantics and color but no dependable depth or scale, and it fears darkness and backlighting.
- The IMU gives high-rate motion information to fill the gaps but drifts.

The most robust systems therefore often use all three: LiDAR SLAM builds a precise geometric skeleton, the camera supplies semantic labeling and loop closure

recognition, and the IMU compensates during LiDAR motion distortion and rapid movement. The LiDAR-inertial tight coupling (LIO) of Chapter 18 is the representative of LiDAR plus IMU, and the semantic SLAM of Chapter 32 the representative of LiDAR (or camera) plus AI.

LiDAR versus Vision: A Debate with No Standard Answer

In the self-driving field, whether LiDAR should be used at all was once a fiercely contested question. Advocates of pure vision argue that humans drive with eyes alone, and that cameras are cheap, carry semantics, and can read signs; advocates of LiDAR argue that the dependable depth it provides is essential safety redundancy and should not be sacrificed to save cost. The debate remains unsettled, but it offers an important lesson: sensor selection is never a purely technical question and involves cost, safety, regulation, and business strategy as well. As an engineer, understanding the boundaries of each sensor's capability is what allows a well-founded judgment in a concrete situation—which is the common purpose of all seven chapters of Part II.

8.7 Low-Cost LiDAR and Teaching Robot Platforms

The good news is that LiDAR has become cheap enough to enter the classroom. This section introduces LiDARs and platforms suited to teaching, in preparation for the exercises of Part IV.

1. Entry-Level 2D LiDAR

The spread of low-cost 2D LiDAR in recent years is what has allowed SLAM education to reach undergraduates and even high school students. Such units mostly use triangulation or low-cost ToF principles, scan 360 degrees with a range of about 8 to 12 meters, start at a few thousand NT dollars, and come with complete ROS drivers—

plug one in and it publishes LaserScan, ready to feed directly to Gmapping or Cartographer. Their accuracy and stability fall short of industrial models but are more than adequate for teaching.

2. Teaching Robot Platforms

- The TurtleBot family: the official standard teaching platform of ROS, carrying a 2D LiDAR and a complete software stack; it is the most widely used platform in SLAM teaching worldwide, with abundant tutorial material online. The exercises of Chapter 29 in this book take it as the principal reference.
- A self-built ROS vehicle: assembled from a differential-drive chassis, motor encoders, and a microcontroller with a low-cost 2D LiDAR and a single-board computer (a Raspberry Pi or Jetson Nano, say). The cost is lower and the learning deeper, which suits undergraduate projects—students handle every link themselves, from the TF frames of Chapter 10 to publishing odometry in Chapter 29.
- A simulation platform: no hardware at all. A virtual LiDAR vehicle can be placed in a simulation environment such as Gazebo or Foxglove to run complete SLAM experiments (Chapter 28). This lets high school courses and large classes give everyone hands-on work, and it is the entry route this book strongly recommends.

3. Advanced: Teaching with 3D LiDAR

Low-beam-count 3D LiDARs (16 beams, say) have also fallen sharply in price in recent years and, paired with an edge computing platform such as a Jetson, can already serve undergraduate 3D SLAM projects (Chapters 17 and 18) and university autonomous patrol research. For high school and introductory undergraduate work,

however, starting from 2D LiDAR or simulation remains advisable: build the core concepts of SLAM firmly first, then enter the world of 3D.

Whichever platform is chosen, this book's advice is the same: understand the pipeline in simulation first, verify it with cheap hardware next, and pursue accuracy and scale only last. Every exercise in Part IV marks out both a simulation route and a hardware route so that students with differing resources can all take part. In the next chapter we take leave of LiDAR's active ranging and turn to the sensor richest in information and most in need of mathematical treatment—the camera—laying the ground for the visual SLAM of Part VI.

Formulations and Procedures

1. Time-of-Flight Ranging

The basic ranging formula of pulsed LiDAR:

$$d = (c \times t) / 2$$

where $c \approx 3\times10^8$ m/s is the speed of light, t the round-trip time of the pulse, and the division by 2 accounts for the out-and-back path. From it the requirement on timing accuracy follows:

$$\text{range error } \Delta d = (c \times \Delta t) / 2$$

Reaching 1 centimeter (0.01 m) of accuracy requires $\Delta t \approx 0.067$ nanosecond—which shows the technical challenge of a LiDAR timing circuit and explains why an accurate LiDAR is far from cheap.

2. Coordinate Conversion of a LaserScan Point

The i-th measurement of a 2D LiDAR is given in polar coordinates (range r_i, angle

θ_i), where the angle follows from the scan parameters:

$$\theta_i = angle_min + i \times angle_increment$$

Converted to rectangular coordinates in the LiDAR frame (for mapping and display):

$$x_i = r_i \times \cos\theta_i$$

$$y_i = r_i \times \sin\theta_i$$

This simple polar-to-rectangular conversion is the first step by which every 2D SLAM system turns a revolution of range values into a revolution of points on a map. The scan matching of Chapter 13 and Gmapping of Chapter 14 are both built upon it.

3. Angular Resolution and Detectable Object Size

The spacing of two adjacent beams at distance d determines the smallest object size that can be detected:

$$point\ spacing\ s \approx d \times \Delta\varphi \quad (\Delta\varphi \text{ being the angular resolution in radians})$$

An object narrower than s may fall entirely between two beams and be missed. For instance, with $\Delta\varphi = 1°$ (≈ 0.0175 radian) and $d = 5$ m, $s \approx 8.7$ cm. This expression is the key to understanding how finely a LiDAR can see and quantifies the phenomenon that small distant objects are easily missed.

4. The LiDAR Data Processing Pipeline

Figure 8-3 The Data Flow of LiDAR from Emission to SLAM Input

① The laser emits a pulse or a frequency-modulated wave

↓

② The target reflects and the sensor receives the return

↓

③ Compute the time of flight → range d (dToF), or the beat frequency → range + velocity (FMCW)

↓

④ The scanning mechanism supplies the angle → the direction of each point

↓

⑤ Assemble the data: 2D → LaserScan (range + angle) / 3D → PointCloud (x, y, z)

↓

⑥ Convert polar to rectangular coordinates → hand to the SLAM front end (scan matching, feature extraction)

8.8 The Contemporary Upheaval in LiDAR: Collapsing Prices and the Move to Solid State

The technical architecture described in the first seven sections of this chapter is quite stable, but the industry itself is undergoing violent change. What students most need to know is not some new principle but a simple fact: LiDAR is turning from expensive research equipment into a cheap standard component.

1. Three Orders of Magnitude in a Decade

Line up a few dates and the trend is unmistakable. In 2016 and 2017 the 64-beam mechanical LiDAR commonly used in self-driving research cost about eighty thousand US dollars per unit; in 2020 a leading solid-state product cost about nineteen hundred dollars; in 2022 one mainstream product fell to about five hundred dollars; and by 2026 the same family had entered volume production below two hundred dollars, with an average price in the Chinese market of about one hundred forty-five dollars. Other manufacturers are targeting under two hundred dollars and planning a push toward one hundred.

What drives the change is manufacturing scale, in-house production of key components, and capacity expansion—a trajectory just like the cost curves of solar modules and smartphone camera modules before it. One major manufacturer has begun volume delivery against orders for more than four million units and plans to raise annual capacity to four million.

The significance for teaching and research is immediate: the threshold for the low-cost LiDAR described in Section 8.7 keeps falling. The 3D LiDAR that only a research laboratory could afford five years ago is now within the budget of an undergraduate project or even a high school club. The 3D LiDAR SLAM of Chapters 17 and 18 has gone from something understood by reading papers to something you can actually try.

2. Shipment Mix: Robotics Is Catching Up with Automotive

The numbers also show a change on the application side. Take one major manufacturer's 2025 shipments: about 1.62 million units for the year, more than tripling year on year; of these about 1.38 million went to advanced driver assistance systems and about 239,000 to robotics—with the latter growing more than fourfold year on year, far faster than automotive. The company further projects total shipments of three

to three and a half million units in 2026.

Robotic LiDAR growing faster than automotive reflects the spread of cleaning robots, delivery robots, warehouse AGVs, and patrol robots. These are exactly the applications that Parts IV, V, and VIII of this book address—what you are studying happens to be the segment whose demand is growing fastest.

3. Going Solid State: From Spinning to Not Spinning

Section 8.2 compared mechanical and solid-state designs. In industry that transition is now largely settled: the price collapse has occurred almost entirely in solid-state architectures, because having no rotating mechanism means fewer precision parts, longer life, smaller size, and the possibility of mass production by semiconductor processes.

For a system designer the shift brings one conceptual adjustment. A mechanical LiDAR offers a full 360-degree field of view, while a solid-state unit usually covers only a forward sector. This means a single sensor can no longer see everything—and the remedy is to fit several. Extrinsic calibration of multiple sensors (their relative poses) therefore changes from an occasional requirement into a subject every robot must handle. The point cloud registration of Chapter 17 and the coordinate transforms of Chapter 10 are the tools for exactly this.

4. Frequency-Modulated Continuous Wave and "4D LiDAR"

The FMCW architecture mentioned in Section 8.1 is regarded as the direction of the next generation. Its key advantage is measuring radial velocity directly while measuring range—each point carries a velocity component in addition to x, y, and z, which is why it is often called 4D LiDAR.

Its significance for SLAM is worth pondering. From Chapter 17 onward one difficulty recurs: dynamic objects (people walking, vehicles driving) contaminate registration and make the robot believe it has moved. If the sensor itself can tell you which points are in motion, rejecting dynamic objects changes from a difficult algorithmic problem into a simple threshold test. FMCW is also naturally resistant to interference from sunlight and from other LiDARs—a practical matter in an era when LiDARs are about to fill the streets.

Three Judgments for Students

1. Do not design systems on old prices. The argument that LiDAR is too expensive so we will use a camera holds over a far narrower range in 2026 than it did five years ago.

2. But cheap does not mean suitable. The price collapse has occurred mainly in forward-facing solid-state units of automotive specification; the small 360-degree LiDAR an indoor robot needs is still another market.

3. The physical meaning of a specification does not change with price. The angular resolution limit of Figure 8-2 and the failures on glass and black surfaces hold just as firmly on a two-hundred-dollar LiDAR as on an eighty-thousand-dollar one.

Case Studies

Case 8-1 The Cleaning Robot's Invisible Trap: The Height Blind Spot of 2D LiDAR

The cleaning robot in many a student's home has played out the same drama: it deftly skirts the sofa and the walls yet always runs headlong into the legs of the dining

chairs, or gets stuck at the edge of the baseboard. The robot is not stupid; the inherent limitation of 2D LiDAR is showing itself.

A 2D LiDAR mounted at some fixed height on top of the robot (8 centimeters, say) scans horizontally and can see obstacles only in that one slice. A slender chair leg may occupy only one or two beams at that height, and if it is also somewhat far away (the angular resolution problem of Section 8.4) it may fall entirely into the gap between beams and be missed. Lower obstacles still (cables, thresholds, a pet bowl) are not in the scan plane at all and the LiDAR is wholly insensible of them.

This case vividly explains why modern cleaning robots have taken to adding other sensors—infrared or ultrasonic obstacle detection at the front, cliff sensors underneath, even a 3D ToF camera instead (Chapter 16). It also foreshadows an important idea: a SLAM map contains only the world the sensor can see, and the physical limitations of the sensor directly determine the completeness of the map and the safety of navigation.

Case 8-2 The LiDAR Dispute in Self-Driving: The Divergence of Waymo and Tesla

A famous divergence of technical approach exists in the self-driving industry. The mainstream camp represented by Waymo covers the roof and body with several 3D LiDARs and holds that the dependable depth LiDAR provides is the cornerstone of safety; the other camp, represented by Tesla, follows a purely visual route and holds that enough cameras plus powerful AI can drive by sight as a human does, at far lower cost and with mass production in view.

Both sides have their reasons: the LiDAR camp stresses poor lighting and safety redundancy, the vision camp cost, semantic understanding, and scalability. As

automotive LiDAR continues to fall in price and go solid-state and as visual AI continues to advance, the debate keeps evolving. For the reader of this book the point is not who is right but reading the trade-off behind it: this is the best teaching material for the idea in Section 8.6 that sensor selection is a combined judgment of technology, cost, safety, and business.

Case 8-3 Backpack 3D Scanning: Giving a Building a Digital Twin

Put a multi-beam 3D LiDAR, an IMU, and a computing unit in a backpack, have an operator walk through a building wearing it, and a three-dimensional point cloud model of the whole structure is built in real time—this is backpack mobile mapping, an application Google's Cartographer system demonstrated from the start (Chapter 15).

As the operator walks corridors, stairs, and rooms, the system stitches each instant's LiDAR scan together in real time by SLAM, finally producing a high-accuracy digital twin usable for building information modeling (BIM), indoor navigation, and heritage preservation. This case threads several themes of the book together: ToF ranging (this chapter), IMU compensation of scan distortion (Chapter 7), 3D point cloud processing (Chapter 17), and loop closure correction (Chapter 15). It also shows that the value of SLAM lies not only in making a robot move but in letting people digitize real space with unprecedented efficiency—a direction with great potential for university field projects.

Hands-On Activities

Exercise 8-1 Visualizing LiDAR Data and Reading a Datasheet

Purpose: using real or simulated LiDAR data, to understand the structure of LaserScan, to perform the polar-to-rectangular conversion by hand so that a revolution of range

values becomes the outline of the environment, and to learn to read a LiDAR datasheet. This exercise offers both a simulation route and a hardware route, suiting classes with different resources.

Route A: Simulation (no hardware required; suits a whole class)

1. Install ROS 2 and the TurtleBot3 simulation environment on a computer (see Chapter 28 for detailed steps, or follow the official tutorial) and launch the Gazebo world and the virtual vehicle.
2. Display the /scan topic in RViz and observe how the red points of the LiDAR scan hug the walls as the vehicle moves.
3. Inspect the content of the LaserScan message from the command line, find angle_min, angle_increment, range_max, and the rest, record them, and compute this virtual LiDAR's field of view and total point count.
4. Record a rosbag and export one revolution of the ranges array.

Route B: Hardware (for classes with a low-cost 2D LiDAR)

5. Connect the 2D LiDAR to a computer, start its ROS driver, and confirm that the /scan topic is publishing normally.
6. Set the LiDAR still in the center of the classroom and use RViz to observe whether the scan outline hugs the four walls and the desks and chairs.
7. Place a white cardboard sheet, a black backpack, and a sheet of glass (or a mirror) in turn 2 meters in front of the LiDAR, compare how the three appear in RViz, and record the anomalies caused by the glass and the mirror.
8. Record and export one revolution of scan data for the next step's analysis.

Common Part: Data Analysis (Python)

9. Read in the exported ranges array and convert each (range, angle) pair into rectangular (x, y) coordinates by the formulation of Section 8.5.
10. Plot all the points with matplotlib and observe whether the plan outline of the environment is reproduced (walls, desks, the doorway).
11. Compute and mark: what is the actual spacing between adjacent points for this LiDAR at 3 meters and at 6 meters? Discuss the effect on detecting small objects.
12. Reading a datasheet: find the datasheet of a real LiDAR (teaching-grade or automotive), and identify its measurement range, angular resolution, scan rate, and the reflectivity premise under which the maximum range is quoted.

Discussion Questions

1. In the point cloud outline you plotted, where is it clearest? Where are there gaps or stray points? What in the environment do the gaps most likely correspond to (glass? a black object? too far away?)?
2. (Route B) What anomalies did the glass and the mirror produce in the LiDAR data? If a cleaning robot mapped with this LiDAR, what would happen when it met a floor-to-ceiling window?
3. How much do the point spacings at 3 meters and at 6 meters differ? To detect a 5-centimeter-wide chair leg dependably, within what distance can this LiDAR manage it?
4. Based on the comparison in this chapter (Table 8-2), if you were choosing the primary sensor for a campus service robot, would you choose LiDAR or a camera? Why? What sensor would you need to add?

Extended Challenge (Undergraduate)

First: write a ROS 2 node that subscribes to /scan, converts LaserScan into a point cloud in real time and publishes it, and displays the raw scan and the converted result together in RViz. Second: implement a simple nearest-obstacle alert—find the minimum range and its direction in each revolution and issue a warning when it falls below a safety threshold; this is the prototype of the obstacle avoidance of Chapter 29.

Further Reading

1. R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, Introduction to Autonomous Mobile Robots, 2nd ed., MIT Press, 2011. (The sensor material in Chapter 4 contains a systematic introduction to ranging principles.)
2. T. Raj et al., "A Survey on LiDAR Scanning Mechanisms," Electronics, 2020. (A complete comparison of scanning mechanisms—mechanical, MEMS, flash, and OPA.)
3. You Li and Javier Ibanez-Guzman, "Lidar for Autonomous Driving," IEEE Signal Processing Magazine, 2020. (An authoritative survey of automotive LiDAR principles and systems.)
4. The official ROS documentation of the sensor_msgs/LaserScan and sensor_msgs/PointCloud2 message definitions. (The first-hand specification of the data formats.)
5. The official Point Cloud Library (PCL) tutorials (pointclouds.org). (The standard tool for 3D point cloud processing, used in the exercises of Part V.)
6. The technical documentation of the SLAMTEC RPLIDAR family and the Livox family. (Specifications and ROS driver examples for entry-level 2D and low-cost 3D LiDAR.)

7. The ROBOTIS TurtleBot3 e-Manual (emanual.robotis.com). (Complete implementation guidance for the teaching platform, with SLAM examples.)

From Range to Image: Where This Chapter Stands

LiDAR has given the robot a nearly ideal geometric sense: direct, precise, and indifferent to illumination. But it has three essential limitations. First, it sees only geometry and not color, text, or semantics—it knows there is a rectangular solid ahead but not whether it is a door or a cabinet. Second, it degenerates in a featureless corridor or an empty hall: every direction looks the same and scan matching has no way to tell how far it has traveled (a problem formally discussed in Section 17.5). Third, it is expensive, heavy, and power-hungry—though as Section 8.8 described, that is changing fast.

The camera of Chapter 9 is the opposite on all three counts: it is cheap, light, and frugal with power, and it is rich in semantic information—color, texture, text, object class. The price is that it cannot measure absolute scale (monocular), it fears darkness and backlighting, and it fears white walls. The complementarity of these two sensors is the heart of Table 8-2 in Section 8.6 and the reason Parts V and VI take LiDAR and vision respectively as their protagonists.

And the fusion route represented by ORB-SLAM3 in Chapter 23 and LIO-SAM in Chapter 18 states the industry's real answer: where the budget allows, fit both. Chapter 32 goes further still—when a vision-language model can understand that a thing is a door, LiDAR's geometric precision and the camera's semantic understanding will become one, forming a navigation system that truly understands what people say.

Summary

1. LiDAR measures distance by time of flight: the pulsed form (dToF) measures the round-trip time directly, and its concentrated energy suits long ranges; FMCW modulates frequency continuously and can measure range and velocity at once with good interference rejection, though at higher cost, and it is an important direction for the next generation.
2. A 2D LiDAR scans a single horizontal plane with light data and a low price and is the mainstay of indoor mapping; a 3D LiDAR obtains three-dimensional information by multiple beams or vertical scanning; scanning mechanisms are moving from mechanical to solid-state schemes such as MEMS, flash, and OPA, trending smaller, more vibration-tolerant, and cheaper.
3. A 2D LiDAR outputs LaserScan (a range array plus regular angles: compact and light) and a 3D LiDAR outputs PointCloud (three-dimensional coordinates stored directly: voluminous and in need of downsampling); the two correspond to Parts IV and V of this book respectively.
4. The key specifications include measurement range, accuracy and precision, angular resolution (which determines the smallest detectable object, with points growing sparser at greater distance for the same angular resolution), scan rate (too low a rate causes motion distortion), and reflectivity (black objects and glass or mirrors being the principal causes of failure).
5. LiDAR is widely used in indoor mapping, warehouse AGVs (doubling as a safety guard), and self-driving cars (three-dimensional perception and map-based localization), as well as in emerging fields such as backpack surveying, agricultural and forestry measurement, and quadruped patrol.
6. LiDAR gives precise geometry but no semantics and fears glass and mirrors; the camera gives rich semantics but no dependable depth or scale; the IMU gives

high-rate motion but drifts; the three are naturally complementary and the most robust systems often use them together.

7. Low-cost 2D LiDAR and teaching platforms such as TurtleBot, together with wholly free simulation environments, have allowed SLAM education to reach undergraduates and high school students; this book recommends the learning path of simulation first, cheap hardware next, and accuracy last.

In the next chapter we turn to the sensor richest in information—the camera. From the pinhole camera model, lens distortion, and calibration to the differences among monocular, stereo, and RGB-D cameras, Chapter 9 explains how an image becomes geometric information rather than a heap of pixels, laying the foundation for the visual SLAM of Part VI.

A Taiwan-Authored University Textbook

@prof

Simultaneous Localization and Mapping

Simultaneous Localization and Mapping, SLAM

Chapter 9

Cameras and Visual Navigation Sensors

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

The LiDAR of Chapter 8 measures geometry directly, precisely and dependably. The camera of this chapter is exactly the opposite—it measures no distance at all, yet it brings what LiDAR can never give: color, texture, text, and semantics.

This is the last chapter of Part II and the turning point of the whole book. The sensors accumulated over the first eight chapters are now all present: absolute position (Chapters 4 and 5), absolute direction (Chapter 6), high-rate motion (Chapter 7), precise geometry (Chapter 8), and rich semantics (this chapter). Every one of them has its moments of lying, and not one can accomplish navigation alone.

What Part III teaches, therefore, is the mathematics of fusing these mutually contradictory readings into the single most reasonable answer—coordinates and pose in Chapter 10, probability and state estimation in Chapter 11, and optimization and graph theory in Chapter 12. The content of this chapter is especially crucial: the pinhole model and the intrinsic matrix K are the bedrock of the whole of Part VI's visual SLAM, and K appears in every expression of the epipolar geometry of Chapter 21, ORB-SLAM2 of Chapter 22, and ORB-SLAM3 of Chapter 23.

Figure 9-0 How Chapter 9 Connects to the Rest of the Book

[Building on the previous chapter]

Chapter 8 LiDAR: geometrically precise, but without semantics, degenerate in corridors, and costly

This chapter The camera: cheap, light, and frugal with power, and rich in semantics, but

without scale and afraid of darkness and white walls

[This chapter is responsible for]

The pinhole model → the intrinsic matrix K → distortion and calibration

The three ways a camera obtains depth: monocular, stereo, RGB-D

The four raw materials of vision: feature points, edges, optical flow, depth

[Where it leads]

→ Section 4.6 The K that AprilTag pose solution requires, completed here

→ Chapter 20 Feature-based and direct methods: the full unfolding of Section 9.6 of this chapter

→ Chapter 21 Epipolar geometry: motion estimation between two images

→ Chapters 22 and 23 ORB-SLAM2 and ORB-SLAM3

→ Chapter 30 Mapping on real hardware with RGB-D and stereo cameras

→ Chapters 32 and 34 Semantic SLAM and VLM language navigation: the full version of Section 9.7 of this chapter

Learning Objectives

The camera is the richest sensor in information and the cheapest—one image contains shape, texture, color, and semantics at once. But an image is in essence a three-dimensional world flattened into two, and recovering the lost depth is precisely the central challenge of visual SLAM. This is the last chapter of Part II and the entrance to the visual SLAM of Part VI. After completing this chapter, you should be able to:

1. Explain the pinhole camera model and write the mathematical relation by which a three-dimensional point projects into a two-dimensional image.

2. Explain the meaning of camera intrinsics and extrinsics, and account for the cause of lens distortion and the purpose of camera calibration.
3. Compare the ways monocular, stereo, and RGB-D cameras obtain depth, with the merits and drawbacks of each.
4. Describe the differences and uses of three kinds of visual information: feature points, edges, and optical flow.
5. Analyze the advantages and limitations of visual navigation relative to LiDAR, the scale ambiguity problem above all.
6. Describe the line of knowledge that leads from image processing into visual SLAM.
7. Gain a first acquaintance with vision-language models (VLMs) and the concept of semantic navigation, in preparation for Chapter 32.

Core Concepts

Chinese Term	English Term	Explanation
針孔相機模型	Pinhole Camera Model	The geometric model describing how a three-dimensional point projects through the optical center onto the image plane.
焦距	Focal Length	The distance from the optical center to the image plane, which sets magnification and field of view.
相機內參	Intrinsic Parameters	The camera's own imaging parameters—focal length, principal point, and so on—which form the intrinsic matrix K.
相機外參	Extrinsic Parameters	The camera's position and orientation in world coordinates (rotation R and

Chinese Term	English Term	Explanation
		translation t).
鏡頭畸變	Lens Distortion	Image deformation caused by the lens, divided into radial and tangential distortion.
相機校正	Camera Calibration	The procedure for measuring and estimating the intrinsics and the distortion coefficients.
視差	Disparity	The horizontal displacement of the same point between the left and right images, used for stereo depth.
基線	Baseline	The distance between the two lenses of a stereo camera, which affects depth range and accuracy.
特徵點	Feature / Keypoint	A salient point in an image that is easy to recognize and repeatably detectable, such as a corner.
光流	Optical Flow	The movement of pixel brightness over time in an image, which reflects motion.
尺度不確定性	Scale Ambiguity	The inherent limitation that a monocular camera cannot determine true size.
視覺語言模型	VLM	An AI model able to understand images and language at once, used in semantic navigation.

9.1 The Pinhole Camera Model

Understanding how a camera sees the world requires first understanding an old and elegant model—the pinhole camera. Its prototype is the camera obscura: a small hole in one face of a sealed box through which light from outside projects an inverted image on the opposite inner wall. This seemingly simple device contains the core geometry of all camera imaging.

The Geometry of Projection

Place the pinhole (the optical center) at the origin with the optical axis along Z pointing forward. Light from a three-dimensional point P = (X, Y, Z) in space passes through the optical center and strikes an image plane at focal length f. By similar triangles the coordinates of the projected point are:

$$x = f \cdot X / Z$$

$$y = f \cdot Y / Z$$

Simple as they are, these two expressions lay bare the source of every difficulty in visual SLAM—note that the denominator in each is Z, the depth. This means:

- Three dimensions are flattened into two: of the three coordinates (X, Y, Z), only two (x, y) survive projection, and the dimension of depth Z has been divided away.
- Depth information is lost: a small object at 2 meters and a large object at 4 meters may occupy exactly the same size in the image. From one image alone, a near small thing cannot be distinguished from a far large one.

This is the geometric essence of the scale ambiguity discussed in depth in Section 9.5. Recovering that divided-away Z is the common goal of stereo cameras, RGB-D

cameras, and all the techniques of visual SLAM.

From Physical Coordinates to Pixel Coordinates

The (x, y) computed above are physical coordinates on the image plane (in millimeters). But a digital image uses pixel coordinates—origin at the top left, units in pixels. The conversion needs three things: the scale that converts millimeters to pixels (fx and fy, the focal length expressed in pixels) and the position of the image center (the principal point) (cx, cy). The complete projection relation can be written in matrix form:

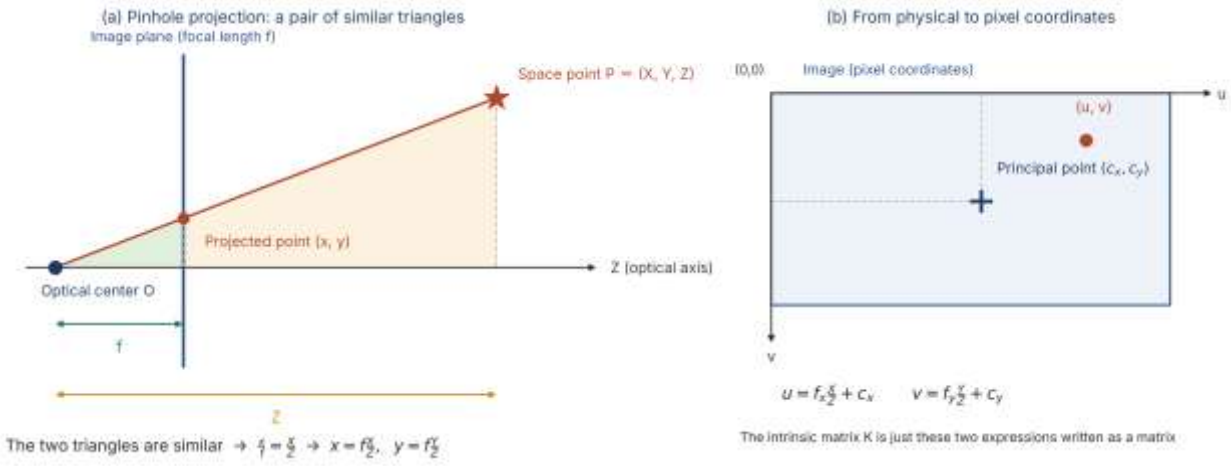
$$[u\ v\ 1]^T = (1/Z) \cdot K \cdot [X\ Y\ Z]^T$$

where K is called the camera intrinsic matrix:

$$K = [\,fx\ 0\ cx\,;\,0\ fy\ cy\,;\,0\ 0\ 1\,]$$

This matrix K is the most basic and most frequently appearing object in visual SLAM. The AprilTag solution of Chapter 4, the epipolar geometry of Chapter 21, and ORB-SLAM of Chapter 22 all use it. And the values in K must be obtained by camera calibration (Section 9.2).

Figure 9-1 The Pinhole Camera Model: From a Point in Space to a Pixel



The left panel is the starting point of all visual geometry, and it is in fact nothing but a pair of similar triangles. The distance from the optical center to the image plane is the

focal length f and the distance from the optical center to the point in space is the depth Z; the two triangles are similar, so $x / f = X / Z$, that is, $x = f \cdot X / Z$. All the complicated mathematics of visual SLAM traces back, in the end, to these two lines.

Note in particular that Z in the denominator. It is the source of every difficulty in this chapter and indeed in the whole of Part VI: Z is divided away in projection, so the same pixel may come from a small thing nearby or a large thing far off. This is the scale ambiguity of Section 9.3, and the reason the IMU of Chapter 7 matters so much to monocular vision—the IMU measures acceleration in physical units and can restore the lost scale.

The right panel deals with something else: a camera outputs pixels, not meters. The origin of pixel coordinates is at the top left of the image, and the point the optical axis passes through (the principal point) is usually not exactly at the center—the assembly of sensor and lens always has some offset. Translations cx and cy must therefore be added, and fx and fy used to convert physical units into pixels (the two may differ slightly, since pixels are not necessarily square).

Arrange these four numbers fx, fy, cx, cy into a matrix and you have the intrinsic matrix K. So K is nothing abstract—it is the four numbers describing how this camera flattens the world into pixels. And the calibration of Section 9.2 does exactly this: it measures those four numbers (plus the distortion coefficients).

A Distinction Often Confused

The intrinsics describe the camera itself: focal length, principal point, distortion.

So long as the lens is not changed and the focus not adjusted, they stay fixed.

The extrinsics describe where the camera is: the rotation R and translation t

relative to the world frame. The moment the robot moves, they change.

What visual SLAM solves for is precisely the extrinsics—the R and t of every frame are the robot's pose. The intrinsics must be calibrated beforehand, or every extrinsic solved for will be wrong.

This division of labor has exactly the same structure as the magnetometer calibration of Section 6.4: correct the sensor itself first, then speak of its relation to the world.

9.2 Lens Distortion and Camera Calibration

The pinhole model assumes light travels in perfect straight lines, but a real camera uses a glass or plastic lens. A lens gathers more light and makes the image brighter and sharper, but it also introduces distortion—geometric deformation of the image.

Two Kinds of Distortion

- Radial distortion: caused by the shape of the lens, growing more severe with distance from the image center. Barrel distortion bows straight lines outward (pronounced in wide-angle and fisheye lenses) and pincushion distortion bows them inward. Photograph a brick wall with a phone's wide-angle lens and the mortar lines near the edges bend visibly—that is radial distortion.
- Tangential distortion: caused by the lens and the sensor not being mounted perfectly parallel, making some regions of the image appear skewed and stretched. It is usually milder than radial distortion.

Distortion is fatal to visual SLAM: all of SLAM's geometric computation assumes the pinhole model (straight-line projection), and if distortion is not corrected first, a line that is truly straight is curved in the image, every geometric inference based on lines

and angles goes systematically wrong, and the resulting map comes out warped.

Camera Calibration: Finding K and the Distortion Coefficients

The purpose of camera calibration is to measure this camera's intrinsic matrix K and its distortion coefficients. The most classic method is the checkerboard calibration proposed by Zhengyou Zhang in 2000, whose principle descends directly from the PnP of AprilTag in Chapter 4:

1. Prepare a checkerboard calibration target of known dimensions (the side length of each square known).
2. With the camera to be calibrated, photograph the checkerboard ten to twenty times from different angles and distances.
3. The algorithm automatically detects the pixel coordinates of the checkerboard corners in each image.
4. Since the true three-dimensional coordinates of the checkerboard corners are known (regularly arranged on a plane) and their image coordinates have been measured, one can infer the K and the distortion coefficients that best fit these correspondences (another least-squares optimization problem).
5. Use the recovered distortion coefficients to undistort the images, correcting the curved image back into conformity with the pinhole model.

OpenCV has complete calibration tools built in (calibrateCamera and related functions), and ROS has the camera_calibration package—wave the camera at a checkerboard and it completes automatically. This is the first step of every visual SLAM implementation and will certainly be used when implementing RGB-D SLAM in Chapter 30.

A Practical Reminder: Calibration Is Not Done Once and for All

• Changing the lens, adjusting the focus, or even a knock that shifts the lens invalidates the old calibration, and it must be redone.

• Calibration images should cover every region of the frame and a variety of tilt angles—and above all do not forget the corners, where radial distortion is most severe.

• The calibration error (the reprojection error) should be below 1 pixel to count as good; if it is too large, usually the checkerboard shots were not varied enough or corners were detected wrongly.

• Industrial cameras often ship with calibration parameters, but for applications demanding high accuracy it is still advisable to calibrate and verify them yourself.

9.3 Monocular, Stereo, and RGB-D Cameras

Section 9.1 explained that a single image loses depth. Three mainstream routes recover it, corresponding to three kinds of camera.

1. The Monocular Camera: Simplest and Hardest

One camera, one image. The hardware is the simplest, cheapest, most frugal with power, and most ubiquitous (every phone has one). But as described, a single image cannot give depth directly—which brings the famous scale ambiguity (Section 9.5).

How, then, does a monocular camera do SLAM? The answer is by using motion: as the camera moves it photographs the same scene from different positions, much as a person can sense distance with one eye closed by rocking the head side to side. From

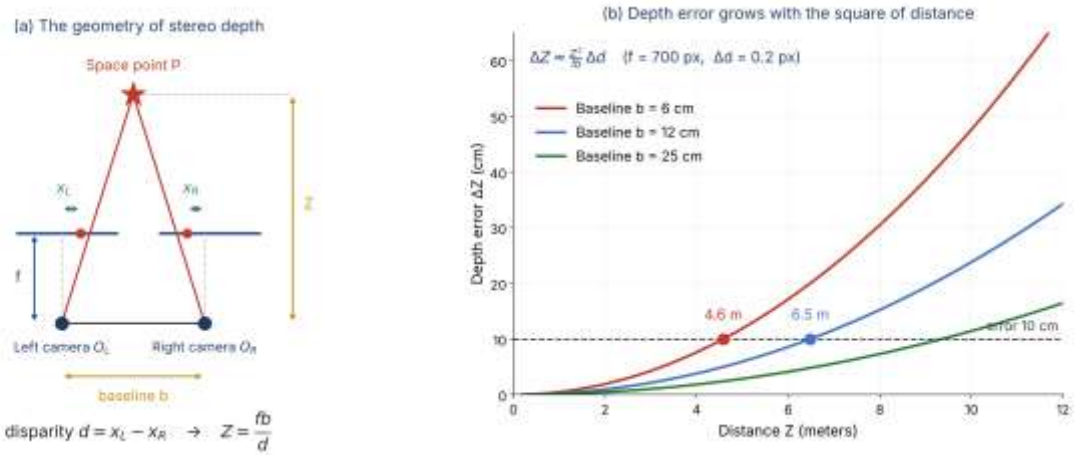
the changing parallax across successive frames, depth can be triangulated—but that depth has only relative proportion and no absolute scale (there is no telling whether it is a real meter or a centimeter in a doll's house) unless other information is brought in (an IMU, a known object size, or wheel odometry). This is precisely the value of the IMU of Chapter 7 in supplying absolute scale to monocular vision.

2. The Stereo Camera: Imitating Two Eyes

Two cameras are mounted side by side at a fixed, known separation (the baseline) and photograph simultaneously. The same point in space falls at different horizontal positions in the left and right images, and the difference is called the disparity. Near objects have large disparity and far objects small—hold a finger before your eyes and close each eye in turn and the finger jumps a long way against the background, while a distant mountain barely moves; this is the same principle.

From the disparity together with the known baseline and focal length, depth follows directly by triangular geometry (the formulation in Section 9.6). The advantages of stereo are that a single (left-right) pair yields depth, that it has absolute scale, and that it works indoors and out. The limitations are that the relative positions of the two cameras must be calibrated precisely, that it fails on untextured surfaces (a white wall) where no correspondence can be found, and that depth accuracy falls off rapidly with distance (the shorter the baseline, the faster the falloff).

Figure 9-2 The Geometry of Stereo Depth and the Growth of Depth Error



The left panel shows why stereo depth works. The same point in space falls at different horizontal positions in the left and right images, and that difference is the disparity d. The nearer the point, the larger the disparity; the farther, the smaller. Similar triangles give Z = f·b / d—focal length and baseline are known constants, so measuring the disparity is measuring the depth.

The right panel shows its limitation, and this is the thing a datasheet does not show and yet decides whether the system succeeds. Differentiating Z = f·b / d with respect to d gives $\Delta Z \approx (Z^2 / f{\cdot}b){\cdot}\Delta d$: depth error grows with the square of distance. With a focal length of 700 pixels and a disparity measurement error of 0.2 pixel, a small stereo module with a 6-centimeter baseline already has 10 centimeters of depth error at 4.6 meters; stretch the baseline to 12 centimeters and that extends to 6.5 meters; 25 centimeters reaches 9.4 meters.

This explains three things seen in practice. First, why the stereo cameras on a self-driving car have such wide baselines—because they must see a hundred meters ahead. Second, why the dual lenses of a phone can only produce portrait-depth effects within a few meters (Case 9-1) and cannot be used for navigation. Third, why RGB-D cameras are nearly always labeled with an effective range of 0.3 to 5 meters—beyond that the

error becomes unusable.

When choosing a camera you may treat this figure as a lookup table: decide first at what distance you need what accuracy, then work back to the baseline required. This process of inferring specifications from requirements is exactly isomorphic to the judgment of LiDAR angular resolution in Section 8.4—the physical principles of the two sensors differ, but the engineering reasoning is one and the same.

3. The RGB-D Camera: Active Depth Measurement

An RGB-D camera outputs a color image (RGB) and the depth (D) of every pixel at once. It obtains depth actively, by two mainstream technologies:

- Structured light: a known infrared pattern (a dense array of dots, say) is projected onto the scene and depth is inferred from how far the pattern is deformed. The first-generation Microsoft Kinect used this method; accuracy is high at short range, but it fails in strong light and the range is short.
- Active IR stereo: essentially stereo ranging, but with an additional projected infrared texture pattern to solve the problem of finding no correspondence on untextured surfaces such as white walls. The Intel RealSense D400 family uses this method and works indoors and out.

The greatest advantage of an RGB-D camera is dense depth out of the box, with no need to perform complicated depth estimation oneself, which makes it the ideal input for indoor RGB-D SLAM (RTAB-Map and ORB-SLAM3 in Chapter 30). The limitations are that most depend on infrared and so perform poorly outdoors in strong sunlight (the sun's infrared swamps the projected pattern), that the effective range is usually 0.2 to 10 meters, and that they are equally helpless against glass and mirrors (the same weakness as the LiDAR of Chapter 8).

Type	How Depth Is Obtained	Absolute Scale	Principal Advantage	Principal Limitation
Monocular	Motion + triangulation	None (must be supplied externally)	Cheapest, most ubiquitous, most frugal with power	Scale ambiguity; requires motion
Stereo	Triangular geometry from left-right disparity	Yes	Depth from a single frame; works indoors and out	Needs calibration; fears untextured surfaces; falls off at distance
RGB-D	Structured light / active IR stereo	Yes	Outputs dense depth directly	Fears strong light; limited range; fears glass

Table 9-1 Comparison of How Three Kinds of Camera Obtain Depth

9.4 Feature Points, Edges, Optical Flow, and Depth Estimation

What a camera gives is a whole image of several hundred thousand pixels. But SLAM cannot, and need not, process every pixel; it extracts the useful information from the image. This section introduces the three most important kinds of visual information.

1. Feature Points (Keypoints): The Landmarks of Visual SLAM

A feature point is a salient point in the image that is easy to recognize and still recognizable from another angle—usually a corner (where two edges meet), such as the corner of a window frame, the junction of floor tiles, or the corner of a letter on a poster.

Why are corners preferred? Imagine choosing a point on a white wall and trying to find it again in the next frame—everywhere on a white wall looks the same and it cannot

be found at all; choose a point on the edge of a straight line and the perpendicular direction can be determined but the position along the line cannot (this is called the aperture problem); only a corner is unique in both directions and can be relocated precisely. This is the bedrock of feature-based visual SLAM (Chapters 20 and 22): treat corners as landmarks in the environment and track their movement across successive images to infer the camera's motion. ORB, SIFT, SURF, and FAST are all well-known methods of feature detection and description, treated in detail in Chapter 20.

2. Edges: The Outlines of Structure

An edge is a place of abrupt brightness change in the image, usually corresponding to the outline of an object or the junction of different surfaces. Edge information plays an important part in some direct-method and line-feature SLAM systems, especially in man-made environments poor in feature points (an empty corridor, a scene with nothing but wall lines), where lines are instead the stable cue.

3. Optical Flow: The Motion of Pixels

Optical flow describes the movement of brightness patterns in an image over time—as the camera or an object moves, the texture in the frame flows. Picture raindrops streaming backward on a car window: that is optical flow visualized. Optical flow can be used to estimate motion, track features, and detect dynamic objects. It is the core of direct-method visual odometry and the key technique in UAV position hold (optical flow positioning, Chapter 26).

4. Depth Estimation

Besides the stereo disparity and the active RGB-D sensing described above, deep learning has recently brought a breakthrough in monocular depth estimation: a neural network is trained to guess the relative depth of every pixel from a single image. Such

models (Depth Anything, mentioned in Chapter 32, for example) exploit prior knowledge learned from a great many images—they know that the sky is far away, that a floor recedes into the distance, that a person is bigger than an ant. Although it is guessing and mostly yields only relative depth, it is already practical enough for many applications and is an important complement to traditional geometric methods.

These four kinds of information are not mutually exclusive: modern visual SLAM commonly mixes them—feature points for a robust localization skeleton, optical flow for fast tracking, and depth (whether from stereo, RGB-D, or a neural network) to reconstruct a dense map.

9.5 The Advantages and Limitations of Visual Navigation

Placing the camera in the panorama of Part II's sensors (Table 8-2 of Chapter 8 already made a first comparison), this section goes deeper into the advantages and the soft spots peculiar to visual methods.

The Distinctive Advantages of Visual Navigation

- Richest in information: one image contains shape, texture, color, and semantics at once. LiDAR and the IMU cannot give this at all—LiDAR knows only that there is something three meters away, while a camera can know that it is a red door. This is precisely why the semantic navigation of Chapter 32 is built on vision.
- Lowest cost and most ubiquitous: a camera is one to two orders of magnitude cheaper than a LiDAR, is everywhere, and is mature in volume production.
- Passive sensing and low power: unlike LiDAR, which must emit a laser actively, a camera only receives ambient light, which saves power and avoids mutual

interference—particularly suited to UAVs and micro robots (Chapters 25 and 26).

- Light: for a UAV severely constrained in payload and power, a camera plus an IMU is the ideal combination.

The Inherent Limitations of Visual Navigation

- Scale ambiguity: the most fundamental soft spot of a monocular camera. As Section 9.1 explained, projection divides depth away, a monocular image cannot know absolute size, and scale must be supplied by an IMU, a stereo baseline, or an object of known size.
- Dependence on light and texture: the most fatal weakness of visual methods. Darkness, backlighting, and overexposure blind the camera; untextured surfaces such as white walls and plain-colored floors yield no feature points; and image blur from rapid motion also breaks tracking. LiDAR, by contrast, carries its own light source and does not fear the dark.
- Computational burden: image data is voluminous, and feature extraction and matching require considerable computing power—which is where edge computing platforms come in (the Jetson of Chapter 33).
- Fear of dynamic scenes: people and vehicles moving through the frame disturb visual SLAM, which assumes a static environment.

A Summary of the Idea: Vision and LiDAR as Complements

LiDAR: precise geometry, carries its own light and does not fear the dark, but has no semantics, fears glass and mirrors, and is more expensive.

The camera: rich semantics, cheap and light, but no dependable scale or depth,

and fears darkness and untextured surfaces.

Their weaknesses hardly overlap—which is exactly the fusion logic of "LiDAR builds the geometric skeleton, the camera applies the semantic labels" (Chapter 32),

and confirms once more the central conviction of Part II: there is no all-powerful sensor, only an appropriate combination.

9.6 From Image Processing to Visual SLAM

Many students have studied image processing before taking this course—filtering, edge detection, thresholding, morphological operations, and so on. This section explains how those fundamentals lead step by step to visual SLAM, joining the knowledge of the two courses together.

The Ladder of Knowledge

6. Image processing: operating on the pixels of a single image—removing noise, enhancing contrast, detecting edges. This is the level of making an image clearer.
7. Feature extraction: pulling corners and descriptors out of an image, condensing a heap of pixels into a few identifiable points. This is the material of Chapter 20.
8. Multiple-view geometry: analyzing the geometric relations among several images of the same scene and inferring camera motion and scene structure from the movement of feature points. This is the epipolar geometry of Chapter 21.

9. Visual odometry (VO): chaining the previous step frame by frame to estimate the camera trajectory continuously—but with accumulating drift.
10. Visual SLAM: adding mapping, loop closure detection, and global optimization on top of VO to eliminate long-term drift and obtain a consistent trajectory and map. This is the subject of Part VI.

This ladder echoes the order of exposition in the UZH course of Chapter 2: image formation → features → multiple-view geometry → visual odometry → SLAM. Each rung is the foundation of the next and none may be skipped. This book treats image processing as prerequisite knowledge and enters visual SLAM formally from the features of Chapter 20; but the camera model and calibration laid down in this chapter are the premise of understanding everything that follows.

Two Schools: Feature-Based and Direct

Visual SLAM has two main technical routes; the concepts are established here and detailed in Chapter 20:

- Feature-based: extract and match feature points and solve for motion from their geometric relations. Robust and mature, with the ORB-SLAM family as its representative.
- Direct: extract no features and estimate motion directly from pixel brightness change (the optical flow idea). It has the advantage in weakly textured scenes but is sensitive to changes in lighting, with LSD-SLAM and DSO as its representatives.

9.7 A First Look at VLMs, AI Vision Models, and Semantic Navigation

This section previews Part IX (Chapters 32–34, AI semantics and language navigation). Traditional visual SLAM answers a geometric question: where is the camera and what shape is the environment? An AI vision model can further answer a semantic question: what is in the frame and what does it mean? This is precisely the camera's most precious value relative to LiDAR—it carries semantics.

From Geometry to Semantics

A robot doing only traditional SLAM builds a map that says there is a wall here and an obstacle there; with AI visual recognition added, the map is upgraded to this is a door, that is a staircase, over there is a fire extinguisher—a semantic map (mentioned in Chapters 1 and 3). This upgrade lets a robot carry out tasks at the level of human language.

Key AI Vision Models (detailed in Chapter 32)

- YOLO: real-time object detection, drawing boxes in the image around this is a person, that is a chair; the mainstay of campus scene recognition (a focus of the supplementary material and the university course).
- SAM (Segment Anything): segmenting an image precisely into regions, object by object.
- Depth Anything: estimating depth from a single image (mentioned in Section 9.4).
- CLIP and DINO: understanding the semantic content of an image and linking images with text.

VLMs: Letting a Robot Both See and Understand

A vision-language model (VLM) can understand images and natural language at once. This lets a robot accept spoken tasks: "walk to the whiteboard," "avoid the

backpack on the floor," "help me find the elevator"—the VLM analyzes the camera view, understands the language instruction, and converts the task into a navigation goal.

This is the paradigm shift now under way in robot navigation: from given coordinates, go there to understand what people say, understand the scene, and decide autonomously how to go. It is also why this chapter stands at the end of Part II—the camera is not just one more ranging sensor but the robot's window onto understanding the world. Chapter 32 unfolds this exciting direction in full, including how to collect and annotate training data, how to train a campus scene recognition model, and how to integrate a VLM with ROS 2.

Part II, "Sensors for Localization and Navigation," is now complete. You have met the robot's six major senses: odometry and the IMU perceive its own motion (Chapter 7), radio and satellites and the geomagnetic field supply absolute references (Chapters 5 and 6), indoor positioning technologies fill the indoor gap (Chapter 4), LiDAR measures geometry precisely with a laser (Chapter 8), and the camera brings the richest visual and semantic information (this chapter). In the next part we turn from what the sensors give to how the mathematics handles this noisy data—entering the mathematical foundations of SLAM.

Formulations and Procedures

1. The Complete Pinhole Projection Equation

The position of a three-dimensional point P_w in world coordinates is transformed into camera coordinates by the extrinsics (rotation R, translation t) and then projected into pixel coordinates by the intrinsics K, the complete relation being:

$$s \cdot [u\ v\ 1]^T = K \cdot [R \mid t] \cdot [X_w\ Y_w\ Z_w\ 1]^T$$

where s is the scale factor (equal to that point's depth Z in the camera frame). This expression threads the three central concepts of this chapter together: the intrinsics K (9.1), the extrinsics [R|t] (the camera's pose in the world, which is exactly what SLAM solves for), and the scale s (the source of the scale ambiguity of 9.5). All of visual SLAM is, in essence, solving for the camera poses [R|t] and the three-dimensional point coordinates simultaneously across a great many such projection equations.

2. The Stereo Depth Formula

Depth computation with a stereo camera is the most intuitive formula in visual ranging. With baseline b, focal length f (in pixels), and disparity d (in pixels) for a point between the left and right images, the depth Z is:

$$Z = f \cdot b \,/\, d$$

This expression reveals three important facts:

- The larger the disparity d (a near object), the smaller the depth Z; as the disparity approaches 0 (a far object) the depth approaches infinity—so stereo cannot measure distant objects accurately.
- The larger the baseline b, the greater the disparity at a given distance, the more accurate the depth, and the farther the measurable range (which is why the RealSense D455 enlarged its baseline to improve accuracy at distance).
- Depth error is proportional to the square of distance: double the distance and the error quadruples. This is the common fate of all passive visual depth measurement.

3. The Growth of Depth Error with Distance

For a disparity measurement error Δd, the depth error is approximately:

$$\Delta Z \approx (Z^2 / (f \cdot b)) \cdot \Delta d$$

The Z^2 in this expression explains why stereo and RGB-D cameras all advertise accuracy at short range and rapid falloff at long. For instance, a camera with 1 centimeter of error at 1 meter may have as much as 16 centimeters at 4 meters. This expression is an important basis for choosing a camera and setting the working range of a robot.

4. The Processing Pipeline of a Camera in Visual SLAM

Figure 9-3 The Data Flow from Camera to Visual SLAM

① The camera captures an image (monocular / stereo / RGB-D)

↓

② Undistortion: correct the image with the distortion coefficients from calibration (9.2)

↓

③ Feature extraction: detect corners and compute descriptors (9.4, Chapter 20)

↓

④ Feature matching / optical flow tracking: find corresponding points between adjacent frames

↓

⑤ Geometric solution: infer camera motion from the correspondences and the intrinsics K (Chapter 21)

↓

⑥ Triangulation / depth: reconstruct three-dimensional points (monocular needs motion, stereo uses disparity, RGB-D gives it directly)

↓

⑦ Back-end optimization + loop closure: eliminate drift and output trajectory and map (Chapters 22 and 23)

9.8 The Contemporary Upheaval in Visual Sensing: When Geometry Is Replaced by Models

This chapter has been about geometry: the pinhole model, the intrinsic matrix, the

disparity formula. These expressions have hardly changed since they took shape in the 1990s and remain the foundation of every system in Part VI. But between 2024 and 2026 something happened in computer vision that may rewrite that foundation, and students must know of it.

1. Monocular Depth Is No Longer an Impossible Task

Section 9.3 said a monocular camera cannot obtain depth, and in the purely geometric sense that remains correct—one image genuinely cannot determine scale mathematically. But recent depth estimation foundation models accomplish something else: they learn from a great deal of data how deep this kind of scene ought to look.

What they rely on is not geometry but prior knowledge: a door is about two meters tall, a corridor about one and a half meters wide, a person about one meter seventy. Having seen enough of the world, a model can produce a quite reasonable depth map from a single photograph, even an absolute depth in meters.

The significance for robots is practical: the RGB-D camera of Chapter 30 fails in strong sunlight (noted in Section 9.3), while a monocular depth model is not so limited; it can also supply a scale reference for initializing the monocular SLAM of Chapter 20. But keep its nature firmly in mind—it is a guess, not a measurement. Confronted with a scene it has never seen (a doll's house, a cavernous exhibition hall), it will give a wrong answer with confidence.

2. VGGT: Replacing the Whole SfM Pipeline with One Model

A more fundamental shock comes from the visual geometry grounded transformer presented in the best paper at CVPR 2025. Its approach is extremely direct: given one

to several hundred images, a single forward pass of the model outputs camera intrinsics and extrinsics, depth maps, point clouds, and three-dimensional point tracks all at once—the whole process in under a second.

Contrast this with the traditional pipeline. Obtaining the same result conventionally requires detecting feature points (Section 9.4) → matching → estimating epipolar geometry (Chapter 21) → triangulation → running bundle adjustment (Chapter 12) in repeated iterations. That pipeline is what Part VI needs four chapters to cover. And the research shows that, without any geometric optimization as post-processing, the model's performance already exceeds that of traditional methods which do post-process.

Must Part VI then still be studied? This book's answer is emphatic: yes, and all the more so. There are three reasons. First, such models currently target offline reconstruction rather than real-time SLAM, and what a robot needs is continuous localization thirty times a second, not a one-off scene reconstruction. Second, they have no loop closure, no long-term consistency, and no uncertainty output—and those three things are the core of SLAM. Third, and most important: someone who does not understand the geometry cannot tell when the model is talking nonsense.

3. The Event Camera: A Sensor That Takes Another Road Entirely

The event camera also deserves mention. A conventional camera outputs complete frames at a fixed rate; an event camera lets every pixel operate independently—reporting an event only when the brightness change exceeds a threshold, with time resolution down to the microsecond and dynamic range far beyond an ordinary camera.

This answers exactly the two great visual limitations listed in Section 9.5: motion blur

and violent changes in illumination. A conventional camera photographs a blur while a UAV tumbles at speed, whereas an event camera still tracks clearly; a conventional camera is wholly overexposed the instant one bursts out of a tunnel into daylight, while an event camera is barely affected. This line of work is mentioned again in Chapter 26 on UAV visual-inertial navigation.

The price is that its output is not an image but a stream of events, so none of the algorithms of Section 9.4 can be applied directly and all must be redesigned. This keeps it largely in the research domain to this day—but in high-speed, high-dynamic-range settings it is currently the only answer.

Three Judgments for Students

1. Geometry is not obsolete. A pose output by a model must still be interpreted with the K of this chapter, must still be expressed in the coordinate frames of Chapter 10, and must still pass through the optimization of Chapter 12 to be globally consistent.

2. The model's most valuable place is the front end. Use it to replace the fragile handcrafted steps—feature matching, monocular initialization, depth priors—while keeping the geometric optimization of the back end. This agrees exactly with the conclusion on learning-based IMUs in Section 7.8.

3. Judgment comes from geometry. Someone who knows that $\Delta Z \propto Z^2$ will know to be suspicious when a model says the wall is fifty meters away; someone who does not will simply accept it.

Case Studies

Case 9-1 Phone Portrait Mode: The Everyday Magic of Two Lenses

The dual-lens phone in your pocket demonstrates the principles of this chapter every day. Turn on portrait mode and the subject is sharp while the background is softly blurred—behind that effect is stereo depth measurement: the two lenses photograph simultaneously across a tiny baseline, the phone computes the disparity of every pixel to estimate a depth map, decides which pixels are the near subject and which the far background, and then applies a simulated bokeh blur to the background.

This case also exposes the limitation of stereo. Portrait mode often fails at the edge of hair, on openwork objects, or against a plain background—because disparity is hard to estimate accurately there (no texture, detail too fine). This is the everyday embodiment of what Section 9.3 said about stereo fearing untextured surfaces. In recent years phones have moved to a hybrid of two-lens disparity, AI semantic segmentation, and neural network depth estimation, which answers exactly to the trend of mixing several kinds of visual information noted in Section 9.4. Next time you use portrait mode, observe where it goes wrong—it is the most personal experiment in visual depth there is.

Case 9-2 The Mars Helicopter Ingenuity: Interplanetary Navigation on Vision Alone

NASA's Mars helicopter Ingenuity completed dozens of flights on Mars, making history as the first powered flight on another planet. There is no GPS on Mars and no usable magnetic compass (Mars has almost no global magnetic field), so how did it know where it was and fly so steadily?

The answer is visual-inertial navigation: a downward-facing monocular camera under the belly photographed the ground at tens of frames per second, tracking

feature points on rocks and surface texture (Section 9.4), with an IMU to estimate motion—precisely the technique the UZH research group mentioned in Chapter 2 excels at and the core of the UAV navigation of Chapter 26. The thin Martian atmosphere (about 1% of Earth's density) makes flight extremely unstable and places severe demands on the immediacy and robustness of navigation. Ingenuity's success proved one thing: in an extreme environment with no external signal whatever, a cheap camera plus an IMU can carry dependable autonomous navigation. This is also why this book attaches such importance to visual-inertial methods—they are the key to a robot leaving infrastructure behind and becoming truly autonomous.

Case 9-3 The Curse of the White Wall: A Classic Failure of Visual SLAM

A scene played out repeatedly in the laboratory: a student eagerly runs visual SLAM on a camera-equipped robot in the classroom, and everything goes well—until the robot turns toward a clean white wall, the system loses track at once, and the map collapses.

The cause is just what Sections 9.4 and 9.5 stressed repeatedly: there is not a single feature point on a white wall, every part of what the camera sees looks the same, and the system has no way to judge whether it has moved or how far. Along with the IMU's need for motion excitation at initialization in Chapter 7 and LiDAR's fear of glass in Chapter 8, this is one of SLAM's three classic environmental traps.

The remedies echo the fusion thinking of the whole book: add an IMU and dead-reckon through the few seconds in which vision has lost track (Chapter 7); or switch to LiDAR, for which a wall is instead a crisp plane; or project texture (the active-infrared approach of RGB-D). The teaching value of this case is that it lets students feel directly

how a sensor's limitation becomes a system's failure—understanding the limitations matters more than admiring the capabilities.

Hands-On Activities

Exercise 9-1 Camera Calibration and a First Experience of Visual Depth

Purpose: to carry out a camera calibration by hand, observe the effect of distortion correction, and experience feature point detection and stereo depth measurement. This exercise can be done with an ordinary webcam or a phone and is a warm-up for the visual SLAM exercises of Chapter 30.

Equipment

- A webcam or phone camera, and a computer (with a Python and OpenCV environment).
- A printed checkerboard calibration target (downloadable from the OpenCV site; mount it on a rigid board to keep it flat).
- (Optional) A stereo or RGB-D camera (a RealSense, say) for the advanced depth work.

Part One: Calibration and Undistortion

1. Photograph the checkerboard 15 to 20 times from different angles and distances, covering as much of the frame and as many tilts as possible.
2. Use OpenCV's findChessboardCorners to detect the corners and calibrateCamera to obtain the intrinsic matrix K and the distortion coefficients, and record them.
3. Examine the K matrix: are fx and fy close to each other? Are cx and cy about half the image width and height? Consider the physical meaning of these numbers.

4. Use the recovered parameters to undistort an image containing straight lines (a brick wall or a bookshelf, say), place the before and after side by side, and observe whether the curved lines near the edges have been straightened.
5. Check whether the reprojection error is below 1 pixel; if it is too large, add more calibration images or reshoot and try again.

Part Two: Feature Point Detection

6. Using OpenCV's ORB detector, extract and draw feature points on an image of the classroom or a desktop, and observe where they concentrate (corners, richly textured areas) and where they are absent (white walls, plain-colored desktops).
7. Photograph the same scene twice (moving the camera slightly), find corresponding points by feature matching and draw the connecting lines, and observe which matches are correct and which are wrong.
8. Deliberately photograph a large expanse of white wall and observe that ORB finds almost no feature points—verifying "the curse of the white wall" first-hand (Case 9-3).

Part Three: Experiencing Depth (choose according to equipment)

9. With an RGB-D camera: read and display the depth map (far and near rendered as a color scale), point the camera at a desktop and then into the distance and observe how the depth values change; then point it at glass or a mirror and observe the missing or erroneous depth data.
10. With a monocular camera only: photograph an object of known size (a sheet of A4 paper, say) at various distances, record its size in pixels in the image, and

verify the pinhole projection relation that doubling the distance halves the pixel size (Section 9.1).

Discussion Questions

1. How pronounced is the change in the straight lines near the image edges before and after undistortion? If you ran visual SLAM without undistorting first, what problems would you expect in the map?
2. Where in your classroom do the ORB feature points concentrate? Which areas have almost none? What does this suggest about where a robot should look so as not to lose track?
3. (With RGB-D) What went wrong with the depth data on glass and mirrors? Is this the same as the weakness of LiDAR in Chapter 8? Explain the common cause behind both.
4. Based on Table 9-1 of this chapter, would you choose a monocular, stereo, or RGB-D camera for an indoor service robot? Would you pair it with an IMU? Give your reasons.

Extended Challenge (Undergraduate)

First: with the calibrated camera, implement a program that estimates the camera pose from an ArUco marker (echoing Chapter 4) and displays the camera's 6-DoF pose relative to the marker in real time—joining Chapter 4 to this one. Second: if a stereo camera is available, implement disparity computation (OpenCV's StereoBM or StereoSGBM), produce a depth map, and use the formulation of Section 9.6 to verify the depth accuracy for an object at a known distance, observing whether the error grows with the square of distance. This program leads directly into Chapter 30.

Further Reading

1. R. Hartley and A. Zisserman, Multiple View Geometry in Computer Vision, 2nd ed., Cambridge University Press, 2004. (The authoritative bible of multiple-view geometry and the principal reference for Chapter 21.)
2. R. Szeliski, Computer Vision: Algorithms and Applications, 2nd ed., Springer, 2022. (The standard textbook of computer vision, freely available, with complete chapters on camera models and calibration.)
3. Z. Zhang, "A Flexible New Technique for Camera Calibration," IEEE TPAMI, 2000. (The original paper on Zhang's checkerboard calibration and the basis of Section 9.2.)
4. The official OpenCV tutorial on Camera Calibration and 3D Reconstruction (docs.opencv.org). (With complete code examples; the first choice for implementation.)
5. L. Keselman et al., "Intel RealSense Stereoscopic Depth Cameras," CVPR Workshops, 2017. (The principles and specifications of the RealSense depth camera family and the basis of Section 9.3.)
6. Xiang Gao and Tao Zhang, Fourteen Lectures on Visual SLAM, Publishing House of Electronics Industry. (The most popular introduction to visual SLAM in the Chinese-speaking world, covering the complete line from camera models to SLAM.)
7. UZH Robotics and Perception Group, the course Vision Algorithms for Mobile Robotics (introduced in Chapter 2). (A complete free course from image formation to visual SLAM.)

The Close of Part II: From Sensors to Mathematics

Part II ends here. Over six chapters the robot's senses have all been assembled, and you will have noticed a structure that recurs: every sensor has the situations it excels in and the situations in which it must fail, and the failure situations of different sensors happen not to overlap.

Indoor positioning needs installed infrastructure, GNSS needs the sky, the magnetometer fears rebar, the IMU drifts, LiDAR has no semantics and degenerates in corridors, and the camera has no scale and fears darkness and white walls. Lay them out in a table and one fact is clear: no single sensor can accomplish navigation alone, but their combination can.

The question then becomes: when five sensors give mutually contradictory readings at once, which should the robot believe? The answer of Chapter 11 is probability—every reading carries its own uncertainty, and the result of fusion is the estimate with the least uncertainty. But before these readings can be compared within one expression, they must first be expressed in one coordinate frame and position and orientation must be described by one set of mathematics.

That is the work of the next chapter. Chapter 10 introduces coordinate systems, the various representations of rotation (rotation matrices, Euler angles, quaternions, Lie groups), and rigid-body motion—it is the most mathematically dense chapter in the book and the toolbox every chapter thereafter uses. Read it slowly if need be, but do read it until you understand it.

Summary

1. The pinhole camera model describes how a three-dimensional point projects into a two-dimensional image; the denominator of the projection formula is

the depth Z, meaning that depth is divided away in imaging—the geometric root of scale ambiguity in visual SLAM. The intrinsic matrix K is the most basic object in visual SLAM.

2. Real lenses have radial and tangential distortion, which bends straight lines and violates the pinhole assumption; camera calibration (Zhang's checkerboard method, for instance) measures the intrinsics K and the distortion coefficients, and undistortion is the first step of all visual SLAM.
3. The three kinds of camera obtain depth in different ways: monocular triangulates from motion but has no absolute scale; stereo uses left-right disparity and has absolute scale but fears untextured surfaces; RGB-D projects actively to obtain dense depth but fears strong light and glass.
4. Visual SLAM extracts three kinds of information from an image: feature points (corners, used as landmarks), edges (outlines), and optical flow (pixel motion); monocular depth estimation by deep learning is a newly emerging complement. Modern systems commonly mix them.
5. The camera is richest in information, lowest in cost, passive, frugal with power, and light; but it has scale ambiguity, depends heavily on light and texture (failing in darkness, against white walls, and under blur), carries a heavy computational load, and fears dynamic scenes. Its weaknesses are nearly complementary to LiDAR's.
6. The ladder of knowledge runs: image processing → feature extraction → multiple-view geometry → visual odometry → visual SLAM; visual SLAM divides into the feature-based school (ORB-SLAM) and the direct school (DSO).
7. The camera carries semantics and is the window onto semantic navigation: AI models such as YOLO, SAM, Depth Anything, and CLIP, together with VLMs, let

a robot advance from knowing shapes to understanding a scene and understanding what people say—the subject of Chapter 32.

Part II, "Sensors for Localization and Navigation," is now complete—we have met all of the robot's senses. But what every sensor gives is raw data carrying noise: odometry drifts, LiDAR has stray points, the camera mismatches features. The next part, "The Mathematical Foundations of SLAM," answers a fundamental question: faced with these imperfect measurements, how does a robot make the most reasonable estimate mathematically? We will begin with coordinate frames and rigid-body motion and build the mathematical tools for handling uncertainty step by step.